

Podman Zero-to-Expert Course

2026-06-07

Contents

1	Front Matter	9
1.1	README.md	9
2	Podman Zero-to-Expert Course (Draft)	9
2.1	Table of Contents	9
2.2	How To Use This Repo	9
2.3	Structure	9
2.4	Safety	10
2.5	Conventions	10
2.6	Suggested Pacing	10
3	License	10
3.1	COURSE_OUTLINE.md	11
4	Course Outline	11
4.1	Table of Contents	11
4.2	Modules	11
4.3	Suggested Pacing (Rough Estimates)	11
4.4	MODULES.md	13
5	Modules (Reading Order)	13
5.1	Table of Contents	13
5.2	Reading Order	13
5.3	Quick References	13
6	Modules	14
7	Module 0: Setup (Fedora/RHEL + systemd)	15
7.1	Table of Contents	15
7.2	Learning Goals	15
7.3	Why Fedora/RHEL and Why Rootless	15
7.4	Install	16
7.5	cgroups v2 Check (Required)	16
7.6	Rootless Prereqs	16
7.7	Understanding the User Namespace Setup	17
7.8	SELinux Quick Check (Fedora/RHEL)	17
7.9	First Container	18
7.10	Where Things Live (Rootless)	18
7.11	Create a Course Workspace	19
7.12	Recommended Shell Safety	19
7.13	Linger — Boot Safety for User Services	19

7.14	Version Matrix and Compatibility Notes	19
7.15	Checkpoint	20
7.16	Quick Quiz	20
7.17	Further Reading	20
8	Module 1: Containers 101	21
8.1	Table of Contents	21
8.2	Learning Goals	21
8.3	What Is a Container, Really?	21
8.4	The Three Things To Get Right	22
8.5	How podman run Works — Step by Step	22
8.6	Containers Are Not VMs	23
8.7	Linux Namespaces — The Isolation Mechanism	24
8.8	cgroups — The Resource Accounting Mechanism	25
8.9	The OCI Standards	25
8.10	Rootless vs Rootful	26
8.11	Terminology You Will See	26
8.12	Lab: Compare Host vs Container	27
8.13	Lab: Namespace Exploration	28
8.14	Checkpoint	29
8.15	Quick Quiz (Answer Without Running Commands)	29
8.16	Further Reading	29
9	Module 2: Everyday Podman Commands	30
9.1	Table of Contents	30
9.2	Learning Goals	30
9.3	The Container Lifecycle	30
9.4	Core Commands	31
9.5	Useful Flags (Learn These Early)	31
9.6	Inspecting and Debugging	32
9.7	Lab: The Writable Layer Is Not Persistence	32
9.8	Lab: Name Your Containers	33
9.9	Lab: Exit Codes and Restart Behavior	33
9.10	Formatting Output	34
9.11	Cleaning Up Without Nuking Your System	35
9.12	Checkpoint	35
9.13	Quick Quiz	35
9.14	Further Reading	36
10	Module 3: Images and Registries	37
10.1	Table of Contents	37
10.2	Learning Goals	37
10.3	What Is an Image	37
10.4	Tags vs Digests	38
10.5	Fully Qualified Image Names	38
10.6	Short-Name Resolution and registries.conf	39
10.7	Lab: Pull by Tag, Record Digest	39
10.8	Registry Authentication	39
10.9	Image Metadata: ENTRYPOINT vs CMD	40
10.10	Inspecting Image Layers	40
10.11	Lab: Local Tagging	41
10.12	Lab (Optional): Push to a Local Registry	41
10.13	Lab (Optional): Save and Load	42
10.14	Image Storage on Disk	42

10.15	Checkpoint	42
10.16	Quick Quiz	42
10.17	Further Reading	43
11	Module 4: Secrets (Local-First)	44
11.1	Table of Contents	44
11.2	Learning Goals	44
11.3	Why Secrets Matter — The Threat Model	44
11.4	Mental Model: Secret as a Mounted File	45
11.5	How Podman Secrets Work Internally	45
11.6	Common Anti-Patterns	46
11.7	Commands Reference	46
11.8	Lab A: Create and Mount a Secret	47
11.9	Lab A2: Prove It Is Not an Env Var	47
11.10	Lab B: Rotation Pattern (Versioned Secrets)	48
11.11	Lab C: Custom Mount Target	49
11.12	Advanced: Build-Time Secrets (Optional)	49
11.13	File Permissions and App Compatibility	50
11.14	What Podman Secrets Do NOT Solve	51
11.15	Common Failure Modes	51
11.16	Checkpoint	51
11.17	Quick Quiz	51
11.18	Further Reading	52
12	Module 5: Storage (Volumes, Bind Mounts, Permissions)	53
12.1	Table of Contents	53
12.2	Learning Goals	53
12.3	The Three Storage Types	53
12.4	Volumes	54
12.5	Bind Mounts	54
12.6	tmpfs Mounts	55
12.7	Inspect Mounts	55
12.8	Rootless Permission Pitfalls	56
12.9	podman unshare — Your Permission Debugging Tool	56
12.10	SELinux Drill (Fedora/RHEL)	57
12.11	Lab: Persistent DB Data	57
12.12	Volume Backup and Restore Pattern	58
12.13	Checkpoint	59
12.14	Quick Quiz	59
12.15	Further Reading	59
13	Module 6: Networking (Ports, DNS, User-Defined Networks)	60
13.1	Table of Contents	60
13.2	Learning Goals	60
13.3	Minimum Path (If You Are Short on Time)	60
13.4	1 How Container Networking Works (Mental Model)	61
13.5	2 Rootless Networking In Depth	62
13.6	3 Port Publishing	63
13.7	4 The Default Network vs User-Defined Networks	64
13.8	5 Container DNS and Service Discovery	65
13.9	6 Connecting Containers to Multiple Networks	66
13.10	7 Inspecting Network State	67
13.11	8 Network Drivers — Deeper Look	68
13.12	9 Network Security Patterns	69

13.1310 Full Lab: Three-Tier Isolated Stack	70
13.1411 Connecting Containers to Pods on a Network	71
13.1512 Networking in Quadlet (systemd) Deployments	71
13.1613 Troubleshooting Networking	72
13.1714 Common Patterns Reference	74
13.18Checkpoint	74
13.19Quick Quiz	74
13.20Further Reading	75
14 Module 7: Pods and Sidecars	76
14.1 Table of Contents	76
14.2 Learning Goals	76
14.3 What Is a Pod?	76
14.4 Why Pods — The Use Cases	77
14.5 The Infra Container	77
14.6 Pod vs User-Defined Network — When To Use Each	78
14.7 The Sidecar Pattern	78
14.8 Pod Lifecycle and Commands	79
14.9 Lab: Two Containers, One Pod	79
14.10Lab: Log-Shipping Sidecar Pattern	80
14.11Pods and Quadlet	81
14.12Pods vs Kubernetes Pods	81
14.13Pod Lifecycle Notes	82
14.14Checkpoint	82
14.15Quick Quiz	82
14.16Further Reading	82
15 Module 8: Building Images (Containerfiles)	84
15.1 Table of Contents	84
15.2 Learning Goals	84
15.3 Minimum Path (If You Are Short on Time)	85
15.4 1 Images, Layers, and the Build Mental Model	85
15.5 2 <code>podman build</code> Fundamentals	85
15.6 3 Containerfile Instructions: The Practical Subset	86
15.7 4 Lab A: Build a Tiny HTTP Image (Warm-Up)	87
15.8 5 Build Context Hygiene (The Most Common Image Leak)	88
15.9 6 Running as Non-Root (Image-Level Least Privilege)	89
15.107 Multi-Stage Builds (Small Images, Fast Builds)	90
15.118 Caching: Make Rebuilds Fast	92
15.129 <code>ARG</code> , <code>ENV</code> , and Configuration	92
15.1310 Secrets and Private Dependencies (Build-Time)	93
15.1411 Labels, Metadata, and Image Introspection	93
15.1512 Tagging, Digests, and Promotion	94
15.1613 Pushing Images to a Registry	94
15.1714 Testing the Image You Built	95
15.1815 Troubleshooting Builds (Common Failures)	96
15.1916 Cleanup: Keep Your Machine Healthy	97
15.2017 Extended Lab: A Small “Real” Service Image	97
15.21Checkpoint	99
15.22Quick Quiz	99
15.23Further Reading	99
16 Module 9: Multi-Service Workflows	100
16.1 Table of Contents	100

16.2	Learning Goals	100
16.3	The Multi-Service Problem	100
16.4	Three Patterns Compared	101
16.5	Pattern 1: User-Defined Network + Separate Containers	101
16.6	Pattern 2: Pod (Shared localhost)	101
16.7	Pattern 3: podman play kube	102
16.8	Lab: A Two-Service Stack (MariaDB + Adminer)	102
16.9	Make It Repeatable (Script)	103
16.10	Idempotency — What It Means and Why It Matters	104
16.11	From Script to Quadlet	104
16.12	Compose-ish Tooling (Context)	105
16.13	Checkpoint	105
16.14	Quick Quiz	105
16.15	Further Reading	106
17	Module 10: podman play kube	107
17.1	Table of Contents	107
17.2	Learning Goals	107
17.3	What Is podman play kube?	107
17.4	What YAML Resources Are Supported?	108
17.5	The YAML-to-Podman Mapping	108
17.6	Lab: Run a Pod from YAML	109
17.7	Examining the Generated Resources	110
17.8	Generating YAML from Existing Containers	110
17.9	Production Note: Quadlet .kube Units	111
17.10	Secrets Note — Base64 Is Not Encryption	111
17.11	Limitations To Know	112
17.12	Teardown — Always Use podman kube down	112
17.13	Checkpoint	113
17.14	Quick Quiz	113
17.15	Further Reading	113
18	Module 11: Production Baseline (systemd + Quadlet)	114
18.1	Table of Contents	114
18.2	Learning Goals	114
18.3	Why Quadlet	114
18.4	Mental Model: Quadlet as a Generator	115
18.5	Boot Safety (Rootless)	115
18.6	Where Quadlet Files Live	115
18.7	Unit File Types	116
18.8	Anatomy of a .container Unit	116
18.9	How “Enable” Works for Quadlet	117
18.10	Helpful Podman Commands	118
18.11	Lab: Your First Quadlet Container	118
18.12	Lab: Pre-Create a Network and Volume (Quadlet)	118
18.13	Debugging Quadlet Syntax	119
18.14	Dependencies Between Quadlets	120
18.15	Upgrades and Rollback	120
18.16	Restart Policies	121
18.17	Secrets	121
18.18	Checkpoint	122
18.19	Quick Quiz	122
18.20	Further Reading	122

19	Module 11a: Secrets with Quadlet + systemd (Rootless)	123
19.1	Table of Contents	123
19.2	Learning Goals	123
19.3	Why This Matters at the systemd Level	123
19.4	Recommended Pattern	124
19.5	Where Secrets Are Stored	124
19.6	The Three Levels of “Not in the Unit File”	124
19.7	Lab: Quadlet Unit Consuming a Secret	125
19.8	Reading the Secret Inside Your App	126
19.9	Rotation	126
19.10	Multi-Secret Services	127
19.11	systemd Credentials as an Alternative	127
19.12	What This Does NOT Solve	128
19.13	Checkpoint	128
19.14	Quick Quiz	128
19.15	Further Reading	128
20	Module 12: Security Deep Dive	130
20.1	Table of Contents	130
20.2	Learning Goals	130
20.3	The Defence-in-Depth Model	130
20.4	Baseline Hardening Checklist	131
20.5	Linux Capabilities — What They Are	131
20.6	Lab: Drop Capabilities	132
20.7	Resource Limits — Why They Matter	132
20.8	Lab: Resource Limits	133
20.9	Lab: No-New-Privileges	133
20.10	Lab: Read-Only Root FS	134
20.11	User Namespace and Non-Root Inside Container	135
20.12	SELinux (Fedora/RHEL)	135
20.13	Secomp Profiles	136
20.14	Image Trust (Practical)	136
20.15	Putting It Together: Hardened Run Template	137
20.16	Checkpoint	138
20.17	Quick Quiz	138
20.18	Further Reading	138
21	Module 13: Troubleshooting and Ops	139
21.1	Table of Contents	139
21.2	Learning Goals	139
21.3	Minimum Path (If You Are Short on Time)	139
21.4	1 The Debug Loop (Mental Model)	140
21.5	2 Container State and Lifecycle Commands	140
21.6	3 Reading Logs	141
21.7	4 Deep Inspection with <code>podman inspect</code>	142
21.8	5 Interactive Debugging	142
21.9	6 Events and Timeline	143
21.10	7 Resource Monitoring	143
21.11	8 Networking Troubleshooting	144
21.12	9 Storage Troubleshooting	145
21.13	10 systemd and Quadlet Troubleshooting	145
21.14	11 SELinux Troubleshooting	146
21.15	12 Failure Drills (Do These in Practice)	147
21.16	13 Recovery Playbooks	148

21.17	Checkpoint	149
21.18	Quick Quiz	149
21.19	Further Reading	150
22	Module 14: Maintenance and Auto-Updates	151
22.1	Table of Contents	151
22.2	Learning Goals	151
22.3	What Auto-Update Is	151
22.4	How It Works — Internals	152
22.5	Tags, Digests, and Policy	152
22.6	The io.containers.autoupdate Label	153
22.7	Lab: Enable Registry Auto-Update (Single Service)	153
22.8	Healthchecks and Auto-Rollback	154
22.9	Automating Auto-Update with a systemd Timer	155
22.10	Rollback Plan (Required If You Auto-Update)	155
22.11	Safe Rollout Rules	156
22.12	Routine Maintenance Tasks	156
22.13	Checkpoint	157
22.14	Quick Quiz	157
22.15	Further Reading	157
23	Capstone: Reboot-Safe Local Stack with Secrets, Backups, and Upgrades	158
23.1	Table of Contents	158
23.2	Goal	158
23.3	Reference Stack	158
23.4	Deliverables	159
23.5	Architecture Overview	159
23.6	Build It	159
23.7	First Data (Required)	161
23.8	Optional: Scheduled Backups	161
23.9	Backup and Restore (Required)	161
23.10	Upgrade and Rollback (Required)	163
23.11	Password Rotation (Required)	164
23.12	Operations Runbook	165
23.13	Notes	165
23.14	Checkpoint	166
23.15	Quick Quiz	166
23.16	Further Reading	166
24	External Secrets Survey (Thorough Intro, Optional Implementation)	167
24.1	Table of Contents	167
24.2	Why This Module Exists	167
24.3	What You Are Optimizing For	167
24.4	The Decision Tree	168
24.5	Option 1: systemd Credentials (Host-Native)	168
24.6	Option 2: SOPS (GitOps-Friendly Encrypted Files)	169
24.7	Option 3: Vault-Class Secret Managers (Centralized)	170
24.8	Comparison Table	171
24.9	What Does Not Change	172
24.10	Migration Path from Podman Secrets	172
24.11	Anti-Patterns to Avoid	172
24.12	Checkpoint	173
24.13	Quick Quiz	173
24.14	Further Reading	173

25 Cheatsheets	174
25.1 podman-cli.md	175
26 Podman CLI Cheat Sheet	175
26.1 Table of Contents	175
26.2 Containers	175
26.3 Images	175
26.4 Port Publishing	175
26.5 Networks	176
26.6 Volumes	176
26.7 Pods	176
26.8 Secrets	176
26.9 quadlet.md	178
27 Quadlet Cheat Sheet	178
27.1 Table of Contents	178
27.2 Files	178
27.3 Workflow	178
27.4 Boot Start	178
27.5 rootless.md	179
28 Rootless Cheat Sheet	179
28.1 Table of Contents	179
28.2 Key Idea	179
28.3 Useful Checks	179
28.4 Common Paths	179
28.5 Boot Start for systemd User Services	179
28.6 security.md	180
29 Security Cheat Sheet	180
29.1 Table of Contents	180
29.2 Baseline	180
29.3 Hardening Flags (Examples)	180
29.4 SELinux (Fedora/RHEL)	180
29.5 troubleshooting.md	181
30 Troubleshooting Cheat Sheet	181
30.1 Table of Contents	181
30.2 Fast Triage	181
30.3 systemd/Quadlet	181
30.4 Network Checks	181
30.5 Storage Checks	181
31 Appendix	182
31.1 ASSESSMENTS.md	183
32 Assessments	183
32.1 Table of Contents	183
32.2 Module Checkpoints	183
32.3 Practical Exam A (Mid-Course)	183
32.4 Practical Exam B (Final)	183
32.5 GLOSSARY.md	185
33 Glossary	185
33.1 Table of Contents	185

33.2 Terms	185
33.3 FAQ.md	186
34 FAQ / Gotchas	186
34.1 Table of Contents	186
34.2 Rootless: “permission denied” publishing port 80	186
34.3 Container name DNS does not work	186
34.4 SELinux: bind mounts fail with permission denied (Fedora/RHEL)	186
34.5 Local registry lab fails with TLS/HTTPS errors	187
34.6 HEALTHCHECK missing after build	187
34.7 <code>exec format error</code>	187
34.8 Quadlet service does not exist after adding a file	187

1 Front Matter

1.1 README.md

2 Podman Zero-to-Expert Course (Draft)

This is a course-in-a-repo for taking a learner from zero container knowledge to running rootless Podman services with systemd (Quadlet), with strong security and troubleshooting fundamentals.



2.1 Table of Contents

- How To Use This Repo
- Structure
- Safety
- Conventions
- Suggested Pacing
- License

2.2 How To Use This Repo

- Read the modules in `modules/` in order.
- Do the labs as you go; each module includes a small checklist.
- Keep everything rootless unless the module explicitly says otherwise.

[^] Go to TOC

2.3 Structure

- `COURSE_OUTLINE.md`: the full syllabus and learning goals
- `MODULES.md`: reading order
- `modules/`: lesson content (Markdown)
- `cheatsheets/`: quick references (see below)
- `examples/`: example YAML and unit files
- `ASSESSMENTS.md`: practical exams and rubrics
- `FAQ.md`: common gotchas and fast fixes

Module numbering:

- 00–14: the core sequence, read in order.
- 11a: an add-on to Module 11 (secrets with Quadlet + systemd).

- 80: the capstone project; 90: an optional elective/survey. The gaps are intentional — they separate the core sequence from the capstone and electives.

Suggested path:

- Start with `modules/00-setup.md`
- Continue in numeric order

Cheatsheets (use alongside the modules, and as a post-course reference):

- `cheatsheets/podman-cli.md`: everyday `podman` commands (pairs with Modules 02–03)
- `cheatsheets/rootless.md`: rootless-specific paths, ranges, and gotchas (Modules 00–01, 05–06)
- `cheatsheets/quadlet.md`: Quadlet unit keys and `systemd` workflow (Modules 11/11a, 14)
- `cheatsheets/security.md`: hardening flags and SELinux labels (Module 12)
- `cheatsheets/troubleshooting.md`: symptom -> diagnosis -> fix tables (Module 13)

^ Go to TOC

2.4 Safety

- Prefer rootless Podman.
- Never put secret material in images, unit files, or logs.
- If you are on a shared system, treat this repo's lab values as examples only.

^ Go to TOC

2.5 Conventions

- Secrets are delivered as files mounted at runtime (not environment variables).
- Production baseline uses rootless Podman + `systemd` user services (Quadlet-first).

^ Go to TOC

2.6 Suggested Pacing

See `COURSE_OUTLINE.md` for rough time estimates per module.

^ Go to TOC

3 License

This project is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0).

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

3.1 COURSE_OUTLINE.md

4 Course Outline



Goal: take an absolute beginner to an operator who can build, run, secure, and troubleshoot rootless Podman workloads, including systemd (Quadlet) production patterns.

4.1 Table of Contents

- Modules
- Suggested Pacing (Rough Estimates)

Assumptions:

- Learners have basic command line familiarity by the end of Module 2.
- Labs target Fedora/RHEL-like systems with systemd. Where commands differ across distros, modules call it out.

[^] Go to TOC

4.2 Modules

0. Setup (install, rootless prerequisites, verification)
1. Containers 101 (images vs containers, OCI, rootless)
2. Everyday Podman commands (**run**, **exec**, **logs**, lifecycle)
3. Images and registries (tags vs digests, inspect, provenance basics)
4. Secrets (local-first with Podman secrets; rotation patterns)
5. Storage basics (volumes, bind mounts, permissions, SELinux notes)
6. Networking (ports, DNS, user-defined networks)
7. Pods and sidecars (Podman pods, shared network namespace)
8. Building images (Containerfiles, multi-stage, non-root)
9. Multi-service workflows (scripted, compose-ish patterns, **play kube** preview)
10. **podman play kube** (Kubernetes YAML locally, parity concepts; YAML secrets caveats)
11. Production baseline: systemd + Quadlet (rootless services, restart, upgrades; secrets add-on) 11a. Quadlet secrets add-on (runtime file secrets + rotation)
12. Security deep dive (capabilities, seccomp/SELinux, read-only FS)
13. Troubleshooting and ops (events, logs, journald, failure drills)
14. Maintenance and auto-updates (policy, **podman auto-update**, safe rollouts)
15. Capstone (Quadlet stack with secrets, backups, upgrades, rollback)
16. External secrets survey (thorough intro; optional implementation paths)

[^] Go to TOC

4.3 Suggested Pacing (Rough Estimates)

These are rough time boxes for a first pass (reading + doing the labs).

- Module 0: 30-60 min
- Module 1: 30-45 min
- Module 2: 60-90 min
- Module 3: 60-90 min
- Module 4: 60-90 min
- Module 5: 60-90 min
- Module 6: 2-3 hours
- Module 7: 45-60 min

- Module 8: 2-3 hours
- Module 9: 60-90 min
- Module 10: 45-75 min
- Module 11: 2-3 hours
- Module 11a: 45-75 min
- Module 12: 60-120 min
- Module 13: 60-120 min
- Module 14: 45-75 min
- Module 80 (Capstone): 3-6 hours
- Module 90 (Survey): 45-90 min

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

4.4 MODULES.md

5 Modules (Reading Order)



5.1 Table of Contents

- Reading Order
- Quick References

5.2 Reading Order

Numbering: 00–14 form the core sequence; 11a is an add-on to Module 11; 80 is the capstone and 90 an optional elective — the gaps before them are intentional.

- `modules/00-setup.md`
- `modules/01-containers-101.md`
- `modules/02-everyday-commands.md`
- `modules/03-images-registries.md`
- `modules/04-secrets.md`
- `modules/05-storage.md`
- `modules/06-networking.md`
- `modules/07-pods.md`
- `modules/08-building-images.md`
- `modules/09-multiservice-workflows.md`
- `modules/10-play-kube.md`
- `modules/11-quadlet.md`
- `modules/11a-quadlet-secrets.md`
- `modules/12-security.md`
- `modules/13-troubleshooting.md`
- `modules/14-autoupdate.md`
- `modules/80-capstone.md`
- `modules/90-external-secrets-survey.md`

[^ Go to TOC](#)

5.3 Quick References

Cheatsheets to keep open while doing the labs:

- `cheatsheets/podman-cli.md` — everyday `podman` commands
- `cheatsheets/rootless.md` — rootless paths, ID ranges, gotchas
- `cheatsheets/quadlet.md` — Quadlet unit keys and `systemd` workflow
- `cheatsheets/security.md` — hardening flags and SELinux labels
- `cheatsheets/troubleshooting.md` — symptom -> diagnosis -> fix

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

6 Modules

7 Module 0: Setup (Fedora/RHEL + systemd)



7.1 Table of Contents

- Learning Goals
- Why Fedora/RHEL and Why Rootless
- Install
- cgroups v2 Check (Required)
- Rootless Prereqs
- Understanding the User Namespace Setup
- SELinux Quick Check (Fedora/RHEL)
- First Container
- Where Things Live (Rootless)
- Create a Course Workspace
- Recommended Shell Safety
- Linger — Boot Safety for User Services
- Version Matrix and Compatibility Notes
- Checkpoint
- Quick Quiz
- Further Reading

This course targets Fedora/RHEL-like systems with systemd, using rootless Podman.

[^ Go to TOC](#)

7.2 Learning Goals

- Install Podman and verify basic functionality.
- Confirm your system supports rootless containers (subuid/subgid).
- Know where logs and state live for rootless Podman.
- Confirm cgroups v2 is enabled (required for Quadlet).
- Set up a lab workspace and a few safety defaults.
- Enable lingering for boot-safe user services.

[^ Go to TOC](#)

7.3 Why Fedora/RHEL and Why Rootless

This course makes three deliberate choices:

1. **Fedora/RHEL** — because they ship a production-grade Podman, cgroups v2 by default, and SELinux enforcement. The security model is coherent out of the box.
2. **Rootless Podman** — because running containers as an unprivileged user means a container escape does not give the attacker root on the host. It is the right default for servers and developer machines alike.
3. **systemd (Quadlet)** — because systemd is already the init system on RHEL/Fedora. Using it for container lifecycle means you get restart policies, log aggregation, and boot integration for free, without a container daemon running as root.

graph TD

```
A["Traditional container model"] --> B["Container daemon<br/>(Docker) running as root"]
B --> C["All containers managed<br/>by root-owned process"]
C --> D["Container escape = root on host"]
```

```
E["Podman rootless model"] --> F["No persistent daemon<br/>Each user runs own containers"]
F --> G["Containers owned by your UID<br/>via user namespace"]
G --> H["Container escape =<br/>unprivileged host user only"]
```

```
style D fill:#3d0000,color:#ffcccc
```

```
style H fill:#003d00,color:#ccffcc
```

^ Go to TOC

7.4 Install

Fedora:

```
sudo dnf install -y podman # install Podman package
```

RHEL 10 (package availability depends on subscription/repos):

```
sudo dnf install -y podman # install Podman package
```

Verify the installation:

```
podman --version # print Podman version
```

```
podman info # print Podman host configuration (storage, runtime, cgroups)
```

Record your exact versions — useful for debugging and for searching release notes:

```
podman --version # Podman version string
```

```
rpm -q podman 2>/dev/null || true # RPM package version
```

```
rpm -q crun 2>/dev/null || true # OCI runtime version
```

```
uname -r # kernel version
```

Minimum versions for this course: - Podman >= 4.4 (for Quadlet support) - cgroups v2 (kernel >= 5.2, all RHEL 9/10, Fedora 31+)

^ Go to TOC

7.5 cgroups v2 Check (Required)

Quadlet requires cgroups v2. Without it, `systemctl --user daemon-reload` generates units that may fail to enforce resource limits.

Check:

```
podman info --format '{{.Host.CgroupsVersion}}' # expected: v2
```

Expected output: v2

If you see v1: - On RHEL 8 or older kernels, cgroups v2 may not be the default. - You can enable it: add `systemd.unified_cgroup_hierarchy=1` to the kernel command line in GRUB and reboot. - For this course, upgrading to RHEL 9/10 or Fedora >= 31 is the easier path.

Verify at the kernel level:

```
stat -fc %T /sys/fs/cgroup # should print: cgroup2fs
```

^ Go to TOC

7.6 Rootless Prereqs

Rootless Podman uses Linux **user namespaces** to map UIDs inside containers to unprivileged UIDs on the host. This requires `/etc/subuid` and `/etc/subgid` entries for your user.

Check your subuids/subgids:

```
grep "^$USER:" /etc/subuid /etc/subgid # show subuid and subgid entries for current user
```

Expected output (example):

```
/etc/subuid:myuser:100000:65536
/etc/subgid:myuser:100000:65536
```

If those are missing, create them (coordinate the range with your admin policy):

```
sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 "$USER" # grant subuid/subgid range
```

Log out and back in after updating subuids/subgids — the kernel only reads these at session start.

Verify the mapping is active:

```
podman unshare cat /proc/self/uid_map # show UID mapping inside user namespace
```

^ Go to TOC

7.7 Understanding the User Namespace Setup

When you run `podman run` as a non-root user:

```
graph LR
    subgraph "Inside container"
        C1["root (uid 0)"]
        C2["app user (uid 1001)"]
    end
    subgraph "Host kernel view"
        H1["uid 100000 (your subuid start)"]
        H2["uid 101001"]
    end
    C1 -->|"mapped to"| H1
    C2 -->|"mapped to"| H2
```

The container's root (uid 0) is mapped to your first subuid (e.g., 100000) on the host. This means: - The container process has “root” privileges inside its namespace. - On the host, it runs as an unprivileged user (uid 100000). - Even if the container escapes, the attacker has only uid 100000 — not real root.

This is why `/etc/subuid` and `/etc/subgid` are security-critical configuration, not just administrative overhead.

^ Go to TOC

7.8 SELinux Quick Check (Fedora/RHEL)

SELinux is usually enforcing on RHEL/Fedora. You do not need to understand the full policy model now, but you should be able to recognize when it affects your containers.

Check SELinux mode:

```
getenforce # print SELinux mode: Enforcing, Permissive, or Disabled
```

What this means for containers:

Mode	Effect on containers
Enforcing	SELinux labels are checked. Bind mounts need <code>:Z</code> or <code>:z</code> .
Permissive	SELinux denials are logged but not enforced. Containers work but you miss real denials.

Mode	Effect on containers
Disabled	No SELinux. Not recommended in production.

Golden rules for this course:

- Prefer **named volumes** over bind mounts — volumes are automatically labelled correctly.
- If you must use a bind mount, always add `:Z` (private) or `:z` (shared).
- Never disable SELinux to fix a container issue.

If you see `permission denied` errors that seem wrong, check for SELinux denials:

```
ausearch -m avc -ts recent 2>/dev/null || journalctl -b -t kernel -g denied # check SELinux denials
```

^ Go to TOC

7.9 First Container

Run a simple container to verify everything is working:

```
podman run --rm docker.io/library/alpine:latest uname -a # run container, print kernel info
```

If this works, you have:

- Network access to pull images from Docker Hub.
- A working storage backend (`overlay` or `vfs`).
- A working OCI runtime (`crun` on RHEL/Fedora).
- A working user namespace setup.

Verify the storage driver in use:

```
podman info --format '{{.Store.GraphDriverName}}' # should print: overlay
```

`overlay` is the preferred driver. `vfs` works but is slower and uses more disk space.

Verify the OCI runtime:

```
podman info --format '{{.Host.OCIRuntime.Name}}' # should print: crun
```

^ Go to TOC

7.10 Where Things Live (Rootless)

Understanding the directory layout helps you debug storage problems and know what to back up.

Purpose	Path
Container image storage	<code>~/.local/share/containers/storage/</code>
Container runtime files	<code>/run/user/<uid>/containers/</code>
Quadlet unit files	<code>~/.config/containers/systemd/</code>
Podman secrets store	<code>~/.local/share/containers/storage/secrets/</code>
Podman config	<code>~/.config/containers/</code>
Registry auth cache	<code>\${XDG_RUNTIME_DIR}/containers/auth.json</code>

Logs:

```
podman logs <name> # container stdout/stderr logs
journalctl --user -u <service>.service # Quadlet/systemd-managed container logs
journalctl --user -b -n 100 --no-pager # all user-session log entries this boot
```

Check disk usage for container storage:

```
podman system df # show disk usage: images, containers, volumes
```

^ Go to TOC

7.11 Create a Course Workspace

Pick a stable working directory for lab files:

```
mkdir -p ~/podman-labs # create lab workspace
```

Set up a simple alias for the course repo (if cloned locally):

```
# Optional: keep a reference to your course repo  
COURSE_REPO=~/.course_podman # path to the course git repo
```

You'll run most labs from this directory. Keep it tidy — label or prefix experiment containers so you can clean them up easily.

^ Go to TOC

7.12 Recommended Shell Safety

These reduce accidents in lab environments:

```
set -o noclobber # prevent overwriting existing files with > redirect
```

Use `set -euo pipefail` in lab scripts:

```
#!/usr/bin/env bash  
set -euo pipefail # exit on error, unset variable, or pipe failure
```

- `-e`: exit immediately on error (prevents silently continuing after a failure).
- `-u`: treat unset variables as errors.
- `-o pipefail`: catch failures inside pipelines (e.g., `cmd1 | cmd2`).

Do not apply `set -e` interactively — it can cause confusing session exits.

^ Go to TOC

7.13 Linger — Boot Safety for User Services

Systemd user sessions normally only run while a user is logged in. For a server that should run containers at boot (before login), enable **lingering**:

```
sudo loginctl enable-linger "$USER" # allow user services to start at boot, without login
```

Verify `linger` is enabled:

```
loginctl show-user "$USER" | grep Linger # expected: Linger=yes
```

Without `linger`: your Quadlet services start only when you SSH in. With `linger`: your Quadlet services start at boot, before any login.

This is a one-time setup per user on each machine.

^ Go to TOC

7.14 Version Matrix and Compatibility Notes

Different RHEL/Fedora versions ship different Podman versions. Key feature availability:

Feature	Minimum Podman version
Quadlet (<code>.container</code> units)	4.4
<code>podman secret</code>	3.1
<code>podman play kube</code>	3.0
<code>podman auto-update</code>	3.2
<code>podman pod</code>	1.0
Build secrets (<code>--secret</code>)	3.0 (via Buildah)
Health checks in Quadlet	4.5

Check your version:

```
podman --version # print Podman version
```

If a lab step fails unexpectedly, check whether your version supports the feature before debugging further.

^ Go to TOC

7.15 Checkpoint

- `podman info` runs without errors.
- `podman run --rm docker.io/library/alpine:latest uname -a` works rootless.
- `podman info --format '{{.Host.CgroupsVersion}}'` prints v2.
- `/etc/subuid` and `/etc/subgid` have entries for your user.
- `getenforce` prints **Enforcing** or **Permissive** (not **Disabled**).
- You know where container images are stored (`~/.local/share/containers/storage/`).

^ Go to TOC

7.16 Quick Quiz

- 1) What does `podman unshare` do, and when would you use it?
- 2) You run `podman run --rm alpine echo hello` and get **permission denied** on a bind-mounted directory. What is the first thing to check on an RHEL system?
- 3) What two things must be true for Quadlet services to start at boot without a login?
- 4) Why is cgroups v2 required for Quadlet? What breaks with cgroups v1?
- 5) Your colleague says “I disabled SELinux so my containers work”. What security risk did they introduce?

^ Go to TOC

7.17 Further Reading

- Podman install docs: <https://podman.io/docs/installation>
- Rootless Podman tutorial: https://github.com/containers/podman/blob/main/docs/tutorials/rootless_tutorial.md
- cgroups v2 (kernel docs): <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>
- `loginctl` (linger for user services): <https://www.freedesktop.org/software/systemd/man/latest/loginctl.html>
- SELinux with containers (RHEL docs): https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/using-selinux-with-containers_using-selinux
- `subuid(5)` man page: <https://man7.org/linux/man-pages/man5/subuid.5.html>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

8 Module 1: Containers 101



8.1 Table of Contents

- Learning Goals
 - What Is a Container, Really?
 - The Three Things To Get Right
 - How `podman run` Works — Step by Step
 - Containers Are Not VMs
 - Linux Namespaces — The Isolation Mechanism
 - cgroups — The Resource Accounting Mechanism
 - The OCI Standards
 - Rootless vs Rootful
 - Terminology You Will See
 - Lab: Compare Host vs Container
 - Lab: Namespace Exploration
 - Checkpoint
 - Quick Quiz (Answer Without Running Commands)
 - Further Reading
-

8.2 Learning Goals

By the end of this module you will be able to:

- Explain what a container is in terms of kernel mechanisms (namespaces + cgroups), not just analogy.
- Distinguish an **image** from a **container**.
- Trace the lifecycle of `podman run` from CLI invocation to PID 1 inside the container.
- Articulate why containers are not VMs and what that means for security.
- Choose rootless Podman by default and explain the tradeoff.
- Use correct OCI terminology in conversation.

[^ Go to TOC](#)

8.3 What Is a Container, Really?

A container is a **normal Linux process (or process tree) with extra kernel-enforced restrictions applied at launch time**.

That is all it is. There is no container hypervisor. There is no container kernel. Your container's PID 1 is just a process on the same Linux kernel as everything else on the machine — it simply has a restricted view of the world.

Those restrictions come from two kernel features:

1. **Namespaces** — limit what the process can *see* (other processes, network interfaces, filesystem mounts, hostnames, user IDs).
2. **cgroups** — limit what the process can *use* (CPU, memory, disk I/O, number of PIDs).

An image is the read-only recipe: a stack of filesystem layers plus metadata (entrypoint, environment variables, exposed ports, labels). A container is a *running instance* of an image — the image layers are used read-only, and a thin writable layer is placed on top for the lifetime of the container.

Analogy: An image is a class definition. A container is an object instance. You can spin up a hundred containers from the same image, each with its own writable state, without modifying the image.

[^ Go to TOC](#)

8.4 The Three Things To Get Right

8.4.1 1. Image — the immutable template

- Built from a **Containerfile** (or pulled from a registry).
- Composed of ordered, read-only layers.
- Identified by a **tag** (mutable pointer) or a **digest** (immutable SHA256 hash).
- Never changes when you run it — all writes go to the container’s writable layer.

8.4.2 2. Container — the running (or stopped) instance

A container adds to the image:

- A **process** (PID 1 and its children).
- A thin **writable layer** on top of the image layers (copy-on-write).
- **Mounts** — volumes, bind mounts, tmpfs, or secret files layered in.
- **Network settings** — IP address, DNS, published ports.
- **Environment variables** and runtime configuration.
- A **state machine**: created -> running -> paused -> exited -> removed.

8.4.3 3. Runtime boundaries

The container runtime (in Podman’s case: **crun** by default) sets up:

- **Namespaces** — isolates the process’s view of PIDs, network, mounts, UTS, IPC, users.
- **cgroups** — enforces resource limits.
- **seccomp** — filters which kernel syscalls are allowed (optional but recommended).
- **capabilities** — drops Linux capabilities the process does not need.
- **SELinux/AppArmor labels** — mandatory access control on Fedora/RHEL.

Critical mental model: The kernel is always shared. A kernel exploit inside a container is a host exploit. Container security is about *reducing blast radius*, not achieving VM-grade isolation.

[^ Go to TOC](#)

8.5 How podman run Works — Step by Step

Understanding the lifecycle from CLI to PID 1 prevents a huge class of “why is my container doing this?” confusion.

sequenceDiagram

```
participant U as "User shell"
participant P as "podman CLI"
participant CS as "common"
participant R as "crun"
participant K as "Linux kernel"
participant C as "Container PID 1"
```

```
U->>P: "podman run --rm nginx"
```

```

P->>P: "Resolve image<br/>(pull if missing)"
P->>P: "Create OCI bundle<br/>(rootfs + config.json)"
P->>CS: "Fork common"
CS->>R: "crun create bundle"
R->>K: "clone() with<br/>namespace flags"
K-->>R: "New namespace set created"
R->>K: "Apply cgroup limits"
R->>K: "Set capabilities + seccomp"
R-->>CS: "Container created"
CS->>R: "crun start"
R->>C: "Exec PID 1 in new namespaces"
C-->>U: "Container is running"

```

Key insight: podman itself does not stay running while your container runs. It forks `common` (container monitor) and exits. `common` is the long-lived process that holds the container's stdio and watches for exit. This is why Podman is daemonless — unlike Docker, there is no central `dockerd` that must stay healthy for your containers to keep running.

If `dockerd` crashes, all Docker containers lose their stdio management. If `podman` exits (which it does immediately after launch), your containers are completely unaffected.

^ Go to TOC

8.6 Containers Are Not VMs

This is the most important conceptual distinction in this course. Getting it wrong leads to both operational mistakes and security blind spots.

graph TD

```

subgraph "Virtual Machine model"
    HW["Physical Hardware"]
    HW --> HYP["Hypervisor"]
    HYP --> GK1["Guest Kernel 1"]
    HYP --> GK2["Guest Kernel 2"]
    GK1 --> GP1["Guest Process A"]
    GK2 --> GP2["Guest Process B"]
end

subgraph "Container model"
    HW2["Physical Hardware"]
    HW2 --> HK["Host Kernel (shared by all)"]
    HK --> NS1["Namespace set 1"]
    HK --> NS2["Namespace set 2"]
    NS1 --> CP1["Container Process A"]
    NS2 --> CP2["Container Process B"]
end

```

Property	VM	Container
Kernel	Each VM has its own	Shared host kernel
Boot time	Seconds to minutes	Milliseconds
Memory overhead	Hundreds of MB per VM	Tens of MB per container
Isolation level	Strong (hypervisor boundary)	Moderate (namespace boundary)
Kernel CVE impact	Guest kernel only	All containers on host

Property	VM	Container
File system	Full virtual disk	Layered image + thin writable layer
Portability	Image includes kernel	Image includes only userspace

What this means practically:

- You cannot run a Windows container on a Linux kernel (the container shares the host kernel).
- A kernel vulnerability that lets a process escape a namespace affects all containers on that host.
- Containers start in milliseconds because there is no kernel to boot.
- Containers are small because they do not need to ship a kernel.

^ Go to TOC

8.7 Linux Namespaces — The Isolation Mechanism

A **namespace** wraps a global system resource so that processes inside the namespace see their own isolated copy. The kernel tracks which namespace each process belongs to.

Podman uses all six isolation namespaces by default:

Namespace	Kernel flag	What it isolates	Effect in container
PID	<code>CLONE_NEWPID</code>	Process IDs	Container sees its own PID 1; cannot see host PIDs
Network	<code>CLONE_NEWNET</code>	Network interfaces, routing, ports	Container gets its own <code>eth0</code> ; cannot see host <code>eth0</code>
Mount	<code>CLONE_NEWNS</code>	Filesystem mount points	Container sees its own root filesystem
UTS	<code>CLONE_NEWUTS</code>	Hostname, domain name	Container can have a different hostname
IPC	<code>CLONE_NEWIPC</code>	SysV IPC, POSIX message queues	Containers cannot IPC with each other by default
User	<code>CLONE_NEWUSER</code>	User and group IDs	Container's root (UID 0) maps to an unprivileged UID on the host

The **User namespace** is the one that makes rootless containers secure. When you run Podman without `sudo`, your container's UID 0 (root) is mapped to your own unprivileged UID on the host. If the container process escapes every other namespace, it is still just your unprivileged user on the host.

```
graph LR
    subgraph "Host"
        HU["Host UID 1000 (your user)"]
        HU2["Host UID 100001 (subUID range)"]
    end
    subgraph "User Namespace (inside container)"
        CU0["Container UID 0 (root inside)"]
        CU1["Container UID 1 (daemon inside)"]
    end
```

```
CU0 -->|"maps to"| HU
CU1 -->|"maps to"| HU2
```

Why this matters: Without user namespaces, a container running as root would be running as root on the host too. User namespaces are why rootless Podman is meaningfully more secure than rootful Docker for most workloads.

^ Go to TOC

8.8 cgroups — The Resource Accounting Mechanism

Control groups (cgroups) let the kernel track and limit the resources a process tree can consume. Without cgroups, a single runaway container could OOM the entire host.

Podman uses **cgroups v2** (the unified hierarchy) on modern RHEL 10 / Fedora systems.

Key cgroup controllers used by containers:

Controller	What it limits
memory	Maximum RAM + swap; triggers OOM kill when exceeded
cpu	CPU share / quota (e.g., “max 0.5 cores”)
pids	Maximum number of processes/threads (prevents fork bombs)
io	Block device I/O bandwidth and IOPS
cpuset	Which CPU cores and NUMA nodes are allowed

flowchart TD

```
A(["podman run --memory 256m --cpus 0.5 nginx"])
A --> B["crun creates container"]
B --> C["Writes limits to cgroup v2 files"]
C --> D["memory.max = 268435456"]
C --> E["cpu.max = 50000 100000"]
C --> F["pids.max = 256"]
D --> G["Kernel enforces limits<br/>on all container PIDs"]
E --> G
F --> G
```

Practical note: cgroups v2 requires a systemd user session when running rootless. This is why the course targets RHEL 10 / Fedora with `loginctl enable-linger` — it keeps your user session and cgroup hierarchy alive even when you are logged out.

^ Go to TOC

8.9 The OCI Standards

OCI stands for the Open Container Initiative. It defines three standards that make container tooling interoperable:

Standard	What it specifies	Why it matters
Image Spec	How image layers and manifests are structured	You can build with Podman, push to a registry, pull with containerd
Runtime Spec	The <code>config.json</code> format that runtimes consume	<code>crun</code> , <code>runc</code> , <code>kata-containers</code> all accept the same bundle

Standard	What it specifies	Why it matters
Distribution Spec	The HTTP API for pushing/pulling images	Every major registry (quay.io, ghcr.io, docker.io) speaks the same protocol

This is why the course uses **Containerfile** rather than **Dockerfile** — both produce OCI images, but **Containerfile** is the name used by Podman/Buildah and makes the OCI-first intent explicit.

Practical implication: Any image you build with Podman will run on Kubernetes, containerd, or any other OCI-compliant runtime. The ecosystem is genuinely interoperable at the image and distribution layers.

^ Go to TOC

8.10 Rootless vs Rootful

graph TD

```

subgraph "Rootful Podman (sudo / root)"
    RP["podman run (as root)"]
    RP --> RN["Binds ports < 1024 directly"]
    RP --> RM["Mounts host paths without UID remapping"]
    RP --> RC["Full host cgroup v1/v2 access"]
    RP --> RR["Container UID 0 = Host UID 0 (risk)"]
end
subgraph "Rootless Podman (your user)"
    LP["podman run (as user)"]
    LP --> LN["Ports via slirp4netns or pasta"]
    LP --> LM["UID remapping via /etc/subuid"]
    LP --> LC["User-scoped cgroup hierarchy"]
    LP --> LR["Container UID 0 = Host unprivileged UID (safer)"]
end

```

When to use rootful: - You genuinely need to bind a port < 1024 without OS-level workarounds. - You need `macvlan` or `ipvlan` network drivers. - You are running a CI agent that itself orchestrates containers. - You are explicitly told by an ops team that rootful is required for the workload.

Default choice: rootless. This course uses rootless Podman throughout. Every lab assumes you are running as an ordinary user, not root.

Warning: Running containers rootful on a multi-user system means a container escape is a root escape. Always question whether you truly need it.

^ Go to TOC

8.11 Terminology You Will See

Term	Definition
Image	Read-only layer stack + metadata. The “recipe”.
Container	Running (or stopped) instance of an image. Has state.
Registry	HTTP server that stores and serves images (quay.io, ghcr.io, docker.io).

Term	Definition
Tag	Mutable human-readable pointer to an image (e.g. <code>nginx:1.27</code>). Can be moved.
Digest	Immutable SHA256 content hash of an image manifest (e.g. <code>sha256:abc123...</code>).
OCI	Open Container Initiative — the standards body for image/runtime/distribution specs.
Containerfile	The build recipe (Podman's name for what Docker calls a Dockerfile).
Layer	A diff of filesystem changes. Layers are stacked to form the image rootfs.
crun	The default OCI runtime used by Podman (written in C; faster and smaller than runc).
common	Container monitor process — manages stdio and exit detection per container.
rootless	Running Podman (and containers) as an unprivileged user, using user namespaces.
rootful	Running Podman as root. Required for some advanced networking and capabilities.
slirp4netns	Userspace network stack that gives rootless containers outbound connectivity.
pasta	Newer, faster replacement for slirp4netns (available in RHEL 10 / Fedora 39+).

[^ Go to TOC](#)

8.12 Lab: Compare Host vs Container

This lab makes the namespace isolation concrete. You will run the same commands on the host and inside a container and compare the output.

Step 1 — Check process isolation (PID namespace)

On the host, list processes:

```
ps aux | head -20 # list running processes on the host
```

You will see dozens of processes with various PIDs. Now run the same command inside a container:

```
podman run --rm docker.io/library/alpine:latest ps aux # run a container and list its processes
```

You should see only two processes: `ps` itself and possibly the shell. The container has a completely separate PID namespace — it cannot see the host's processes.

Step 2 — Check network isolation (network namespace)

On the host:

```
ip addr show # show network interfaces on the host
```

You will see your real `eth0` or `enpXs0`. Inside a container:

```
podman run --rm docker.io/library/alpine:latest ip addr show # run a container and show its interfaces
```

The container sees only `lo` (loopback) and `eth0` inside its own network namespace. The `eth0` inside is a virtual ethernet device — not the host's real interface.

Step 3 — Check hostname isolation (UTS namespace)

```
hostname # show current hostname on the host
podman run --rm docker.io/library/alpine:latest hostname # run a container and show its hostname
```

The container gets an auto-generated hostname (the container ID prefix) by default.

Step 4 — Observe the UID mapping (user namespace)

```
id # show your user and group IDs on the host
podman run --rm docker.io/library/alpine:latest id # run a container and show its uid/gid
```

Inside the container you are uid=0(root). But from the host's perspective:

```
podman run -d --name uid-check docker.io/library/alpine:latest sleep 60 # run a container in background
sleep 1
ps aux | grep "sleep 60" # check what UID the sleep process has on the host
podman rm -f uid-check # clean up
```

You will see it running as your own UID on the host — not as root. This is the user namespace mapping in action.

What just happened?

Each `podman run` call set up a fresh set of namespaces. The container got its own PID tree, its own network stack, its own hostname, and a user namespace that remapped root to your unprivileged UID. All of these are kernel features — no special container software was needed beyond the runtime that called `clone()` with the right flags.

^ Go to TOC

8.13 Lab: Namespace Exploration

This lab looks directly at the namespace file descriptors to make the abstraction tangible.

Step 1 — Start a long-running container

```
podman run -d --name ns-demo docker.io/library/alpine:latest sleep 300 # run a detached container
```

Step 2 — Find the PID of the container's main process on the host

```
CPID=$(podman inspect ns-demo --format '{{.State.Pid}}') # get the container's host PID
echo "Container PID on host: $CPID"
```

Step 3 — List the namespace symlinks

```
ls -la /proc/$CPID/ns/ # list the namespace file descriptors for this PID
```

You will see entries like:

```
lrwxrwxrwx ... cgroup -> cgroup:[4026531835]
lrwxrwxrwx ... ipc    -> ipc:[4026532768]
lrwxrwxrwx ... mnt    -> mnt:[4026532766]
lrwxrwxrwx ... net    -> net:[4026532771]
lrwxrwxrwx ... pid    -> pid:[4026532769]
lrwxrwxrwx ... user   -> user:[4026532765]
lrwxrwxrwx ... uts    -> uts:[4026532767]
```

Each inode number (the number after the colon) is the unique identity of that namespace. Compare these to your shell's namespaces:

```
ls -la /proc/self/ns/ # list your own shell's namespace file descriptors
```

The inode numbers for `net`, `pid`, `mnt`, and `user` will be different — confirming the container is truly in separate namespaces.

Step 4 — Clean up

```
podman stop ns-demo && podman rm ns-demo # stop and remove the container
```

^ Go to TOC

8.14 Checkpoint

Before moving on, confirm you can answer these without referring to notes:

- ☐ I can explain the difference between an image and a container.
- ☐ I know that containers share the host kernel and why that matters for security.
- ☐ I can name the six namespaces Podman uses and what each one isolates.
- ☐ I understand that rootless Podman maps container UID 0 to an unprivileged host UID.
- ☐ I can explain what cgroups do and why they matter.
- ☐ I know what OCI stands for and why the standard matters for portability.
- ☐ I have run the host vs container comparison lab and seen namespace isolation with my own eyes.

^ Go to TOC

8.15 Quick Quiz (Answer Without Running Commands)

1. You run `podman run --rm alpine ps aux` and see only two processes. Why does the container not see the hundreds of processes running on the host?
2. A coworker says “just run it as root, it’s fine, it’s in a container.” What is the specific risk they are dismissing?
3. What is the difference between a tag like `nginx:1.27` and a digest like `sha256:abc123...`? When would you prefer each?
4. Why does Podman not have a daemon that must keep running for your containers to stay alive?
5. A container is using 512 MB of RAM but you set `--memory 256m`. What does the kernel do?

^ Go to TOC

8.16 Further Reading

- OCI image spec: <https://github.com/opencontainers/image-spec>
- OCI runtime spec: <https://github.com/opencontainers/runtime-spec>
- Linux namespaces man page: <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- Linux cgroups man page: <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- User namespaces man page: https://man7.org/linux/man-pages/man7/user_namespaces.7.html
- crun (OCI runtime): <https://github.com/containers/crun>
- conmon (container monitor): <https://github.com/containers/conmon>
- Podman rootless overview: https://github.com/containers/podman/blob/main/docs/tutorials/rootless_tutorial.md
- pasta (network tool): <https://passt.top>
- Podman overview docs: <https://podman.io/docs>

^ Go to TOC

9 Module 2: Everyday Podman Commands



9.1 Table of Contents

- Learning Goals
- The Container Lifecycle
- Core Commands
- Useful Flags (Learn These Early)
- Inspecting and Debugging
- Lab: The Writable Layer Is Not Persistence
- Lab: Name Your Containers
- Lab: Exit Codes and Restart Behavior
- Formatting Output
- Cleaning Up Without Nuking Your System
- Checkpoint
- Quick Quiz
- Further Reading

[^ Go to TOC](#)

9.2 Learning Goals

- Run containers interactively and in the background.
- Use `logs` and `exec` to debug running containers.
- Understand the container lifecycle (create -> start -> running -> stopped -> removed).
- Clean up containers and images safely.
- Understand naming, exit codes, and restart behavior.
- Format `podman ps` and `podman inspect` output for scripts.

[^ Go to TOC](#)

9.3 The Container Lifecycle

Understanding the lifecycle prevents a common confusion: the difference between a container that “exists” and one that is “running”.

flowchart LR

```
A["podman run"] --> B["Created<br/>(image pulled if needed)"]
B --> C["Running<br/>(PID 1 executing)"]
C -->|"process exits"| D["Stopped / Exited<br/>(state preserved)"]
C -->|"podman stop"| D
D -->|"podman start"| C
D -->|"podman rm"| E["Removed<br/>(gone)"]
C -->|"podman rm -f"| E
A -->|"--rm flag"| F["Removed on exit<br/>(shortcut for labs)"]
```

Key states:

State	podman ps shows it	Container data exists	Process running
Running	Yes (default)	Yes	Yes
Stopped/Exited	Only with <code>-a</code>	Yes	No
Removed	Never	No	No

This is why `podman ps` without `-a` shows nothing after a container exits — the state still exists, just stopped. Use `podman ps -a` to see all containers.

^ Go to TOC

9.4 Core Commands

Run and remove when done (good default for experiments):

```
podman run --rm docker.io/library/alpine:latest echo hello # run, then auto-remove on exit
```

Run a long-lived container in the background:

```
podman run -d --name sleep1 docker.io/library/alpine:latest sleep 600 # detached, named container
podman ps # list running containers
```

List all containers (including stopped):

```
podman ps -a # list all containers regardless of state
```

View container logs:

```
podman logs sleep1 # print all logs since container start
podman logs -f sleep1 # follow/tail logs in real time
podman logs --tail 20 sleep1 # show only last 20 lines
```

Execute a command in a running container:

```
podman exec -it sleep1 sh # interactive shell inside running container
```

Tip: if the image doesn't have `sh`, try `bash`. For minimal images (Alpine uses `ash`), `sh` works. For distroless images, you may need an ephemeral debug container.

Stop, start, and remove:

```
podman stop sleep1 # graceful stop (SIGTERM, then SIGKILL after timeout)
podman start sleep1 # restart a stopped container (reuses same writable layer)
podman rm sleep1 # remove a stopped container
podman rm -f sleep1 # force-remove (stops if running, then removes)
```

Wait for a container to exit:

```
podman wait sleep1 # blocks until container exits, prints exit code
```

Inspect exit code:

```
podman inspect sleep1 --format '{{.State.ExitCode}}' # get last exit code
```

^ Go to TOC

9.5 Useful Flags (Learn These Early)

Flag	Purpose	Example
<code>--name <name></code>	Stable name for scripts and <code>exec</code>	<code>--name mydb</code>
<code>-d</code>	Detached (background)	<code>podman run -d ...</code>
<code>--rm</code>	Auto-remove on exit	For experiments only
<code>-it</code>	Interactive + TTY	<code>podman run -it alpine sh</code>
<code>-e KEY=VALUE</code>	Environment variable	Avoid for secrets
<code>-v name:/path</code>	Mount named volume	<code>--v dbdata:/var/lib/mysql</code>
<code>-p host:container</code>	Publish port	<code>-p 8080:80</code>
<code>--network <net></code>	Join a network	<code>--network mynet</code>
<code>--user <uid[:gid]></code>	Run as specific user	<code>--user 1001:1001</code>

Flag	Purpose	Example
<code>--secret <name></code>	Mount a Podman secret	<code>--secret db_password</code>
<code>--read-only</code>	Read-only root FS	Security hardening
<code>--tmpfs /path</code>	In-memory writable directory	<code>--tmpfs /tmp</code>
<code>--memory 256m</code>	Memory limit	Resource limiting
<code>--restart on-failure</code>	Restart policy	For <code>podman run</code> , not <code>Quadlet</code>

Mnemonic: “**D-NAME-IT**: **D**etach, **N**ame, **I**nteractive, **E**nv, **V**olume, **P**ublish, **N**etwork, **U**ser, **S**ecret”

^ Go to TOC

9.6 Inspecting and Debugging

Quick status overview:

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}' # formatted container list
```

Inspect all metadata as JSON:

```
podman inspect sleep1 # full JSON metadata: config, state, mounts, network, etc.
```

Extract a specific field:

```
podman inspect sleep1 --format '{{.State.Status}}' # running / exited
podman inspect sleep1 --format '{{.State.ExitCode}}' # exit code
podman inspect sleep1 --format '{{.NetworkSettings.IPAddress}}' # container IP
podman inspect sleep1 --format '{{json .HostConfig}}' # host config as JSON
```

Check what ports are published:

```
podman port sleep1 # list all port mappings for a container
```

Inspect a running process list inside the container:

```
podman top sleep1 # show running processes inside container
```

View resource usage:

```
podman stats --no-stream # single snapshot of CPU/memory/IO usage for all running containers
```

Debug a minimal image without a shell:

If your production image is distroless or has no shell, use `podman debug`:

```
podman debug sleep1 # attach a debug container to an existing container's namespaces
```

Or use an ephemeral container on the same network:

```
podman run --rm --network container:sleep1 docker.io/library/busybox:latest netstat -tlnp # share network
```

^ Go to TOC

9.7 Lab: The Writable Layer Is Not Persistence

Every container has a **writable layer** on top of its image. Writes go there, not into the image. When the container is removed, the writable layer is gone.

Step 1: Create a file inside a container:

```
podman run -it --name scratch docker.io/library/alpine:latest sh # run interactive container
```

Inside the container:

```
echo "I was here" > /tmp/hello.txt  # write a file to the writable layer
cat /tmp/hello.txt                  # confirm it's there
exit
```

Step 2: The container stopped. The file still exists in the stopped container:

```
podman ps -a                        # scratch is stopped but exists
podman start scratch                 # restart the same container
podman exec scratch cat /tmp/hello.txt # file is still there (same writable layer)
podman stop scratch
```

Step 3: Remove the container and recreate it — file is gone:

```
podman rm scratch                  # destroy the writable layer
podman run -it --name scratch docker.io/library/alpine:latest sh # fresh writable layer
```

Inside:

```
ls /tmp/hello.txt # file is gone - expected!
exit
```

Cleanup:

```
podman rm scratch # clean up
```

The lesson: `podman stop` preserves state; `podman rm` destroys it. To persist data across container replacements, use **named volumes** (Module 5).

^ Go to TOC

9.8 Lab: Name Your Containers

Unnamed containers get random two-word names (`romantic_currie`, `busy_wozniak`). These are fine for experiments but break automation.

Step 1: Run two containers with explicit names:

```
podman run -d --name worker-a docker.io/library/alpine:latest sleep 300 # named container A
podman run -d --name worker-b docker.io/library/alpine:latest sleep 300 # named container B
podman ps # both should be listed
```

Step 2: Reference by name in exec:

```
podman exec worker-a hostname # prints worker-a (container hostname defaults to container name)
podman exec worker-b hostname # prints worker-b
```

Step 3: Try to reuse a name and observe the error:

```
podman run -d --name worker-a docker.io/library/alpine:latest sleep 300 # should fail
```

Expected: Error: container name "worker-a" is already in use

This teaches you to clean up before re-running scripts, or use `podman rm -f worker-a` first.

Cleanup:

```
podman rm -f worker-a worker-b # stop and remove both
```

^ Go to TOC

9.9 Lab: Exit Codes and Restart Behavior

Exit codes tell you why a container stopped. Knowing them speeds up debugging.

Exit code	Meaning
0	Clean exit (process finished normally)
1	General error (check logs)
125	Podman itself errored (not the container)
126	Command found but not executable
127	Command not found
130	Killed by Ctrl+C (SIGINT)
137	Killed by SIGKILL (OOM killer, or <code>podman kill</code>)
143	Killed by SIGTERM (<code>podman stop</code>)

Step 1: Observe a clean exit (code 0):

```
podman run --name exit0 docker.io/library/alpine:latest echo hello # exits with code 0
podman inspect exit0 --format '{{.State.ExitCode}}' # should print: 0
podman rm exit0
```

Step 2: Observe an error exit (code 1):

```
podman run --name exit1 docker.io/library/alpine:latest sh -c 'exit 1' # exits with code 1
podman inspect exit1 --format '{{.State.ExitCode}}' # should print: 1
podman rm exit1
```

Step 3: Observe command-not-found (code 127):

```
podman run --rm docker.io/library/alpine:latest notacommand # should fail with 127
echo "Exit code: $?" # print the exit code
```

Step 4: Observe OOM kill (code 137) — safe with a memory limit:

```
podman run --rm --memory 4m docker.io/library/alpine:latest sh -c 'dd if=/dev/zero of=/tmp/x bs=1M count=1'
echo "Exit code: $?" # 137 if OOM-killed, or the dd may just fail
```

^ Go to TOC

9.10 Formatting Output

`podman ps`, `podman images`, and `podman inspect` all support Go template formatting.

Common ps formats:

```
podman ps --format '{{.Names}}' # just names
podman ps --format '{{.Names}}\t{{.Status}}' # names and status
podman ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}' # table with headers
podman ps -a --format json # full JSON output
```

Inspect with templates:

```
podman inspect mycontainer --format '{{.State.Status}}' # status string
podman inspect mycontainer --format '{{.Config.Image}}' # image used
podman inspect mycontainer --format '{{range .Mounts}}{{.Destination}} {{end}}' # list mount paths
```

Images:

```
podman images --format '{{.Repository}}:{{.Tag}}\t{{.Size}}' # name:tag and size
podman images --digests --format '{{.Repository}}\t{{.Digest}}' # show digests
```

^ Go to TOC

9.11 Cleaning Up Without Nuking Your System

See what you have:

```
podman ps -a          # all containers (running and stopped)
podman images         # all images
podman volume ls      # all named volumes
podman network ls     # all networks
```

Safe cleanup — stopped containers only:

```
podman container prune -f    # remove all stopped containers (leaves running ones alone)
```

Safe cleanup — dangling images (untagged, not used by any container):

```
podman image prune -f        # remove dangling images only
```

More aggressive — all unused images (not used by any container, running or stopped):

```
podman image prune -a -f     # removes ALL images not referenced by a container
```

(!) This will force a re-pull next time you run a container. On slow connections, be selective.

System-wide prune (containers + networks + images, no volumes):

```
podman system prune -f       # remove stopped containers, unused networks, dangling images
```

Check disk usage before pruning:

```
podman system df             # show disk usage breakdown: images, containers, volumes
```

Rule of thumb: `container prune` is safe to run daily. `image prune -a` should be deliberate.

^ Go to TOC

9.12 Checkpoint

- You can use `podman logs` and `podman exec` without guessing.
- You understand why `podman ps` vs `podman ps -a` give different results.
- You understand why volumes exist — the writable layer is not persistence.
- You can read a container exit code and know what it means.
- You can format `podman ps` output for scripts.

^ Go to TOC

9.13 Quick Quiz

- 1) What is the difference between `podman run` and `podman start`?
- 2) A container exits immediately after you start it. Where do you look first?
- 3) `podman ps` shows nothing. Does that mean no containers exist? How do you check?
- 4) You run the same `podman run --name myapp ... command` twice. The second run fails. Why, and how do you fix it?
- 5) Exit code 137 — what likely happened?
- 6) How would you get the IP address of a running container without using `podman exec`?

^ Go to TOC

9.14 Further Reading

- `podman-run(1)`: <https://docs.podman.io/en/latest/markdown/podman-run.1.html>
- `podman-ps(1)`: <https://docs.podman.io/en/latest/markdown/podman-ps.1.html>
- `podman-logs(1)`: <https://docs.podman.io/en/latest/markdown/podman-logs.1.html>
- `podman-exec(1)`: <https://docs.podman.io/en/latest/markdown/podman-exec.1.html>
- `podman-inspect(1)`: <https://docs.podman.io/en/latest/markdown/podman-inspect.1.html>
- Go template formatting: <https://pkg.go.dev/text/template>

[^] Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

10 Module 3: Images and Registries



10.1 Table of Contents

- Learning Goals
- What Is an Image
- Tags vs Digests
- Fully Qualified Image Names
- Short-Name Resolution and registries.conf
- Lab: Pull by Tag, Record Digest
- Registry Authentication
- Image Metadata: ENTRYPOINT vs CMD
- Inspecting Image Layers
- Lab: Local Tagging
- Lab (Optional): Push to a Local Registry
- Lab (Optional): Save and Load
- Image Storage on Disk
- Checkpoint
- Quick Quiz
- Further Reading

^ Go to TOC

10.2 Learning Goals

- Understand what a container image is and how it is structured in layers.
- Pull images by tag and by digest.
- Explain why digests are safer than tags for production.
- Inspect image metadata (entrypoint, CMD, exposed ports, labels).
- Understand short-name resolution and why fully qualified names matter.
- Authenticate to a registry without leaking credentials to shell history.

^ Go to TOC

10.3 What Is an Image

A container image is a **stack of read-only filesystem layers** plus metadata. Each layer represents a filesystem diff from the layer below it.

graph TD

```
subgraph "Image layers (read-only)"
    L1["Layer 1: Base OS<br/>(e.g., Alpine 3.19)"]
    L2["Layer 2: Package install<br/>(e.g., RUN apk add nginx)"]
    L3["Layer 3: App files<br/>(e.g., COPY . /app)"]
end
subgraph "Container (runtime)"
    W["Writable layer<br/>(overlay on top)"]
end
L1 --> L2 --> L3 --> W
```

Key properties:

- **Layers are content-addressed** by SHA256 digest. Two images sharing a layer share the same data on disk.

- **Layers are immutable** — once built, they never change. A new `podman build` creates new layers.
- **The writable layer** is added per-container at runtime. It is discarded on `podman rm`.
- **The image digest** is a SHA256 of the image manifest (which includes all layer digests). It is globally unique and immutable.

^ Go to TOC

10.4 Tags vs Digests

A **tag** (like `:latest`, `:stable`, `:3.19`) is a mutable human-readable pointer. The registry can move a tag to point to a different image at any time.

A **digest** (like `@sha256:abc123...`) is a content-addressed, immutable identifier. It always refers to the exact same bytes.

flowchart LR

```
T["Tag: nginx:stable"] -->|"points to today"| I1["Image sha256:aaaa"]
T -->|"after registry update"| I2["Image sha256:bbbb"]
D1["Digest: nginx@sha256:aaaa"] -->|"always"| I1
D2["Digest: nginx@sha256:bbbb"] -->|"always"| I2
```

```
style T fill:#2d1a00,color:#ffcc88
style D1 fill:#001a2d,color:#88ccff
style D2 fill:#001a2d,color:#88ccff
```

Practical consequences:

Scenario	Tag	Digest
Reproduce a bug	[X] Tag may have moved	[OK] Exact bytes
Audit what ran in production	[X] Tag is ambiguous	[OK] Exact bytes
Auto-update to latest patch	[OK] Tag moves forward	[X] Must update manually
Incident response: “what version is running?”	[X] Need to check digest at time of deploy	[OK] Digest is the version

For production: pin by digest. Record it. Upgrade by explicitly choosing a new digest.

^ Go to TOC

10.5 Fully Qualified Image Names

Always use the full registry + namespace + image + tag/digest form:

```
docker.io/library/nginx:stable
~~~~~ ~~~~~ ~~~~~ ~~~~~
registry namespace image tag
```

Avoid relying on short names:

```
nginx:stable      # short name - Podman must guess the registry
alpine           # even shorter - registry AND tag are guessed
```

Why it matters: short-name resolution depends on `/etc/containers/registries.conf`. On different systems, `alpine` might resolve to `docker.io/library/alpine` or `registry.access.redhat.com/ubi9-minimal` depending on search order. This makes automation unpredictable.

Short names in labs are fine for exploration. Short names in scripts, Containerfiles, and Quadlet units should be replaced with fully qualified names.

^ Go to TOC

10.6 Short-Name Resolution and registries.conf

Podman's short-name resolution is configured in `/etc/containers/registries.conf` (system-wide) and `~/.config/containers/registries.conf` (per-user).

View system config:

```
cat /etc/containers/registries.conf # show registry search order and aliases
```

Common settings:

```
# Ordered list of registries to search for short names:
unqualified-search-registries = ["docker.io", "registry.access.redhat.com"]

# Alias: "fedora" -> fully qualified name:
[[registry.aliases]]
"fedora" = "registry.fedoraproject.org/fedora"
```

On RHEL systems, the `registries.conf` is often configured to prompt for registry selection interactively when a short name is used. This is intentional — it prevents silently pulling from the wrong registry.

In automation (scripts, CIs, Containerfiles), always use fully qualified names to avoid this prompt.

^ Go to TOC

10.7 Lab: Pull by Tag, Record Digest

Step 1: Pull by tag:

```
podman pull docker.io/library/alpine:latest # pull Alpine latest tag
podman images | grep alpine                # confirm image is stored locally
```

Step 2: Inspect the image to get the digest:

```
podman images --digests | grep alpine # show digest column
podman inspect docker.io/library/alpine:latest --format '{{.Digest}}' # print digest only
```

Record the digest — it looks like `sha256:abc123...`

Step 3: Pull the exact same image by digest:

```
# Replace <digest> with your actual digest from step 2:
podman pull docker.io/library/alpine@sha256:<digest> # pull by immutable digest
podman images --digests | grep alpine                # now you see both entries (same layers, different tags)
```

Step 4: Run by digest:

```
podman run --rm docker.io/library/alpine@sha256:<digest> uname -a # run pinned image
```

Step 5: Verify layer sharing:

```
podman system df # images section - pinned and tag-based share the same layers
```

^ Go to TOC

10.8 Registry Authentication

Login to a registry:

```
podman login docker.io # prompts for username and password interactively
podman login registry.example.com # login to a private registry
```

Credentials are stored in: `${XDG_RUNTIME_DIR}/containers/auth.json`

Logout:

```
podman logout docker.io # remove stored credentials for this registry
```

Do not put credentials in shell history. Always let podman login prompt interactively, or use a credentials file:

```
podman login --username myuser --password-stdin docker.io < /run/secrets/registry_password # read password from file
```

For CI environments, use environment variables supported by your registry or pass a pre-created `auth.json` via `--authfile`.

^ Go to TOC

10.9 Image Metadata: ENTRYPOINT vs CMD

Every image has two pieces of default execution config:

- **ENTRYPOINT**: the program that always runs. Cannot be overridden by arguments.
- **CMD**: the default arguments to pass to ENTRYPOINT (or the default command if ENTRYPOINT is not set).

How they combine:

ENTRYPOINT	CMD	podman run myimage runs	podman run myimage foo runs
["/app"]	["--help"]	/app --help	/app foo
(none)	["sh"]	sh	foo
["sh", "-c"]	["echo hi"]	sh -c 'echo hi'	sh -c 'foo'

Inspect these fields:

```
podman image inspect docker.io/library/nginx:stable \
  --format '{{.Config.Entrypoint}} | {{.Config.Cmd}}' # show ENTRYPOINT and CMD
```

Override CMD at runtime:

```
podman run --rm docker.io/library/nginx:stable nginx -v # override CMD: run nginx -v instead of default
```

Override ENTRYPOINT at runtime:

```
podman run --rm --entrypoint sh docker.io/library/nginx:stable -c 'echo hello' # replace ENTRYPOINT with sh
```

Inspect exposed ports:

```
podman image inspect docker.io/library/nginx:stable --format '{{.Config.ExposedPorts}}' # what ports are exposed
```

Note: `ExposedPorts` is documentation — it does not automatically publish ports. You must use `-p` `host:container` to publish.

^ Go to TOC

10.10 Inspecting Image Layers

List layers (history):

```
podman history docker.io/library/nginx:stable # show image layer history with sizes and commands
```

This shows each layer, when it was created, what command produced it, and how large it is.

Why this matters for security: if a RUN step accidentally includes a secret (even if a later step tries to “delete” it), the secret is still in the intermediate layer and visible via `podman history`.

Inspect image manifest:

```
podman manifest inspect docker.io/library/nginx:stable # show OCI manifest (multi-arch info)
```

Diff two containers' filesystems:

```
podman diff mycontainer # show what changed in the writable layer vs the image
```

This is useful after running a container to see what it wrote.

^ Go to TOC

10.11 Lab: Local Tagging

Tags are just aliases. You can create local aliases for any image.

Step 1: Pull an image:

```
podman pull docker.io/library/nginx:stable # pull nginx stable
```

Step 2: Tag it with a local alias:

```
podman tag docker.io/library/nginx:stable localhost/nginx:course # create a local alias
podman images | grep nginx # both entries exist, same image ID
```

Step 3: Run using the local alias:

```
podman run --rm localhost/nginx:course nginx -v # run using the local tag
```

Step 4: Remove the local alias (does not remove the original):

```
podman rmi localhost/nginx:course # untag (removes the alias, not the image if other tags exist)
podman images | grep nginx # original docker.io tag still exists
```

^ Go to TOC

10.12 Lab (Optional): Push to a Local Registry

This teaches the full pull -> build -> tag -> push flow without needing a real external registry.

Step 1: Start a local registry:

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 # start local registry
```

Step 2: Tag and push to the local registry:

```
podman pull docker.io/library/alpine:latest # pull base image
podman tag docker.io/library/alpine:latest localhost:5000/alpine:course # tag for local registry
podman push --tls-verify=false localhost:5000/alpine:course # push to local registry (HTTP, la
```

Note: `--tls-verify=false` is acceptable for a local lab registry on loopback. Never use it for production.

Step 3: Pull from the local registry:

```
podman rmi localhost:5000/alpine:course # remove local copy
podman pull --tls-verify=false localhost:5000/alpine:course # pull from local registry
```

Step 4: Cleanup:

```
podman rm -f registry # stop and remove the local registry container
```

For a production-grade local registry with TLS, use certificates and a proper configuration. The Registry v2 API documentation in Further Reading explains how.

^ Go to TOC

10.13 Lab (Optional): Save and Load

podman save and podman load transfer images as tar archives — useful for air-gapped environments.

```
podman save -o nginx.tar docker.io/library/nginx:stable  # export image to tar file
ls -lh nginx.tar                                         # check file size
podman rmi docker.io/library/nginx:stable                # remove local image
podman load -i nginx.tar                                 # import from tar file
podman images | grep nginx                               # confirm restored
```

Note: - save/load are file-based transport, not a registry. - The digest of a saved/loaded image is preserved — it is still content-addressed. - Multiple images can be saved in one tar: `podman save -o multi.tar image1 image2`.

^ Go to TOC

10.14 Image Storage on Disk

Rootless Podman stores images at:

```
~/.local/share/containers/storage/overlay/
```

Each layer is a directory of filesystem diffs. The overlay driver stacks them at runtime.

Disk usage:

```
podman system df # summarized view: images, containers, volumes
du -sh ~/.local/share/containers/storage/ # raw disk usage of the storage root
```

Remove all locally cached images (aggressive — forces re-pull everything):

```
podman image prune -a -f # remove all images not used by any container
```

Remove a specific image:

```
podman rmi docker.io/library/nginx:stable # remove by tag
podman rmi sha256:<digest>                 # remove by digest
```

^ Go to TOC

10.15 Checkpoint

- You can explain the difference between a tag and a digest.
- You can explain why `:latest` is risky in production.
- You can find and record an image digest.
- You can explain what ENTRYPOINT and CMD do and how to override them.
- You understand why fully qualified image names matter in automation.

^ Go to TOC

10.16 Quick Quiz

- 1) If you deploy by tag (e.g., `:stable`), what can change without you changing your config?
- 2) What is the advantage of a digest during incident response?
- 3) You run `podman run myimage mycommand`. The image has ENTRYPOINT `["/app"]` and CMD `["--help"]`. What actually runs?
- 4) A colleague says `alpine` and `docker.io/library/alpine:latest` are the same. When might they not be?

- 5) You run `podman history myimage` and see that a `RUN` step near the bottom has a very large size and a secret-looking value in the command. What is the security implication?

^ Go to TOC

10.17 Further Reading

- OCI image spec (tags vs digests context): <https://github.com/opencontainers/image-spec>
- `podman-pull(1)`: <https://docs.podman.io/en/latest/markdown/podman-pull.1.html>
- `podman-image(1)`: <https://docs.podman.io/en/latest/markdown/podman-image.1.html>
- Registries config (`registries.conf`): <https://github.com/containers/image/blob/main/docs/containers-registries.conf.5.md>
- Docker Registry HTTP API V2: <https://distribution.github.io/distribution/spec/api/>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

11 Module 4: Secrets (Local-First)



11.1 Table of Contents

- Learning Goals
- Why Secrets Matter — The Threat Model
- Mental Model: Secret as a Mounted File
- How Podman Secrets Work Internally
- Common Anti-Patterns
- Commands Reference
- Lab A: Create and Mount a Secret
- Lab A2: Prove It Is Not an Env Var
- Lab B: Rotation Pattern (Versioned Secrets)
- Lab C: Custom Mount Target
- Advanced: Build-Time Secrets (Optional)
- File Permissions and App Compatibility
- What Podman Secrets Do NOT Solve
- Common Failure Modes
- Checkpoint
- Quick Quiz
- Further Reading

This module replaces the common beginner pattern of putting passwords in `.env` files and environment variables.

[^ Go to TOC](#)

11.2 Learning Goals

- Explain why environment variables are not a good secret transport.
- Create and use Podman secrets in rootless mode.
- Mount secrets as files and consume them safely.
- Rotate a secret with minimal downtime.
- Build a “no-secrets-in-logs” habit.

[^ Go to TOC](#)

11.3 Why Secrets Matter — The Threat Model

Before choosing a mechanism, you need to understand what you are actually protecting against. Secrets are credentials — database passwords, API keys, TLS private keys, OAuth tokens. The risk is **accidental disclosure** through these vectors:

Exposure vector	How it happens
Shell history	<code>export DB_PASSWORD=hunter2</code> is stored in <code>~/.bash_history</code>
Process listing	<code>ps auxww</code> shows env vars of running processes on many Linux systems
Container inspect	<code>podman inspect <container></code> reveals all env vars in plain text
Image layers	<code>ARG/ENV</code> instructions bake values into image layers permanently

Exposure vector	How it happens
Log aggregation	App or framework logs the full env at startup (Spring Boot, Next.js, etc.)
Committed <code>.env</code> files	<code>.env</code> accidentally committed, pushed, and publicly indexed
Core dumps	A crashing process can dump env vars into a core file

The goal of **secrets as files** is to reduce the surface area of every row in that table.

flowchart TD

```

A["Developer creates secret<br/>via podman secret create"] --> B["Podman stores blob<br/>in XDG_DATA_DIRS"]
B --> C["Container starts<br/>with --secret name"]
C --> D["Podman mounts tmpfs<br/>at /run/secrets/"]
D --> E["App reads file<br/>at /run/secrets/name"]
E --> F["Secret value in<br/>process memory only"]

```

```

style A fill:#1a1a2e,color:#e0e0e0
style B fill:#16213e,color:#e0e0e0
style C fill:#0f3460,color:#e0e0e0
style D fill:#533483,color:#e0e0e0
style E fill:#0f3460,color:#e0e0e0
style F fill:#16213e,color:#e0e0e0

```

The secret value never appears in: - The image layer - `podman inspect` env output - `ps auxww` - Shell history (if you used `printf` or a file)

^ Go to TOC

11.4 Mental Model: Secret as a Mounted File

Think of a Podman secret as a **named slot** that holds an opaque blob. When a container starts with `--secret name`, Podman:

1. Looks up the blob by name in the local secrets store.
2. Writes it to an in-memory `tmpfs` mount at `/run/secrets/` inside the container.
3. The file disappears when the container stops — it is never written to the writable container layer.

graph LR

```

subgraph "Host"
    S["Podman secrets store<br/>~/.local/share/containers/<br/>storage/secrets/"]
end
subgraph "Container filesystem"
    R["/run/secrets/<br/>(tmpfs, in-memory)"]
    F["/run/secrets/db_password<br/>(file, mode 0444)"]
end
S -->|"mount at start"| R
R --> F
APP["Application process"] -->|"open() read()"| F

```

Key insight: the secret exists as a file for the lifetime of the container process and is gone when the container exits. It is never persisted in the container's writable layer.

^ Go to TOC

11.5 How Podman Secrets Work Internally

Podman's local secrets driver stores blobs at:

```
~/.local/share/containers/storage/secrets/
```

Each secret is a JSON metadata record plus the encrypted (or plain, depending on driver) blob. The default driver is `file`, which stores blobs base64-encoded on disk. This means:

- Secrets are local to the machine.
- They are only as secure as the filesystem permissions on `~/.local/share/containers/`.
- There is no built-in encryption at rest with the default driver (though you can add one via a custom driver).

The `podman secret inspect` command shows metadata (name, ID, driver, timestamps) but never the value.

```
podman secret inspect db_password # shows metadata, not value
```

For multi-host or encrypted-at-rest requirements, see Module 90 (External Secrets Survey).

^ Go to TOC

11.6 Common Anti-Patterns

These patterns are common in tutorials and dangerous in real environments:

flowchart TD

```
A["Anti-pattern: ENV var<br/>-e DB_PASSWORD=hunter2"] -->|"visible in"| B["podman inspect<br/>ps au  
C["Anti-pattern: ARG in Dockerfile<br/>ARG DB_PASSWORD"] -->|"baked into"| D["Image layer<br/>(dock  
E["Anti-pattern: .env file<br/>committed to git"] -->|"leaked via"| F["git log / GitHub<br/>public  
G["Good pattern: --secret name"] -->|"mounted as"| H["run/secrets/name<br/>(tmpfs, not in inspect)"]
```

```
style A fill:#3d0000,color:#ffcccc
```

```
style C fill:#3d0000,color:#ffcccc
```

```
style E fill:#3d0000,color:#ffcccc
```

```
style G fill:#003d00,color:#ccffcc
```

Summary of what to avoid:

- `export DB_PASSWORD=...` in your shell
- putting passwords in `.env` and committing it
- `podman run -e DB_PASSWORD=...` for anything beyond a throwaway lab
- `ARG/ENV` in a `Containerfile` for secret material
- logging connection strings that contain credentials

^ Go to TOC

11.7 Commands Reference

Create a secret from `stdin` (avoids shell history):

```
printf '%s' 'correct-horse-battery-staple' | podman secret create db_password - # create secret from s
```

Create a secret from a file (file must have tight permissions):

```
chmod 600 ./db_password.txt # restrict read to owner only
```

```
podman secret create db_password ./db_password.txt # create secret from file
```

List secrets (metadata only, no values):

```
podman secret ls # list all secrets with metadata
```

Inspect a secret (shows driver, creation time — never the value):

```
podman secret inspect db_password # inspect secret metadata
```

Remove a secret:

```
podman secret rm db_password # delete the secret by name
```

Use a secret at runtime (default mount path /run/secrets/<name>):

```
podman run --rm --secret db_password docker.io/library/busybox:latest \
  sh -lc 'test -f /run/secrets/db_password && echo "secret file present"' # verify secret is mounted
```

Override the mount target inside the container:

```
podman run --rm --secret db_password,target=/etc/myapp/db.pass \
  docker.io/library/busybox:latest \
  sh -lc 'test -f /etc/myapp/db.pass && echo OK' # custom mount path
```

Change uid/gid/mode on the mounted file:

```
podman run --rm --secret db_password,uid=1000,gid=1000,mode=0400 \
  docker.io/library/busybox:latest \
  sh -lc 'ls -la /run/secrets/db_password' # inspect ownership and permissions
```

Notes:

- Keep the secret value out of your shell history. Prefer `printf '%s' value | ...` or read from a file.
- Never print secret contents in logs.
- Use `read -s VALUE` in shell scripts when value must come from stdin interactively.

^ Go to TOC

11.8 Lab A: Create and Mount a Secret

- 1) Create a secret from stdin (example password — do not use in production):

```
printf '%s' 'correct-horse-battery-staple' | podman secret create db_password - # print text without t
```

- 2) Confirm the secret appears in the list:

```
podman secret ls # list secrets, expect db_password
```

- 3) Run a container that confirms the secret file exists (without printing the value):

```
podman run --rm --secret db_password docker.io/library/busybox:latest \
  sh -lc 'test -f /run/secrets/db_password && echo OK' # run a container, secret as file
```

- 4) Confirm the secret is NOT visible via `podman inspect`:

```
podman run -d --name secret-demo --secret db_password docker.io/library/busybox:latest sleep 600 # run
podman inspect secret-demo | grep -i password || echo "not in inspect output" # should print 'not in i
podman rm -f secret-demo # cleanup
```

Checkpoint:

- Secret appears as a file at `/run/secrets/db_password`.
- You never printed the secret value.
- `podman inspect` does not expose the value.

^ Go to TOC

11.9 Lab A2: Prove It Is Not an Env Var

This lab builds the “prove it” habit — always verify your security assumptions experimentally.

- 1) Start a long-lived container with the secret:

```
podman run -d --name secret-demo --secret db_password docker.io/library/busybox:latest sleep 600 # run
```

- 2) Check environment does not contain the secret:

```
podman exec secret-demo sh -lc 'env | grep -i password || echo "no password in env"' # verify secret n
```

Expected output: no password in env

3) Confirm the file exists with restricted permissions:

```
podman exec secret-demo sh -lc 'ls -la /run/secrets' # list secret files
```

Expected: file owned by root (uid 0) inside the container, mode 0444 by default.

4) Read the file (confirm you can):

```
podman exec secret-demo sh -lc 'wc -c /run/secrets/db_password' # count bytes, don't print value
```

5) Cleanup:

```
podman rm -f secret-demo # stop and remove the demo container
```

^ Go to TOC

11.10 Lab B: Rotation Pattern (Versioned Secrets)

Use versioned names so you can run old and new versions in parallel during a deploy window.

1) Create two versions of the secret:

```
printf '%s' 'v1-value' | podman secret create db_password_v1 - # create version 1
printf '%s' 'v2-value' | podman secret create db_password_v2 - # create version 2
```

2) Start v1 service:

```
podman run -d --name app-v1 \
  --secret db_password_v1,target=db_password \
  docker.io/library/busybox:latest \
  sh -lc 'echo started-v1; sleep 3600' # run v1 with secret mounted as db_password
```

3) Verify v1 is using its secret:

```
podman exec app-v1 sh -lc 'test -f /run/secrets/db_password && echo v1-secret-present' # verify
```

4) Roll out v2 (new container, new secret name, same mount target):

```
podman run -d --name app-v2 \
  --secret db_password_v2,target=db_password \
  docker.io/library/busybox:latest \
  sh -lc 'echo started-v2; sleep 3600' # run v2 with new secret mounted at same path
```

5) Verify v2 is healthy, then stop v1:

```
podman logs app-v2 # confirm started
podman rm -f app-v1 # stop old version
```

6) Only now remove the old secret:

```
podman secret rm db_password_v1 # delete old secret after rollback window has passed
```

7) Cleanup:

```
podman rm -f app-v2
podman secret rm db_password_v2
```

```
sequenceDiagram
    participant Ops
    participant Podman
    participant AppV1
    participant AppV2
```

```

Ops->>Podman: secret create db_password_v2
Ops->>Podman: run app-v2 --secret db_password_v2
Podman->>AppV2: mount /run/secrets/db_password (v2 value)
Note over AppV1,AppV2: Both versions running in parallel
Ops->>AppV2: verify healthy
Ops->>AppV1: stop and remove
Note over Ops: Rollback window closed
Ops->>Podman: secret rm db_password_v1

```

Guideline:

- Keep the old secret around until rollback is no longer needed.
- Many apps only read secrets on startup — rotation means deploying a new container.
- In Quadlet-managed services, this means updating **Secret=** in the unit file and restarting.

^ Go to TOC

11.11 Lab C: Custom Mount Target

Some apps expect credentials at a specific path (e.g., `/etc/app/config/db.pass`). Use the `target=` option.

1) Create the secret:

```
printf '%s' 'mydbpass' | podman secret create db_password - # create secret
```

2) Mount at a custom path:

```
podman run --rm \
--secret db_password,target=/etc/myapp/db.pass \
docker.io/library/busybox:latest \
sh -lc 'ls -la /etc/myapp/db.pass && echo mounted' # verify custom path
```

3) Mount with specific ownership (for apps running as non-root UID inside container):

```
podman run --rm \
--secret db_password,target=/etc/myapp/db.pass,uid=1001,mode=0400 \
docker.io/library/busybox:latest \
sh -lc 'ls -la /etc/myapp/db.pass' # verify ownership and mode
```

Cleanup:

```
podman secret rm db_password # remove secret
```

^ Go to TOC

11.12 Advanced: Build-Time Secrets (Optional)

Goal: authenticate to a private resource during image build without leaking tokens into image layers.

The key rule: **never use ARG or ENV for secret material** — both are baked into image layers and visible via `podman history`.

flowchart LR

```
subgraph "WRONG: ARG bakes secret into layer"
```

```
W1["ARG TOKEN"] --> W2["RUN pip install --extra-index-url<br/>https://user:$TOKEN@..."]
```

```
W2 --> W3["Layer stored with TOKEN<br/>visible in podman history"]
```

```
end
```

```
subgraph "RIGHT: --secret mount"
```

```
R1["--secret id=token,src=./token.txt"] --> R2["RUN --mount=type=secret,id=token<br/>TOKEN=$(cat
```

```

    R2 --> R3["Layer has NO token<br/>secret was tmpfs only"]
end

```

If your Podman/Buildah supports build secrets, use `--mount=type=secret`:

```

# In your Containerfile:
RUN --mount=type=secret,id=pip_token \
    pip install --extra-index-url \
        "https://user:$(cat /run/secrets/pip_token)@pypi.internal/" \
        my-private-package

```

Build command:

```
podman build --secret id=pip_token,src=./pip_token.txt -t myapp . # build with secret, token not in la
```

If your version does not support build secrets:

- Do NOT work around it by embedding secrets with ARG.
- Fetch private dependencies in CI and copy artifacts into the build context instead.
- Use a multi-stage build to discard intermediate layers.

Check your Podman version supports it:

```
podman build --help | grep secret # check if --secret flag is available
```

^ Go to TOC

11.13 File Permissions and App Compatibility

The default secret mount is: - Path: `/run/secrets/<name>` - Owner: `root:root` (uid 0, gid 0) inside the container - Mode: `0444` (world-readable)

If your app runs as a non-root user, it can still read a `0444` file. But if it needs `0400` (only owner can read), you must explicitly set uid and mode:

```
podman run --secret db_password,uid=1001,mode=0400 ... # restrict to uid 1001 only
```

If your app expects an env var (legacy app you cannot modify), you can read the file in your entrypoint:

```

#!/bin/sh
# entrypoint.sh - bridge from file to env var (last resort)
export DB_PASSWORD=$(cat /run/secrets/db_password)
exec "$@"

```

This is a last resort. If you control the app, prefer native file reads.

Common app secret file conventions:

Stack	How to read a secret file
Node.js	<code>fs.readFileSync('/run/secrets/db_password', 'utf8').trim()</code>
Python	<code>open('/run/secrets/db_password').read().strip()</code>
Go	<code>os.ReadFile("/run/secrets/db_password")</code>
Shell script	<code>DB_PASS=\$(cat /run/secrets/db_password)</code>
Java / Spring	Use <code>spring.datasource.password=file:/run/secrets/db_password</code>

^ Go to TOC

11.14 What Podman Secrets Do NOT Solve

Be honest about the limitations to avoid false confidence:

Problem	Does Podman secrets help?
Accidental env var exposure	[OK] Yes — keeps secret out of env
Leaking to image layers	[OK] Yes — secrets are runtime-only
Shell history exposure	[OK] Yes — if you use printf/file input
Encryption at rest on disk	[X] No — default driver stores base64 on disk
Multi-host secret distribution	[X] No — secrets are per-machine
Automatic rotation	[X] No — you must manually rotate
Access control between users	[X] No — relies on filesystem permissions
Audit logging of secret reads	[X] No — no built-in audit trail

For the [X] rows, see Module 90 (External Secrets Survey) for HashiCorp Vault, AWS SSM, and systemd credentials patterns.

^ Go to TOC

11.15 Common Failure Modes

App expects an env var but you mounted a file. - Fix: update the app to read from a file, or use an entrypoint bridge script.

File permissions don't match what the app's UID needs. - Symptom: `Permission denied` reading `/run/secrets/name`. - Fix: add `uid=<UID>,mode=0400` to the `--secret` flag.

Trailing newline in the secret value breaks passwords. - Symptom: authentication fails with correct-looking password. - Cause: `echo 'value' | podman secret create ...` adds a newline. - Fix: always use `printf '%s' 'value' | ...` (no newline).

Secret not available because name was misspelled. - Symptom: container fails to start with “secret not found”. - Fix: `podman secret ls` to verify the name, check `Secret=` spelling in Quadlet unit.

You accidentally log the secret during debugging. - Habit: never `cat /run/secrets/...` in a script that runs in production. Use `wc -c` to verify presence/size without printing.

Build-time secret leaked into image layer via ARG. - Fix: switch to `--mount=type=secret` in the RUN step, never ARG for secrets.

^ Go to TOC

11.16 Checkpoint

- You can create a secret without the value appearing in shell history.
- You can mount a secret as a file and verify it is present without printing it.
- You can explain why `--secret` is safer than `-e`.
- You can describe a versioned rotation plan with rollback.
- You understand what Podman secrets do and do not protect against.

^ Go to TOC

11.17 Quick Quiz

- 1) A colleague runs `podman run -e DB_PASSWORD=hunter2 myapp`. What are three ways this value could be exposed accidentally?

- 2) You have an app that only reads credentials on startup. You rotate `db_password_v1` -> `db_password_v2`. What must you do to the running container?
- 3) You run `echo 'mypass' | podman secret create db_password -`. Later the app fails to authenticate even though the password looks correct. What is the likely cause?
- 4) What does `podman secret inspect db_password` show, and what does it NOT show?
- 5) Your app runs as UID 1001 inside the container and needs to read the secret. The default mount uses UID 0 / mode 0444. Can the app read it? Why?

^ Go to TOC

11.18 Further Reading

- `podman-secret(1)`: <https://docs.podman.io/en/latest/markdown/podman-secret.1.html>
- OWASP Secrets Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- systemd credentials (service-provisioned files): <https://www.freedesktop.org/software/systemd/man/latest/systemd.exec.html#CredentialStore>
- Kubernetes Secrets (base64 caveat context): <https://kubernetes.io/docs/concepts/configuration/secret/>
- Buildah build secrets: <https://buildah.io/blogs/2018/09/14/new-stream-builds.html>
- Module 90: External Secrets Survey (HashiCorp Vault, AWS SSM)

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

12 Module 5: Storage (Volumes, Bind Mounts, Permissions)



12.1 Table of Contents

- Learning Goals
- The Three Storage Types
- Volumes
- Bind Mounts
- tmpfs Mounts
- Inspect Mounts
- Rootless Permission Pitfalls
- podman unshare — Your Permission Debugging Tool
- SELinux Drill (Fedora/RHEL)
- Lab: Persistent DB Data
- Volume Backup and Restore Pattern
- Checkpoint
- Quick Quiz
- Further Reading

[^ Go to TOC](#)

12.2 Learning Goals

- Choose between volumes, bind mounts, and tmpfs for the right use case.
- Use volumes for persistent state across container replacements.
- Debug rootless permission issues using `podman unshare`.
- Understand SELinux labeling (:Z vs :z) on Fedora/RHEL.
- Back up and restore named volumes.

[^ Go to TOC](#)

12.3 The Three Storage Types

graph TD

```
subgraph "Storage types"
    WL["Writable layer<br/>(per-container, automatic)"]
    V["Named volume<br/>(Podman-managed path)"]
    B["Bind mount<br/>(you specify host path)"]
    T["tmpfs<br/>(in-memory, ephemeral)"]
end
WL -->|"destroyed on podman rm"| X1["Data gone"]
V -->|"survives container rm"| X2["Data persists"]
B -->|"tied to host path"| X3["Shared with host"]
T -->|"in-memory only"| X4["Gone on container stop"]
```

Decision guide:

Need	Use
Database data, state that must survive container replacement	Named volume
Config file, certificate, source code (dev)	Bind mount
Scratch space that must not persist (tmp, cache, PID files)	<code>--tmpfs</code>

Need	Use
Read-only root FS with writable exceptions	<code>--read-only --tmpfs /path</code>
Quick experiment, throwaway data	Writable layer (accept data loss)

The core rule: Never store important data in the writable layer. Use a volume.

^ Go to TOC

12.4 Volumes

Named volumes are managed by Podman. You do not need to know or care where they live on disk. Podman handles permissions correctly for rootless use.

Create and inspect:

```
podman volume create dbdata           # create a named volume
podman volume ls                      # list all volumes
podman volume inspect dbdata          # show metadata: mountpoint, driver, labels
```

Use a volume:

```
podman run --rm -v dbdata:/data docker.io/library/alpine:latest \
  sh -lc 'echo hi > /data/x && echo wrote' # write to volume
podman run --rm -v dbdata:/data docker.io/library/alpine:latest \
  sh -lc 'cat /data/x'                    # read from same volume
```

Volume survives container removal:

```
podman run --name writer -v dbdata:/data docker.io/library/alpine:latest \
  sh -lc 'echo persistent > /data/hello' # write
podman rm writer                        # remove container
podman run --rm -v dbdata:/data docker.io/library/alpine:latest \
  cat /data/hello                       # data is still there
```

Remove a volume (!! DATA LOSS):

```
podman volume rm dbdata # delete the volume and all its data
```

Where volumes live on disk (rootless):

```
podman volume inspect dbdata --format '{{.Mountpoint}}' # typically ~/.local/share/containers/storage/
```

^ Go to TOC

12.5 Bind Mounts

A bind mount maps a **host filesystem path** into the container. The container sees and modifies real host files.

When to use:

- Passing configuration files into a container.
- Development mode — mounting source code so the container sees changes live.
- Passing certificates or read-only data that lives in a known host location.

Basic bind mount:

```
mkdir -p ./mnt-demo # create host directory
echo "hello from host" > ./mnt-demo/hi.txt # create a test file
podman run --rm -v ./mnt-demo:/mnt:Z \
  docker.io/library/alpine:latest cat /mnt/hi.txt # :Z required on SELinux systems
```

Read-only bind mount:

```
podman run --rm -v ./mnt-demo:/mnt:Z,ro \
  docker.io/library/alpine:latest \
  sh -lc 'cat /mnt/hi.txt; echo nope > /mnt/x || echo "write blocked"' # write attempt fails
```

SELinux mount options (Fedora/RHEL):

Option	Meaning	Use when
:Z	Relabel for private use (this container only)	Single container accessing the path
:z	Relabel for shared use (multiple containers)	Multiple containers sharing the same path
(none)	No relabelling	Volume mounts (handled automatically)

When NOT to use bind mounts:

- Long-running services in production (use volumes instead).
- When you need Podman to manage permissions (volumes handle rootless UID mapping).
- When portability matters — bind mounts require the host path to exist.

See Module 11 for the rule: **always ask before using bind mounts in production Quadlet units.**

^ Go to TOC

12.6 tmpfs Mounts

tmpfs mounts provide in-memory, ephemeral writable directories inside a container. They are gone when the container stops — no data persists.

Use cases:

- /tmp for scratch space (prevents writing to read-only root FS).
- /var/run for PID files (nginx, databases write here at startup).
- /var/cache/nginx for nginx's cache directory.
- Any directory an app writes to transiently, that you do not want persisted.

```
podman run --rm \
  --read-only \
  --tmpfs /tmp \
  --tmpfs /var/run \
  docker.io/library/alpine:latest \
  sh -lc 'touch /tmp/scratch && echo ok' # /tmp is writable, root FS is not
```

Size limit on tmpfs:

```
podman run --rm --tmpfs /tmp:size=64m \
  docker.io/library/alpine:latest \
  sh -lc 'df -h /tmp' # tmpfs with 64 MB limit
```

^ Go to TOC

12.7 Inspect Mounts

View mounts on a running container:

```
podman run -d --name mount1 \
  -v dbdata:/data \
  -v ./mnt-demo:/mnt:Z \
  --tmpfs /tmp \
  docker.io/library/alpine:latest sleep 300 # start container with multiple mounts

podman inspect mount1 --format '{{json .Mounts}}' # show all mount definitions as JSON
```

Simpler view:

```
podman inspect mount1 \
  --format '{{range .Mounts}}Type={{.Type}} Source={{.Source}} Dest={{.Destination}} Mode={{.Mode}}{{"\n"}}
```

Cleanup:

```
podman rm -f mount1 # remove the container
```

^ Go to TOC

12.8 Rootless Permission Pitfalls

The most common storage problem in rootless Podman is **permission denied** when a container tries to write to a mounted path.

flowchart TD

```
A["Permission denied on mount"] --> B["What kind of mount?"]
B -->|"Bind mount"| C["Who owns the host dir?"]
C -->|"root"| D["Container (non-root UID) can't write<br/>Fix: podman unshare chown, or use volume"]
C -->|"your UID"| E["SELinux enforcing?"]
E -->|"Yes"| F["Missing :Z label<br/>Fix: add :Z to bind mount flag"]
E -->|"No"| G["Check image UID - container may run as different UID"]
B -->|"Named volume"| H["First use of volume?"]
H -->|"Yes"| I["Image's VOLUME init may need to run<br/>Let the container start once to initialize"]
H -->|"No"| J["Check if previous data has wrong UID<br/>Use podman unshare ls -la <mountpoint>"]
```

Check what UID the container process runs as:

```
podman run --rm docker.io/library/alpine:latest id # show UID inside container
```

Check what that UID maps to on the host:

```
podman unshare id # show UID as the user namespace sees it on the host
```

^ Go to TOC

12.9 podman unshare — Your Permission Debugging Tool

`podman unshare` runs a command inside your **user namespace** — the same UID mapping that container processes use. This lets you see and fix filesystem ownership from the container's perspective.

See UID mapping:

```
podman unshare id # shows uid/gid as rootless container sees it
podman unshare cat /proc/self/uid_map # full UID map table
```

Inspect a directory as the container sees it:

```
podman unshare ls -la ~/.local/share/containers/storage/volumes/dbdata/_data # check volume ownership
```

Fix ownership for a bind-mounted path:

```
podman unshare chown -R 1000:1000 ./mnt-demo # set ownership inside user namespace
```

Warning: `chown -R` on a large directory takes time. And the numbers 1000:1000 are container UIDs, not host UIDs. Use `podman unshare` to run the command in the correct namespace.

Do not blindly `chmod 777` — this grants access to everyone on the host. Fix the ownership instead.

^ Go to TOC

12.10 SELinux Drill (Fedora/RHEL)

SELinux enforces label-based access control independent of Unix permissions. A container process has label `container_t` and can only access files labelled `container_file_t` or `svirt_sandbox_file_t`.

Named volumes are automatically labelled correctly. Bind mounts are not — you must tell Podman to relabel them.

Step 1: Try a bind mount WITHOUT labels (expected: fail on SELinux enforcing):

```
mkdir -p ./selinux-test
echo "test" > ./selinux-test/data.txt
podman run --rm -v ./selinux-test:/mnt \
  docker.io/library/alpine:latest cat /mnt/data.txt # may get permission denied
```

Step 2: Fix it with `:Z`:

```
podman run --rm -v ./selinux-test:/mnt:Z \
  docker.io/library/alpine:latest cat /mnt/data.txt # should work now
```

Step 3: Check what `:Z` did to the host path's label:

```
ls -laZ ./selinux-test/ # show SELinux context (requires `ls` with -Z flag)
```

The label should now be `svirt_sandbox_file_t` — the type that containers are allowed to access.

Step 4: Check for SELinux denial messages:

```
ausearch -m avc -ts recent 2>/dev/null | head -20 # show recent SELinux denials
```

Cleanup:

```
rm -rf ./selinux-test # remove test directory
```

^ Go to TOC

12.11 Lab: Persistent DB Data

Goal: demonstrate that container data survives container replacement when using a named volume.

Step 1: Create a named volume:

```
podman volume create mariadb-lab # create persistent volume
```

Step 2: Start MariaDB with the volume:

```
printf '%s' 'labpass123' | podman secret create lab_db_pass - # create secret for password
podman run -d \
  --name mariadb-lab \
  -v mariadb-lab:/var/lib/mysql \
  --secret lab_db_pass \
  -e MARIADB_ROOT_PASSWORD_FILE="/run/secrets/lab_db_pass" \
  docker.io/library/mariadb:11 # start MariaDB with persistent volume
```

Step 3: Wait for the DB to initialise (watch logs):

```
podman logs -f mariadb-lab # watch logs; wait for "ready for connections"
```

Press Ctrl+C when ready.

Step 4: Create a test record:

```
podman exec mariadb-lab mariadb -uroot -p"labpass123" \  
-e "CREATE DATABASE lab; USE lab; CREATE TABLE test (id INT PRIMARY KEY, name VARCHAR(50)); INSERT INTO test VALUES (1, 'test');"
```

Step 5: Remove the container:

```
podman rm -f mariadb-lab # destroy the container (writable layer gone)  
podman volume ls | grep mariadb-lab # volume still exists!
```

Step 6: Start a NEW container with the same volume:

```
podman run -d \  
--name mariadb-lab2 \  
-v mariadb-lab:/var/lib/mysql \  
--secret lab_db_pass \  
-e MARIADB_ROOT_PASSWORD_FILE="/run/secrets/lab_db_pass" \  
docker.io/library/mariadb:11 # start fresh container with the same volume  
podman logs -f mariadb-lab2 # wait for ready
```

Step 7: Verify the data is still there:

```
podman exec mariadb-lab2 mariadb -uroot -p"labpass123" \  
-e "SELECT * FROM lab.test;" # should return: 1 | persistent
```

Cleanup:

```
podman rm -f mariadb-lab2  
podman volume rm mariadb-lab # DATA LOSS - remove the volume  
podman secret rm lab_db_pass
```

^ Go to TOC

12.12 Volume Backup and Restore Pattern

Named volumes can be backed up by running a helper container that reads from the volume and writes a tar file.

Backup:

```
podman run --rm \  
-v mariadb-lab:/data:ro \  
-v ./backups:/backup \  
docker.io/library/alpine:latest \  
tar czf /backup/mariadb-lab-$(date +%Y%m%d).tar.gz -C /data . # backup volume to tar
```

Restore (to a fresh volume):

```
podman volume create mariadb-lab-restore # create destination volume  
podman run --rm \  
-v mariadb-lab-restore:/data \  
-v ./backups:/backup:ro \  
docker.io/library/alpine:latest \  
tar xzf /backup/mariadb-lab-20260326.tar.gz -C /data # restore from tar
```

This is a **logical backup** at the filesystem level. For database-consistent backups, prefer `mysqldump` / `mariadb-dump` over raw volume backups — they handle transactions correctly.

^ Go to TOC

12.13 Checkpoint

- You can choose volume vs bind mount vs tmpfs intentionally.
- You can explain when `:Z` is required on Fedora/RHEL.
- You can use `podman unshare` to inspect and fix permission issues.
- You can demonstrate that volume data survives `podman rm`.
- You know the basic backup/restore pattern for volumes.

^ Go to TOC

12.14 Quick Quiz

- 1) Your container writes to `/var/lib/myapp` but you want the data to survive `podman rm`. What do you do?
- 2) You get `permission denied` reading a bind-mounted directory even though the host permissions look correct. You are on RHEL. What is the likely cause?
- 3) What does `podman unshare ls -la /path` tell you that a plain `ls -la /path` does not?
- 4) You use a named volume for your database and later need to move to a new server. Describe the volume backup/restore workflow.
- 5) `--tmpfs /tmp` and `-v tmpdata:/tmp` both make `/tmp` writable inside the container. What is the key difference?
- 6) You want a config file to be read-only inside the container and the bind mount path must not be modified. What mount option do you add?

^ Go to TOC

12.15 Further Reading

- `podman-volume(1)`: <https://docs.podman.io/en/latest/markdown/podman-volume.1.html>
- Rootless storage and UID mapping: https://github.com/containers/podman/blob/main/docs/tutorials/rootless_tutorial.md
- SELinux mount labeling for containers (RHEL docs): https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/SELinux_with_containers_using_selinux/
- `subuid(5)` and `subgid(5)` (man7): <https://man7.org/linux/man-pages/man5/subuid.5.html>
- `podman-unshare(1)`: <https://docs.podman.io/en/latest/markdown/podman-unshare.1.html>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

13 Module 6: Networking (Ports, DNS, User-Defined Networks)



13.1 Table of Contents

- Learning Goals
- Minimum Path (If You Are Short on Time)
- 1 How Container Networking Works (Mental Model)
- 2 Rootless Networking In Depth
- 3 Port Publishing
- 4 The Default Network vs User-Defined Networks
- 5 Container DNS and Service Discovery
- 6 Connecting Containers to Multiple Networks
- 7 Inspecting Network State
- 8 Network Drivers — Deeper Look
- 9 Network Security Patterns
- 10 Full Lab: Three-Tier Isolated Stack
- 11 Connecting Containers to Pods on a Network
- 12 Networking in Quadlet (systemd) Deployments
- 13 Troubleshooting Networking
- 14 Common Patterns Reference
- Checkpoint
- Quick Quiz
- Further Reading

[^ Go to TOC](#)

13.2 Learning Goals

By the end of this module you will be able to:

- Explain how rootless networking differs from rootful networking and why it matters.
- Publish container ports to the host and verify them.
- Create, inspect, and remove user-defined networks.
- Connect containers to multiple networks simultaneously.
- Use container DNS names for reliable inter-container service discovery.
- Choose the right network driver (bridge, host, none, macvlan) for a given scenario.
- Configure network-level security: isolate backends, expose only what you need.
- Troubleshoot DNS, port, routing, and firewall problems methodically.
- Connect containers across pods and across user-defined networks.
- Understand how networking interacts with Quadlet (systemd) deployments.

[^ Go to TOC](#)

13.3 Minimum Path (If You Are Short on Time)

If you only do a small slice of this module, do these:

- Publish a port and verify with `podman port` (Section 3).
- Create a user-defined network and verify DNS name resolution (Sections 4-5).
- Practice the multi-network isolation pattern (Section 6).
- Run the troubleshooting checklist once (Section 13).

[^ Go to TOC](#)

13.4 1 How Container Networking Works (Mental Model)

Before running commands, build the mental model. Every container gets:

1. **A network namespace** — an isolated network stack with its own interfaces, routes, and firewall rules.
2. **A virtual Ethernet pair (veth)** — one end lives inside the container, the other end connects to a virtual bridge (or the host).
3. **An IP address** assigned from the network's subnet.

When two containers are on the **same user-defined network**, the bridge lets them talk to each other by IP. Podman's embedded DNS resolver makes them also reachable **by name**.

When a container is only on the **default network** (Podman's built-in `podman` bridge), DNS-based discovery is disabled. This is a deliberate design choice — it motivates you to create explicit named networks.

```
flowchart TD
    subgraph "Host OS"
        subgraph "User-Defined Network (appnet)"
            A["Container A<br/>10.89.1.2"] <-->|"bridge + DNS"| B["Container B<br/>10.89.1.3"]
        end
        subgraph "Default Network (podman)"
            C["Container C<br/>10.88.0.2"] ---|"IP only - no DNS"| D["Container D<br/>10.88.0.3"]
        end
        BR1["Linux Bridge<br/>(cni-podman0 / netavark)"]
        BR2["Linux Bridge<br/>(podman0)"]
    end
    A --> BR1
    B --> BR1
    C --> BR2
    D --> BR2
    BR1 -->|"NAT / pasta"| I["Internet"]
    BR2 -->|"NAT / pasta"| I
```

13.4.1 1.1 The Four Network Drivers

Driver	What it does	When to use it
<code>bridge</code>	Virtual L2 bridge; default for user-defined networks	Almost everything
<code>host</code>	Container shares the host network namespace	Low-level tools, benchmarking, rootful only (rootless has caveats)
<code>none</code>	No network interface except loopback	Batch jobs, maximum isolation
<code>macvlan</code>	Container appears as a separate MAC on your LAN	IoT, legacy apps that need a real LAN address

Rootless note: `host` network mode has limited usefulness in rootless Podman because the container still cannot bind privileged ports without extra capability. `macvlan` requires root on most kernels. Stick to `bridge` unless you have a specific reason.

[^ Go to TOC](#)

13.5 2 Rootless Networking In Depth

13.5.1 2.1 User-Mode Networking Helpers

In rootless mode Podman cannot create kernel-level bridges as a normal user. Instead it delegates packet forwarding to a user-space helper:

Helper	Notes
pasta	Newer, faster, preferred on modern distros; fewer quirks with UDP/ICMP
slirp4netns	Older, still common; slower but very portable

```
flowchart LR
    subgraph "Container Namespace"
        C["App Process<br/>eth0: 10.0.2.100"]
    end
    subgraph "User Process (rootless)"
        P["pasta / slirp4netns<br/>(user-space forwarder)"]
    end
    subgraph "Host Network Namespace"
        H["Host Interface<br/>eth0 / wlan0"]
        I["Internet"]
    end
    C -->|"packets via veth"| P
    P -->|"forwarded as host user traffic"| H
    H --> I
```

Check which backend your installation uses:

```
podman info --format '{{.Host.NetworkBackend}}' # show Podman host configuration
```

Check which per-network helper is active:

```
podman info --format '{{.Host.Slirp4NetnsOptions}}' # show Podman host configuration
```

```
podman info --format '{{.Host.PastaOptions}}' # show Podman host configuration
```

You can switch the rootless backend in `~/.config/containers/containers.conf`:

```
[network]
default_rootless_network_cmd = "pasta"
```

13.5.2 2.2 What Rootless Networking Cannot Do (by default)

- Bind ports < 1024 without extra OS configuration.
- Create macvlan / ipvlan adapters (kernel requires CAP_NET_ADMIN).
- Use host network mode and see the real host interfaces in the traditional sense.

13.5.3 2.3 Allowing Privileged Ports for Rootless (When Needed)

Option A — lower the unprivileged port minimum (system-wide, only if you own the machine):

```
sudo sysctl -w net.ipv4.ip_unprivileged_port_start=80 # allow low ports for rootless (system-wide)
# make permanent
echo "net.ipv4.ip_unprivileged_port_start=80" | sudo tee /etc/sysctl.d/99-lowport.conf # persist across
sudo sysctl -p /etc/sysctl.d/99-lowport.conf # apply the persistent config
```

Option B — use a high port and put a reverse proxy (nginx, Caddy) in front. Strongly preferred in production.

Option C — use `systemd` socket activation (covered in Module 11).

[^ Go to TOC](#)

13.6 3 Port Publishing

13.6.1 3.1 Basic Port Mapping

Syntax: `-p <host-port>:<container-port>`

```
podman run -d --name web1 -p 8080:80 docker.io/library/nginx:stable # run a container
curl -sS http://127.0.0.1:8080/ | head # verify HTTP endpoint
```

13.6.2 3.2 Bind to a Specific Host Address

By default `-p 8080:80` listens on all host interfaces (0.0.0.0). To restrict to loopback only:

```
podman run -d --name web-lo -p 127.0.0.1:8080:80 docker.io/library/nginx:stable # run a container
```

To listen on a specific network interface IP:

```
podman run -d --name web-iface -p 192.168.1.100:8080:80 docker.io/library/nginx:stable # run a container
```

This is important for security: a backend service should never be published to 0.0.0.0 when it only needs to be reachable by a local proxy.

13.6.3 3.3 Multiple Port Mappings

```
podman run -d --name multi -p 8080:80 -p 8443:443 docker.io/library/nginx:stable # run a container
```

13.6.4 3.4 UDP Port Mapping

```
podman run -d --name dns-demo -p 5053:53/udp -p 5053:53/tcp docker.io/library/alpine:latest sleep 600
```

13.6.5 3.5 Random Host Port (Ephemeral)

```
podman run -d --name rand-port -p 80 docker.io/library/nginx:stable # run a container
podman port rand-port # see what port was assigned
```

13.6.6 3.6 Inspect Published Ports

Quick view

```
podman port web1 # show published ports
```

Full JSON (useful in scripts)

```
podman inspect web1 --format '{{json .NetworkSettings.Ports}}' # inspect container/image metadata
```

Everything in one JSON dump

```
podman inspect web1 | python3 -m json.tool | grep -A10 '"Ports"' # inspect container/image metadata
```

Cleanup:

```
podman rm -f web1 web-lo web-iface multi rand-port # cleanup containers
```

[^ Go to TOC](#)

13.7 4 The Default Network vs User-Defined Networks

13.7.1 4.1 Why the Default Network Is Not Enough

When you run `podman run` without `--network`, the container joins the default `podman` bridge.

Problems with the default network:

1. **No automatic DNS.** Containers cannot find each other by name.
2. **Shared blast radius.** All containers on the default network can reach each other at the IP level.
3. **No isolation.** A compromised container can attempt connections to any other container on the same bridge.

13.7.2 4.2 Creating a User-Defined Network

```
podman network create appnet # create a network
```

List networks:

```
podman network ls # list networks
```

Expected output includes your new `appnet` plus built-in networks.

Inspect the network (shows subnet, gateway, driver):

```
podman network inspect appnet # inspect a network
```

Key fields to understand:

```
"driver": "bridge"
"subnets": [{ "subnet": "10.89.x.0/24", "gateway": "10.89.x.1" }]
"dns_enabled": true
```

Notice `dns_enabled: true` — this is the key difference from the default network.

13.7.3 4.3 Custom Subnet and Gateway

```
podman network create --subnet 172.28.0.0/24 --gateway 172.28.0.1 myapp-net # create a network
```

Use custom subnets when: - You need deterministic IPs (rare; prefer DNS names instead). - You need to avoid subnet collisions with your VPN or office network.

13.7.4 4.4 Internal Networks (No External Access)

An internal network has no route to the outside world. Containers on it cannot reach the internet.

```
podman network create --internal db-internal # create a network
```

Use this for databases, caches, and any service that has no business reaching the internet.

Verify:

```
podman run --rm --network db-internal docker.io/library/alpine:latest sh -lc 'wget -q0- --timeout=3 http://www.google.com'
```

Expected: connection times out or is refused. That is the intended behavior.

13.7.5 4.5 Remove a Network

```
podman network rm appnet # remove the network
```

You cannot remove a network that has active containers attached. Stop and remove containers first:

```
podman network rm appnet # may fail if containers are running
podman ps --filter network=appnet # find connected containers
```

```
podman rm -f $(podman ps -q --filter network=appnet) # force remove connected containers
podman network rm appnet # remove a network
```

^ Go to TOC

13.8 5 Container DNS and Service Discovery

13.8.1 5.1 How It Works

Podman runs an embedded DNS resolver (backed by **aardvark-dns** on modern versions). When `dns_enabled: true` on a network:

- Every container on that network is registered with its **container name** and any **network aliases**.
- DNS queries inside containers are answered by the Podman DNS resolver.
- The resolver is reachable at the network gateway address (usually the first usable IP on the subnet).

13.8.2 5.2 Basic DNS Lab

```
podman network create testdns # create a network
```

```
# Start a named container
```

```
podman run -d --name server-a --network testdns docker.io/library/alpine:latest sleep 600 # run a container
```

```
# From another container, resolve the name
```

```
podman run --rm --network testdns docker.io/library/alpine:latest sh -lc 'getent hosts server-a' # run a container
```

Expected output: an IP address followed by `server-a`.

Test TCP connectivity:

```
podman run --rm --network testdns docker.io/library/alpine:latest sh -lc 'nc -zv server-a 80 2>&1 || echo fail'
```

13.8.3 5.3 Network Aliases

An alias lets you give a container an **additional DNS name** on a specific network. This is useful for:

- Running multiple containers that all answer as `db` (blue/green rotation).
- Giving a container a short service name regardless of its actual container name.

```
podman network create alias-demo # create a network
```

```
podman run -d --name primary-db --network alias-demo --network-alias db docker.io/library/alpine:latest sleep 600
```

```
# Resolve by alias
```

```
podman run --rm --network alias-demo docker.io/library/alpine:latest sh -lc 'getent hosts db' # run a container
```

Both the container name (`primary-db`) and the alias (`db`) resolve to the same IP.

13.8.4 5.4 Multiple Containers Sharing an Alias (Load-Balancing Pattern)

When multiple containers share the same alias on a network, DNS returns **all IPs** (round-robin).

```
podman network create lb-demo # create a network
```

```
podman run -d --name app-1 --network lb-demo --network-alias app docker.io/library/alpine:latest sleep 600
```

```
podman run -d --name app-2 --network lb-demo --network-alias app docker.io/library/alpine:latest sleep 600
```

```
# Resolve - you may see both IPs
```

```
podman run --rm --network lb-demo docker.io/library/alpine:latest sh -lc 'for i in 1 2 3 4; do getent h
```

```
# Cleanup
```

```
podman rm -f app-1 app-2 # cleanup containers
```

```
podman network rm lb-demo # remove the network
```

This is primitive load balancing. For production you want a real load balancer in front. But the DNS pattern is real.

13.8.5 5.5 Custom DNS Servers

Override the DNS server used inside a container (useful on corporate networks or when using a split-horizon DNS):

```
podman run --rm --dns 1.1.1.1 docker.io/library/alpine:latest sh -lc 'cat /etc/resolv.conf' # run a co
```

Add DNS search domains:

```
podman run --rm --dns-search corp.example.com docker.io/library/alpine:latest sh -lc 'cat /etc/resolv.c
```

Set a custom /etc/hosts entry:

```
podman run --rm --add-host myservice:10.0.1.50 docker.io/library/alpine:latest sh -lc 'getent hosts mys
```

[^] Go to TOC

13.9 6 Connecting Containers to Multiple Networks

A container can be a member of more than one network simultaneously. This is the correct way to build a tiered architecture:

```
[internet] -> [frontend-net] -> [app] -> [backend-net] -> [db]
```

- app is on both frontend-net and backend-net.
- db is only on backend-net.
- frontend is only on frontend-net.

flowchart LR

```
I["Internet"] -->|"port 8080"| FE["frontend<br/>(nginx proxy)<br/>frontend-net only"]
subgraph "frontend-net"
    FE
    APP["app<br/>(API server)<br/>both networks"]
end
subgraph "backend-net (--internal)"
    APP
    DB["db<br/>(database)<br/>backend-net only"]
end
FE -->|"HTTP -> api alias"| APP
APP -->|"SQL -> db name"| DB
DB -. "NO outbound" .-> I
```

13.9.1 6.1 Multi-Network Example

```
podman network create frontend-net # create a network
```

```
podman network create backend-net # create a network
```

```
# DB: only on backend
```

```
podman run -d --name db --network backend-net docker.io/library/alpine:latest sleep 600 # run a contain
```

```

# App: on both networks
podman run -d --name app --network frontend-net docker.io/library/alpine:latest sleep 600 # run a container

# Connect app to backend AFTER it is running
podman network connect backend-net app # attach a container to a network

# Frontend: only on frontend
podman run -d --name frontend --network frontend-net docker.io/library/alpine:latest sleep 600 # run a container

# Verify: frontend can reach app
podman exec frontend sh -lc 'getent hosts app' # run a command in a running container

# Verify: frontend CANNOT reach db (different network)
podman exec frontend sh -lc 'getent hosts db || echo "NOT REACHABLE"' # run a command in a running container

# Verify: app CAN reach db
podman exec app sh -lc 'getent hosts db' # run a command in a running container

# Cleanup
podman rm -f db app frontend # stop and remove containers
podman network rm frontend-net backend-net # remove networks

```

13.9.2 6.2 Disconnect from a Network Without Stopping

```
podman network disconnect backend-net app # detach a container from a network
```

Verify the container no longer has the interface:

```
podman exec app ip addr # run a command in a running container
```

Reconnect:

```
podman network connect backend-net app # attach a container to a network
```

[^ Go to TOC](#)

13.10 7 Inspecting Network State

13.10.1 7.1 List All Networks

```
podman network ls # list networks
```

13.10.2 7.2 Detailed Network Info

```
podman network inspect appnet # inspect a network
```

Shows: driver, subnets, gateways, connected containers, DNS state.

13.10.3 7.3 Which Network Is a Container On?

```
podman inspect <name> --format '{{json .NetworkSettings.Networks}}' # inspect container/image metadata
```

Or see all networks and their connected containers:

```
podman network inspect appnet --format '{{json .Containers}}' # inspect a network
```

13.10.4 7.4 Show Container IP Address

```
podman inspect <name> --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' # inspect container
```

For multi-network containers:

```
podman inspect app --format '{{range $name, $net := .NetworkSettings.Networks}}{{$name}}: {{$net.IPAddress}} {{end}}'
```

13.10.5 7.5 View Interfaces Inside a Running Container

```
podman exec <name> ip addr # run a command in a running container
```

```
podman exec <name> ip route # run a command in a running container
```

```
podman exec <name> cat /etc/resolv.conf # run a command in a running container
```

13.10.6 7.6 Host-Side View

On the host, Podman bridge networks appear as podman prefixed virtual bridges:

```
ip link show type bridge # show network links
```

```
ip addr show # show interfaces
```

[^ Go to TOC](#)

13.11 8 Network Drivers — Deeper Look

13.11.1 8.1 Bridge (Default)

```
podman network create --driver bridge mybridge # create a network
```

Characteristics: - Creates a Linux bridge on the host. - Uses NAT (masquerade) for outbound traffic. - Containers get private IPs; host reaches them via the bridge.

13.11.2 8.2 None (No Networking)

```
podman run --rm --network none docker.io/library/alpine:latest ip addr # run a container
```

Only lo (loopback) is present. Useful for: - Batch jobs that need complete network isolation. - Security-sensitive workloads that must never dial out.

13.11.3 8.3 Host (Rootful Only — with Caveats)

Note: limited usefulness in rootless mode

```
podman run --rm --network host docker.io/library/alpine:latest ip addr # run a container
```

The container sees the host's network interfaces directly. There is no NAT, no port mapping needed. Avoid this in production rootless workloads.

13.11.4 8.4 macvlan (Requires Root or Capabilities)

rootful or with NET_ADMIN capability only

```
podman network create --driver macvlan --opt parent=eth0 --subnet 192.168.1.0/24 --gateway 192.168.1.1
```

The container appears as a distinct host on your physical LAN. Useful for legacy protocols (DHCP from upstream, mDNS, etc.).

[^ Go to TOC](#)

13.12 9 Network Security Patterns

13.12.1 9.1 The Principle: Expose Nothing You Don't Need To

Every port you publish is an attack surface. Every network link you create is a potential pivot point.

Default stance:

- **Databases** -> no port published, internal network only.
- **Caches** -> no port published, internal network only.
- **APIs** -> port published to loopback or internal network, reverse proxy in front.
- **Reverse proxy** -> the only container with a public port.

flowchart TD

```
USER["External User"] -->|"TCP 80/443"| PROXY["Reverse Proxy<br/>public-tier only<br/>port 80:80 published"]
subgraph "public-tier"
    PROXY
end
PROXY -->|"HTTP -> api"| API["API Server<br/>app-tier only<br/>no published port"]
subgraph "app-tier"
    API
end
API -->|"SQL -> db"| DB["Database<br/>data-tier (--internal)<br/>no published port"]
subgraph "data-tier (--internal)"
    DB
    CACHE["Cache<br/>data-tier (--internal)<br/>no published port"]
end
API -->|"Redis -> cache"| CACHE
DB -. "blocked" .-> INET["Internet"]
CACHE -. "blocked" .-> INET
```

13.12.2 9.2 Segment Networks by Trust Zone

```
[public-net]    web / proxy containers only
[app-net]       app containers + proxy
[db-net]        db containers + app
```

The DB is never on public-net. The proxy is never on db-net.

13.12.3 9.3 Combine with --internal Flag

```
podman network create --internal private-db # create a network
podman run -d --name postgres --network private-db -e POSTGRES_PASSWORD=secret docker.io/library/postgres
```

This DB can never initiate outbound connections. It cannot call home, exfiltrate data to an external server, or participate in an outbound botnet.

13.12.4 9.4 Use --network-alias for Service Contracts

Name your services after their role, not their implementation:

```
--network-alias db          # not "postgres-16-container-prod"
--network-alias cache       # not "redis-7.2"
--network-alias api         # not "my-app-v3"
```

When you upgrade a service, you swap the container and preserve the alias. Nothing else needs to change.

13.12.5 9.9 Avoid Publishing to 0.0.0.0 Unnecessarily

```
# Bad for an internal API
-p 8080:8080           # listens on all interfaces

# Better
-p 127.0.0.1:8080:8080 # loopback only
```

^ Go to TOC

13.13 10 Full Lab: Three-Tier Isolated Stack

Build a realistic, isolated three-tier stack:

- **reverse proxy** (nginx): published to host on 8080, on **frontend-net**
- **app** (alpine with netcat): on **frontend-net** and **app-net**
- **db** (alpine simulating a database): on **app-net** only, no published port

13.13.1 Step 1 — Create Networks

```
podman network create frontend-net # create a network
podman network create --internal app-net # create a network
```

13.13.2 Step 2 — Start the “DB”

```
podman run -d --name db --network app-net docker.io/library/alpine:latest sh -lc 'while true; do echo "I
```

13.13.3 Step 3 — Start the “App”

```
podman run -d --name app --network app-net --network-alias api docker.io/library/alpine:latest sleep 60
```

Connect app to the frontend network as well:

```
podman network connect frontend-net app # attach a container to a network
```

13.13.4 Step 4 — Start the Reverse Proxy

```
podman run -d --name proxy --network frontend-net -p 127.0.0.1:8080:80 docker.io/library/nginx:stable
```

13.13.5 Step 5 — Verify Connectivity

App can reach DB:

```
podman exec app sh -lc 'getent hosts db && echo DNS OK' # run a command in a running container
```

Proxy can reach app:

```
podman exec proxy sh -lc 'getent hosts api && echo DNS OK' # run a command in a running container
```

Proxy CANNOT reach DB (different network):

```
podman exec proxy sh -lc 'getent hosts db 2>&1 || echo "ISOLATED: expected"' # run a command in a running container
```

DB CANNOT reach the internet (internal network):

```
podman exec db sh -lc 'wget -qO- --timeout=3 http://example.com 2>&1 || echo "BLOCKED: expected"' # run a command in a running container
```

Host can reach proxy via published port:

```
curl -sSI http://127.0.0.1:8080/ # verify HTTP endpoint
```

13.13.6 Step 6 — Cleanup

```
podman rm -f db app proxy          # stop and remove containers
podman network rm frontend-net app-net # remove networks
```

[^ Go to TOC](#)

13.14 11 Connecting Containers to Pods on a Network

Pods (covered in Module 7) and user-defined networks interact naturally. You can place an entire pod on a named network:

```
podman network create podnet # create a network

podman pod create --name mypod --network podnet -p 8090:80 # create a pod

podman run -d --pod mypod --name pod-nginx docker.io/library/nginx:stable # run a container

# A container outside the pod resolves the pod by its infra container's IP
# or by any container name inside:
podman run --rm --network podnet docker.io/library/alpine:latest sh -lc 'getent hosts pod-nginx' # run

podman pod rm -f mypod # stop and remove the pod and its containers
podman network rm podnet # remove the network
```

[^ Go to TOC](#)

13.15 12 Networking in Quadlet (systemd) Deployments

Quadlet `.network` unit files let you declare Podman networks as systemd-managed resources. This ensures networks exist before containers start.

13.15.1 12.1 Declare a Network Unit

Create `~/.config/containers/systemd/appnet.network`:

```
[Unit]
Description=Application private network

[Network]
Driver=bridge
Internal=true
```

13.15.2 12.2 Reference the Network in a Container Unit

In your `.container` unit file:

```
[Container]
Image=docker.io/library/nginx:stable
Network=appnet.network
```

Systemd will automatically create `appnet` before starting your container and the dependency chain is managed for you.

Full Quadlet networking is covered in Module 11.

^ Go to TOC

13.16 13 Troubleshooting Networking

13.16.1 13.1 Symptom: Container Cannot Reach Another Container by Name

Checklist:

1. Are both containers on the **same user-defined network**? (Not the default podman network.)
2. Is `dns_enabled: true` on that network?
3. Are both containers **running** (not exited)?
4. Are you using the **container name** (or an alias), not the hostname?

flowchart TD

```
S(["DNS resolution fails"]) --> Q1{"Same user-defined<br/>network?"}
Q1 -->|"No"| F1["Fix: podman network connect<br/>OR restart on correct network"]
Q1 -->|"Yes"| Q2{"dns_enabled: true<br/>on network?"}
Q2 -->|"No"| F2["Fix: recreate network<br/>(default networks have dns disabled)"]
Q2 -->|"Yes"| Q3{"Both containers<br/>running?"}
Q3 -->|"No"| F3["Fix: podman start <name>"]
Q3 -->|"Yes"| Q4{"Using container name<br/>or alias?"}
Q4 -->|"No - using hostname"| F4["Fix: use --name, not --hostname<br/>for DNS registration"]
Q4 -->|"Yes"| F5["Run debug sidecar:<br/>podman run --rm --network <net><br/>alpine getent hosts <target>"]
```

Check network membership

```
podman inspect <name> --format '{{json .NetworkSettings.Networks}}' # inspect container/image metadata
```

Verify DNS is enabled on the network

```
podman network inspect <net> --format '{{.DNSEnabled}}' # inspect a network
```

Try a live DNS lookup from a debug container

```
podman run --rm --network <net> docker.io/library/alpine:latest sh -lc 'getent hosts <target-name>' #
```

13.16.2 13.2 Symptom: Cannot Connect Even Though DNS Resolves

DNS working but TCP failing means the service is not listening, is on the wrong port, or there is a firewall rule.

Check if the port is open

```
podman run --rm --network <net> docker.io/library/alpine:latest sh -lc 'nc -zv <target> <port>' # run
```

Check what the container is actually listening on

```
podman exec <target> ss -tlnp # run a command in a running container
```

or

```
podman exec <target> netstat -tlnp # run a command in a running container
```

13.16.3 13.3 Symptom: Port Published But Cannot Reach from Host

Confirm the port mapping

```
podman port <name> # show published ports
```

Confirm the process is listening inside the container

```
podman exec <name> ss -tlnp # run a command in a running container
```

Check host firewall

```
sudo firewall-cmd --list-all # firewalld
```

```

sudo iptables -L -n          # iptables / nftables

# Check the container's host binding
podman inspect <name> --format '{{json .NetworkSettings.Ports}}' # inspect container/image metadata
# Look for "HostIp" - if it's 127.0.0.1, you can only reach from localhost

```

13.16.4 13.4 Symptom: nc or wget Not Available in Container

Use a debug sidecar with networking tools:

```
podman run --rm --network <net> docker.io/library/nicolaka/netshoot:latest curl -v http://<target>:<port>
```

Or use a minimal alpine with a one-liner install:

```
podman run --rm --network <net> docker.io/library/alpine:latest sh -lc 'apk add -q curl && curl -v http
```

13.16.5 13.5 Symptom: Container Cannot Reach the Internet

Verify DNS

```
podman exec <name> sh -lc 'cat /etc/resolv.conf' # run a command in a running container
```

Try pinging a well-known IP (not DNS-dependent)

```
podman exec <name> ping -c3 8.8.8.8 # run a command in a running container
```

Try DNS resolution

```
podman exec <name> sh -lc 'getent hosts example.com' # run a command in a running container
```

Check if the network is internal

```
podman network inspect <net> --format '{{.Internal}}' # inspect a network
```

If the network is `internal: true`, outbound traffic is intentionally blocked.

If DNS fails but the IP works, the problem is your DNS resolver configuration.

13.16.6 13.6 Symptom: Sporadic Connection Failures (Rootless)

This is often a pasta/slirp4netns quirk with UDP under high load, or a port exhaustion issue.

Check for errors in the rootless network helper

```
journalctl --user -u podman.socket # view user-service logs
```

```
podman events --filter type=network # show Podman lifecycle events
```

13.16.7 13.7 Useful Debugging One-Liners

All running container IPs

```
podman ps -q | xargs -I{} podman inspect {} --format '{{.Name}}: {{range .NetworkSettings.Networks}}{{.S
```

All networks and their subnets

```
podman network ls -q | xargs -I{} podman network inspect {} --format '{{.Name}}: {{range .Subnets}}{{.S
```

Which containers are on a given network

```
podman network inspect <net> --format '{{range $id, $c := .Containers}}{{$.Name}} {{end}}' # inspect
```

Container's effective DNS config

```
podman exec <name> cat /etc/resolv.conf # run a command in a running container
```

^ Go to TOC

13.17 14 Common Patterns Reference

13.17.1 Pattern A — Single Shared App Network (Simple Stack)

```
podman network create app # create a network
podman run -d --name db --network app docker.io/library/postgres:16-alpine # run a container
podman run -d --name cache --network app docker.io/library/redis:7-alpine # run a container
podman run -d --name api --network app -p 127.0.0.1:8000:8000 myapp:latest # run a container
podman run -d --name proxy --network app -p 0.0.0.0:80:80 nginx:stable # run a container
```

13.17.2 Pattern B — Segmented Networks (Recommended for Production)

```
podman network create --internal data-tier # create a network
podman network create app-tier # create a network
podman network create public-tier # create a network
```

```
podman run -d --name db --network data-tier postgres:16-alpine # run a container
podman run -d --name cache --network data-tier redis:7-alpine # run a container
podman run -d --name api --network app-tier myapp:latest # run a container
podman network connect data-tier api # api reaches db and cache
```

```
podman run -d --name proxy --network public-tier -p 80:80 nginx:stable # run a container
podman network connect app-tier proxy # proxy reaches api
```

13.17.3 Pattern C — Debug Sidecar (Ephemeral)

```
podman run --rm -it --network <same-net> docker.io/library/alpine:latest sh # run a container
# Now you have a shell inside the network with tools
```

13.17.4 Pattern D — One-Time Migration Container

```
podman run --rm --network app-tier --env-file .env myapp:latest ./migrate.sh # run a container
```

[^ Go to TOC](#)

13.18 Checkpoint

You should be able to answer the following without looking at commands:

- What is the fundamental difference between the default `podman` network and a user-defined network?
- Why does `--internal` improve security for a database network?
- How do containers find each other by name (what Podman feature enables this)?
- When would you use a network alias instead of a container name?
- What flag restricts a port binding to loopback only?
- How do you connect a running container to a second network without restarting it?
- What is the first tool you reach for when a container cannot be found by DNS?

[^ Go to TOC](#)

13.19 Quick Quiz

1. You have two containers on the default `podman` network. Container A tries to `curl http://container-b/`. It fails with “could not resolve host”. What is the most likely cause and fix?

2. You start a database with `-p 5432:5432`. A security reviewer flags this as a problem. Why, and how do you fix it?
3. You have an `app` container that needs to talk to both a `frontend` network and a `backend` network. What is the cleanest way to set this up, and what command do you use to add the second network connection after the container is running?
4. Two containers share the network alias `api`. A third container queries `getent hosts api`. What does it get back? What does this enable?
5. A container can ping `8.8.8.8` but cannot resolve `example.com`. What is the most likely problem?
6. You check `podman port mycontainer` and it shows port 8080 mapped. But `curl http://127.0.0.1:8080/` fails. Name three things to check next.

[^] Go to TOC

13.20 Further Reading

- Podman networking documentation: `man podman-network`
- aardvark-dns project (embedded DNS): <https://github.com/containers/aardvark-dns>
- pasta (rootless networking helper): <https://passt.top/>
- slirp4netns: <https://github.com/rootless-containers/slirp4netns>
- CNI vs Netavark: <https://podman.io/blogs/2022/05/05/podman-rootful-rootless.html>
- nftables and container networking: see Module 13 (Troubleshooting)

[^] Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

14 Module 7: Pods and Sidecars



14.1 Table of Contents

- Learning Goals
- What Is a Pod?
- Why Pods — The Use Cases
- The Infra Container
- Pod vs User-Defined Network — When To Use Each
- The Sidecar Pattern
- Pod Lifecycle and Commands
- Lab: Two Containers, One Pod
- Lab: Log-Shipping Sidecar Pattern
- Pods and Quadlet
- Pods vs Kubernetes Pods
- Pod Lifecycle Notes
- Checkpoint
- Quick Quiz
- Further Reading

Podman pods group multiple containers so they share certain namespaces — most importantly the **network namespace**. Understanding pods unlocks the sidecar pattern and is the conceptual bridge to Kubernetes.

[^ Go to TOC](#)

14.2 Learning Goals

By the end of this module you will be able to:

- Explain what a Podman pod is and how it differs from a user-defined network.
- Describe what the infra container is and why it exists.
- Run a pod with multiple containers sharing localhost.
- Implement a basic sidecar pattern.
- Decide when to use a pod vs when to use a shared network.
- Know how Podman pods relate to Kubernetes pods.

[^ Go to TOC](#)

14.3 What Is a Pod?

A **pod** is a group of containers that share a set of namespaces. In Podman's implementation, the most important shared namespace is the **network namespace**: every container in the pod sees the same network interfaces, the same IP address, and can reach each other on **localhost**.

graph TD

```
subgraph "Pod: webpod"
  INF["Infra container<br/>(pause process)<br/>holds namespaces"]
  NGX["nginx container<br/>listens on :80"]
  SID["sidecar container<br/>reads logs on localhost"]
  INF -->|"shares net namespace"| NGX
end
```

```

    INF -->|"shares net namespace"| SID
    NGX <-->|"communicate via 127.0.0.1"| SID
end
HOST["Host"] -->|"port 8080 -> :80"| INF

```

The key insight: **port publishing is attached to the pod, not to individual containers**. When you publish `-p 8080:80`, that binding lives on the infra container. Any container in the pod that listens on port 80 is reachable at `127.0.0.1:8080` from outside.

^ Go to TOC

14.4 Why Pods — The Use Cases

Pods are useful when:

1. **Sidecar agents need to reach the main process on localhost** — log shippers (Filebeat, Fluent Bit), service mesh proxies (Envoy), metrics exporters, and TLS terminators all benefit from localhost access. They do not need their own published port; they just connect to `127.0.0.1:<service-port>`.
2. **You want a single unit of deployment for a tightly-coupled pair** — if two containers are always deployed together, always stopped together, and always communicate on localhost, a pod models that relationship explicitly.
3. **You are bridging to Kubernetes YAML** — Podman's pod model is intentionally similar to Kubernetes pods. If you plan to export your workload to a cluster, designing with pods now makes the `podman generate kube` output meaningful.

They are **not required** for most workloads. If two services just need to talk to each other over a network and are independently scaled/deployed, a user-defined network is simpler and more flexible.

^ Go to TOC

14.5 The Infra Container

Every Podman pod contains an **infra container** — a minimal container that does almost nothing (runs a pause equivalent process) but exists solely to **hold the shared namespaces**.

Why is this necessary? Linux namespaces are owned by processes. If all containers in a pod shared namespaces directly, the first container to exit would destroy the namespace, killing the network for all other containers. The infra container solves this: it holds the namespace open for the lifetime of the pod, regardless of which application containers start or stop.

```

podman pod create --name webpod -p 8080:80 # create a pod
podman pod ps # list pods - shows 1 container (the infra container)
podman ps --pod # show all containers including the infra container

```

You will see a container named something like `webpod-infra`. It is always present and should be left alone. When you `podman pod rm`, the infra container is removed automatically.

Note: The infra container uses a very small image (`k8s.gcr.io/pause` or a Podman equivalent). It consumes almost no resources. Do not be alarmed by seeing it.

^ Go to TOC

14.6 Pod vs User-Defined Network — When To Use Each

This is a common confusion point. Here is the mental model:

flowchart TD

```
Q1{"Do containers need<br/>to communicate on localhost<br/>(127.0.0.1)?"}
Q1 -->|"Yes"| POD["Use a Pod"]
Q1 -->|"No"| Q2{"Are they<br/>always deployed/stopped<br/>together?"}
Q2 -->|"Yes, tightly coupled"| POD
Q2 -->|"No, independent lifecycle"| NET["Use a user-defined network"]
NET --> DNS["Containers reach each other<br/>by container name via DNS"]
POD --> LOC["Containers reach each other<br/>via 127.0.0.1"]
```

User-defined network is better when: - Services have independent scaling or lifecycle. - Services are run by different teams or deployed at different times. - You need more than a handful of containers (pods with 10 containers get unwieldy). - Service-to-service communication by DNS name is cleaner than localhost.

Pod is better when: - A sidecar needs to proxy, observe, or augment the main process with localhost access. - You are targeting Kubernetes and want YAML that maps cleanly. - You want a single `podman pod rm` to clean up everything atomically.

[^ Go to TOC](#)

14.7 The Sidecar Pattern

A **sidecar** is an auxiliary container that runs alongside a main application container inside the same pod. Because they share a network namespace, the sidecar can:

- Read the main app's logs from a shared volume.
- Proxy traffic to/from the main app on localhost.
- Scrape metrics from the main app's internal port without publishing it externally.
- Perform health checks or certificate rotation.

graph LR

```
subgraph "Pod"
    APP["Main app<br/>port :8080 (internal only)"]
    PROXY["Envoy/nginx sidecar<br/>port :80 (published)"]
    PROXY -->|"localhost:8080"| APP
end
INTERNET["External traffic"] -->|"host:80"| PROXY
```

Common sidecar examples:

Sidecar type	What it does
Log forwarder	Reads log files written by the main app, ships to a central log store
Service mesh proxy	Intercepts all network traffic in/out of the pod
Metrics exporter	Scrapes app metrics and exposes them on a Prometheus endpoint
TLS terminator	Handles TLS, forwards plain HTTP to the app on localhost
Init sidecar	Runs a one-time setup step before the main container starts

[^ Go to TOC](#)

14.8 Pod Lifecycle and Commands

Create a pod with a published port

```
podman pod create --name mypod -p 8080:80 # create a pod
```

Add containers to an existing pod

```
podman run -d --pod mypod --name web docker.io/library/nginx:stable # run a container in a pod
```

```
podman run -d --pod mypod --name sidecar docker.io/library/alpine:latest sleep 3600 # run a sidecar container
```

Inspect the pod

```
podman pod ps # list pods with container counts
```

```
podman pod inspect mypod # detailed pod info including all containers
```

```
podman pod ps --pod # list all containers showing their pod membership
```

Control the pod

```
podman pod stop mypod # stop all containers in the pod
```

```
podman pod start mypod # start all containers in the pod
```

```
podman pod restart mypod # restart all containers in the pod
```

Remove the pod (stops and removes all containers including infra)

```
podman pod rm -f mypod # force stop and remove pod and all its containers
```

Key lifecycle rule: Stopping a pod stops all containers. Removing a pod removes all containers including the infra container. You cannot remove a pod's infra container independently.

^ Go to TOC

14.9 Lab: Two Containers, One Pod

In this lab you will see localhost networking between two containers inside a pod.

Step 1 — Create a pod and publish a port

```
podman pod create --name webpod -p 8080:80 # create a pod
```

Confirm the infra container was created:

```
podman pod ps --pod # list containers showing pod membership
```

You will see the infra container already running even though you have not added application containers yet.

Step 2 — Run an HTTP server inside the pod

```
podman run -d --pod webpod --name nginx docker.io/library/nginx:stable # run nginx inside the pod
```

Verify it is reachable from the host:

```
podman port webpod # show published ports for the pod
```

From the host (or inside a debug shell):

```
podman run --rm --pod webpod docker.io/library/alpine:latest sh -lc 'wget -qO- http://127.0.0.1:80/ | h'
```

What just happened? The Alpine container ran inside the same pod as nginx. Because they share a network namespace, 127.0.0.1:80 inside the Alpine container is nginx's port — exactly as if both processes were running on the same machine.

Step 3 — Add a persistent sidecar for debugging

```
podman run -d --pod webpod --name debug docker.io/library/alpine:latest sleep 3600 # run a debug sidecar
```

Exec into the sidecar and probe nginx without publishing any new ports:

```
podman exec -it debug sh # exec into the sidecar
```

Inside:

```
# Nginx is accessible via localhost - no new port needed
wget -q0- http://127.0.0.1:80/ | head -5 # fetch nginx on localhost
# Check what is listening on port 80
netstat -tlnp 2>/dev/null || ss -tlnp # show listening ports
exit # exit the shell
```

Step 4 — Inspect the shared network

```
podman inspect nginx --format '{{.NetworkSettings.SandboxKey}}' # show nginx's network namespace path
podman inspect debug --format '{{.NetworkSettings.SandboxKey}}' # show sidecar's network namespace path
```

Both containers will show the **same network namespace** — confirming they share it via the infra container.

Step 5 — Cleanup

```
podman pod rm -f webpod # stop and remove the pod and all its containers
```

^ Go to TOC

14.10 Lab: Log-Shipping Sidecar Pattern

This lab simulates a real-world pattern: the main app writes logs to a shared volume, and a sidecar reads them.

Step 1 — Create the pod with a shared volume

```
podman volume create logvol # create a shared log volume
podman pod create --name logpod -p 8081:80 # create the pod
```

Step 2 — Start the app (nginx writing access logs)

```
podman run -d --pod logpod --name app \
-v logvol:/var/log/nginx:Z \
docker.io/library/nginx:stable # run nginx with log volume
```

Step 3 — Start the “log shipper” sidecar

```
podman run -d --pod logpod --name log-shipper \
-v logvol:/logs:Z \
docker.io/library/alpine:latest \
sh -lc 'while true; do echo "--- log snapshot ---"; ls -la /logs/; sleep 5; done' # run a log-reading sidecar
```

Step 4 — Generate some traffic and watch the sidecar

```
# Generate traffic
for i in 1 2 3 4 5; do
  podman run --rm --pod logpod docker.io/library/alpine:latest wget -q0 /dev/null http://127.0.0.1:80/
done
```

```
# Watch sidecar output
podman logs -f log-shipper # follow the sidecar's log output
```

Press Ctrl+C to stop following logs.

Step 5 — Cleanup

```
podman pod rm -f logpod # remove the pod and all containers
podman volume rm logvol # remove the log volume
```

^ Go to TOC

14.11 Pods and Quadlet

You can manage a pod with Quadlet using a `.pod` unit. Containers in the pod reference it with `Pod=`:

```
# ~/.config/containers/systemd/mypod.pod
[Pod]
PodName=mypod
PublishPort=8080:80

# ~/.config/containers/systemd/web.container
[Unit]
Description=Web container in mypod
After=mypod-pod.service

[Container]
Image=docker.io/library/nginx:stable
Pod=mypod.pod

[Service]
Restart=always

[Install]
WantedBy=default.target
```

The generated service for the pod is `mypod-pod.service`. Container units declare `After=` and `Requires=` on it automatically when using `Pod=`.

See Module 11 for full Quadlet details.

^ Go to TOC

14.12 Pods vs Kubernetes Pods

Podman's pod model is deliberately modeled after Kubernetes pods. The key similarities:

Feature	Podman Pod	Kubernetes Pod
Shared network namespace	[OK]	[OK]
Shared IPC namespace	[OK] (optional)	[OK]
Infra/pause container	[OK]	[OK]
Sidecar pattern	[OK]	[OK]
Port published on pod	[OK]	[OK]
Scheduling across nodes	[X]	[OK]
Liveness/readiness probes	Limited	[OK]
Pod auto-restart policies	Via systemd	Via kubelet

The critical difference: **Podman pods run on a single machine**. Kubernetes pods can be scheduled to any node in the cluster. But because the interface is similar, a `podman generate kube mypod` can produce YAML that Kubernetes understands — making Podman pods a useful local prototyping tool.

[^ Go to TOC](#)

14.13 Pod Lifecycle Notes

- **Creating a pod does not start application containers** — only the infra container starts automatically.
- **Stopping a pod stops all containers** — including the infra container.
- **Removing a pod removes the infra container** — you cannot remove the infra container independently.
- **A stopped pod retains its containers** — they can be started again with `podman pod start`.
- **Exiting a container does not stop the pod** — other containers keep running; the pod remains in a degraded but not stopped state.
- **Port conflicts are detected at pod creation time** — not at container start.

[^ Go to TOC](#)

14.14 Checkpoint

Before moving on, confirm you can answer these:

- ☐ I know that all containers in a pod share the same network namespace.
- ☐ I understand why the infra container exists and what happens if it exits.
- ☐ I can describe the sidecar pattern and give two real-world examples.
- ☐ I know when to prefer a user-defined network over a pod.
- ☐ I have run the two-container pod lab and confirmed localhost access between containers.
- ☐ I understand how Podman pods relate to Kubernetes pods.

[^ Go to TOC](#)

14.15 Quick Quiz

1. Why does a pod show an extra container you did not explicitly run?
2. You have nginx running in a pod and want to add a Prometheus metrics exporter sidecar that scrapes nginx's internal `/metrics` endpoint (not published to the host). How does the sidecar reach nginx?
3. When would you prefer a user-defined network over a pod for two communicating services?
4. What happens to the other containers in a pod when one container exits?

[^ Go to TOC](#)

14.16 Further Reading

- `podman-pod(1)`: <https://docs.podman.io/en/latest/markdown/podman-pod.1.html>
- `podman-generate-kube(1)`: <https://docs.podman.io/en/latest/markdown/podman-generate-kube.1.html>
- Kubernetes Pods concept: <https://kubernetes.io/docs/concepts/workloads/pods/>
- Sidecar containers (Kubernetes): <https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/>
- Quadlet pod units: <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

15 Module 8: Building Images (Containerfiles)



15.1 Table of Contents

- Learning Goals
- Minimum Path (If You Are Short on Time)
- 1 Images, Layers, and the Build Mental Model
- 2 `podman build` Fundamentals
- 3 Containerfile Instructions: The Practical Subset
- 4 Lab A: Build a Tiny HTTP Image (Warm-Up)
- 5 Build Context Hygiene (The Most Common Image Leak)
- 6 Running as Non-Root (Image-Level Least Privilege)
- 7 Multi-Stage Builds (Small Images, Fast Builds)
- 8 Caching: Make Rebuilds Fast
- 9 `ARG`, `ENV`, and Configuration
- 10 Secrets and Private Dependencies (Build-Time)
- 11 Labels, Metadata, and Image Introspection
- 12 Tagging, Digests, and Promotion
- 13 Pushing Images to a Registry
- 14 Testing the Image You Built
- 15 Troubleshooting Builds (Common Failures)
- 16 Cleanup: Keep Your Machine Healthy
- 17 Extended Lab: A Small “Real” Service Image
- Checkpoint
- Quick Quiz
- Further Reading

This module teaches you how to build images you can trust in production:

- reproducible enough to debug
- small enough to ship
- least-privilege by default
- free of secrets

The focus is Podman-first: `podman build`, `podman image`, `podman push`.

[^ Go to TOC](#)

15.2 Learning Goals

By the end of this module you will be able to:

- Explain what an image layer is and why instruction order matters.
- Write a Containerfile that is secure-by-default (non-root, minimal writes).
- Build, tag, inspect, run, and push images with Podman.
- Use multi-stage builds and `--target` to keep runtime images small.
- Keep secrets out of build layers and out of the build context.
- Debug common build and runtime failures (missing files, permissions, wrong arch).

[^ Go to TOC](#)

15.3 Minimum Path (If You Are Short on Time)

- Do Lab A (build + run a tiny image).
- Add a `.containerignore` and switch from `COPY . .` to explicit copies.
- Do the non-root lab (Lab B).
- Build a multi-stage image once (Go example or `examples/build/hello-bun`).

[^ Go to TOC](#)

15.4 1 Images, Layers, and the Build Mental Model

An image build is a series of filesystem snapshots.

- Each instruction in a **Containerfile** creates a new **layer**.
- A layer is immutable; rebuilding changes layers above the first changed step.
- `COPY` and `RUN` are the two instructions that most often invalidate caching.

Practical implications:

- Put slow-changing steps early (base image selection, OS packages).
- Put fast-changing steps late (your app source code).
- Keep the build context small so `COPY` does not force expensive rebuilds.

flowchart TD

```
L0["Layer 0<br/>Base image (FROM)"]
L1["Layer 1<br/>RUN apt-get install ..."]
L2["Layer 2<br/>COPY package.json ./"]
L3["Layer 3<br/>RUN bun install"]
L4["Layer 4<br/>COPY src/ ./src"]
L5["Layer 5<br/>USER app / CMD"]

L0 --> L1 --> L2 --> L3 --> L4 --> L5

note1["Changes here<br/>-> invalidates L1..L5"]
note2["Changes here<br/>-> invalidates L4..L5 only"]
note1 -.-> L0
note2 -.-> L4
```

Terminology:

- **Containerfile**: the build recipe (Dockerfile-compatible syntax).
- **Build context**: the directory sent to the builder for `COPY/ADD`.
- **Tag**: a mutable pointer (`myapp:1`, `myapp:latest`).
- **Digest**: immutable content address (`sha256:...`).

[^ Go to TOC](#)

15.5 2 podman build Fundamentals

15.5.1 2.1 Basic Build

```
podman build -t localhost/myapp:1 . # build an image
```

Common flags:

```
podman build -t localhost/myapp:1 -f Containerfile . # build an image
podman build --pull=always -t localhost/myapp:1 . # build an image
```

```
podman build --no-cache -t localhost/myapp:1 . # build an image
podman build --layers -t localhost/myapp:1 . # build an image
podman build --target runtime -t localhost/myapp:1 . # build an image
```

Notes:

- `--pull=always` is useful in CI to ensure you build from the newest base.
- `--no-cache` is useful when debugging, but do not make it your default.
- `--target` builds only a named stage from a multi-stage Containerfile.

15.5.2 2.2 Naming: Why localhost/ Is Used in Labs

Using `localhost/<name>` makes it explicit that the tag is local and not in a remote registry namespace.

```
podman images | head # list images
```

You will see `localhost/myapp:1` locally even if you are not logged into a registry.

15.5.3 2.3 What Builds What

Podman builds are typically executed by Buildah under the hood.

You do not need to become a Buildah expert, but this matters when you search for docs and flags.

[^ Go to TOC](#)

15.6 3 Containerfile Instructions: The Practical Subset

You can build most real images with these instructions:

- `FROM` choose base image (pin versions; consider digest pinning for prod)
- `WORKDIR` avoid brittle `cd` chains
- `COPY` bring in files (prefer explicit paths)
- `RUN` install/build steps
- `ENV` defaults (not secrets)
- `USER` run as non-root
- `EXPOSE` document ports (does not publish)
- `CMD` default command
- `ENTRYPOINT` fixed entry wrapper (use sparingly)
- `LABEL` attach metadata
- `HEALTHCHECK` basic liveness signal (optional)

15.6.1 3.1 COPY vs ADD

Rule of thumb:

- Use `COPY` almost always.
- Use `ADD` only when you specifically need its extra behaviors (such as extracting a local tar archive).

Do not use `ADD` to fetch URLs.

15.6.2 3.2 Shell Form vs Exec Form

Exec form (recommended for servers):

```
CMD ["nginx", "-g", "daemon off;"]
```

Shell form:

```
CMD nginx -g 'daemon off;'
```

Why exec form is better:

- Signal handling works as expected.
- Your process becomes PID 1 (no intermediate shell).
- Arguments are not re-parsed by a shell.

15.6.3 3.3 ENTRYPOINT vs CMD

- CMD is the default that users commonly override.
- ENTRYPOINT is for the command you almost never want overridden.

If you use both:

```
ENTRYPOINT ["/app/entrypoint.sh"]  
CMD ["/app/server"]
```

15.6.4 3.4 EXPOSE Does Not Publish Ports

EXPOSE 8080 is documentation inside the image.

Publishing is runtime:

```
podman run -p 8080:8080 localhost/myapp:1 # run a container
```

[^ Go to TOC](#)

15.7 4 Lab A: Build a Tiny HTTP Image (Warm-Up)

Create a new directory:

```
mkdir -p ./image-lab # create directory  
cd ./image-lab # change directory
```

Create index.html:

```
printf '%s\n' '<h1>Hello from Podman</h1>' > index.html # print text without trailing newline
```

Create Containerfile:

```
FROM docker.io/library/nginx:stable  
COPY index.html /usr/share/nginx/html/index.html
```

Build and run:

```
podman build -t localhost/hello-nginx:1 . # build an image  
podman run --rm -p 8080:80 localhost/hello-nginx:1 # run a container
```

Verify in another terminal:

```
curl -sS http://127.0.0.1:8080/ # verify HTTP endpoint
```

Inspect the image:

```
podman image inspect localhost/hello-nginx:1 | less # inspect image metadata  
podman image history localhost/hello-nginx:1 # show image layer history
```

Cleanup:

```
podman rmi localhost/hello-nginx:1 # remove the image from local storage  
cd .. # change directory  
rm -rf ./image-lab # delete the lab directory
```

[^ Go to TOC](#)

15.8 5 Build Context Hygiene (The Most Common Image Leak)

Your build context is everything in the directory you pass to `podman build`.

If your directory contains:

- `.env`
- SSH keys
- kubeconfigs
- `node_modules`
- build artifacts

...then a sloppy `COPY . .` can accidentally ship them inside your image.

flowchart LR

```
    subgraph "Build Context (sent to builder)"
        SRC["src/<br/>package.json"]
        ENV[".env <- DANGER"]
        KEYS["id_rsa <- DANGER"]
        NM["node_modules/ <- bloat"]
    end
    CI[".containerignore<br/>(blocks bad files)"] -->|"filters out"| ENV
    CI -->|"filters out"| KEYS
    CI -->|"filters out"| NM
    SRC -->|"COPY src/ /app/src"| IMG["Final Image<br/>(clean + small)"]
```

15.8.1 5.1 Use `.containerignore`

Create `.containerignore` next to your `Containerfile`:

```
.git
.github
.env
*.key
*.pem
node_modules
dist
build
tmp
*.log
```

Podman commonly supports `.containerignore` and often also `.dockerignore`.

15.8.2 5.2 Prefer Explicit Copies

Instead of:

```
COPY . /app
```

Prefer:

```
COPY package.json bun.lockb /app/
COPY src/ /app/src/
```

This prevents accidental inclusion and improves caching.

[^ Go to TOC](#)

15.9 6 Running as Non-Root (Image-Level Least Privilege)

Rootless Podman protects the host.

Running as non-root inside the container protects you from:

- container escape bugs that still require privileges inside the container
- accidental writes to system locations in the image
- overly-permissive defaults (root can write almost anywhere)

15.9.1 6.1 The Three Places You Usually Need Write Access

- /tmp
- an app state directory (like /var/lib/myapp)
- log directory (often better to log to stdout/stderr)

Best practice:

- treat the root filesystem as read-only when possible (Module 12)
- use a dedicated volume or tmpfs for the few paths that must be writable

15.9.2 6.2 Pattern: Create a User and Own the App Directory

FROM docker.io/library/alpine:3.20

```
RUN addgroup -S app && adduser -S -G app app
WORKDIR /app
COPY --chown=app:app . /app
USER app
CMD ["sh", "-lc", "id && ls -la"]
```

Notes:

- COPY --chown=... is often cleaner than RUN chown -R
- Some minimal images do not include adduser/addgroup (use their native tools).

15.9.3 6.3 Lab B: Verify Non-Root Actually Works

```
mkdir -p ./nonroot-lab # create directory
cd ./nonroot-lab # change directory

cat > Containerfile <<'EOF'
FROM docker.io/library/alpine:3.20

RUN addgroup -S app && adduser -S -G app app
WORKDIR /app
COPY --chown=app:app . /app
USER app
CMD ["sh", "-lc", "echo user=$(id -u) group=$(id -g); touch /app/ok; ls -la /app"]
EOF

podman build -t localhost/nonroot:1 . # build an image
podman run --rm localhost/nonroot:1 # run a container
podman rmi localhost/nonroot:1 # remove the image from local storage
cd .. # change directory
rm -rf ./nonroot-lab # delete the lab directory
```

If touch /app/ok fails, you did not set ownership correctly.

[^ Go to TOC](#)

15.10 7 Multi-Stage Builds (Small Images, Fast Builds)

Multi-stage builds let you:

- compile/build in a stage that has toolchains
- copy only the runtime artifacts into a final minimal image

Key properties:

- stages have names: `FROM ... AS build`
- later stages can `COPY --from=build ...`
- `podman build --target <stage>` stops early (useful for debugging)

```
flowchart LR
    subgraph "Stage 1: build"
        B1["FROM golang:1.22 AS build"]
        B2["COPY source code"]
        B3["RUN go build -o /app/server"]
    end
    subgraph "Stage 2: runtime"
        R1["FROM scratch (or alpine)"]
        R2["COPY --from=build /app/server /server"]
        R3["CMD ['/server']"]
    end
    B1 --> B2 --> B3
    B3 -->|"only binary copied<br/>(no compiler, no src)"| R2
    R1 --> R2 --> R3

    SIZE1["Build image<br/>~1 GB (compiler + src)"]
    SIZE2["Runtime image<br/>~10 MB (binary only)"]
    SIZE1 -.-> B1
    SIZE2 -.-> R1
```

15.10.1 7.1 Lab C (Optional): Provided Go Multi-Stage Example

This repository includes:

- `examples/build/hello-go/Containerfile`
- `examples/build/hello-go/main.go`

Build and run:

```
podman build -t localhost/hello-go:1 examples/build/hello-go # build an image
podman run --rm -p 8085:8080 localhost/hello-go:1 # run a container
curl -sS http://127.0.0.1:8085/ # verify HTTP endpoint
```

Inspect size:

```
podman images | head # list images
podman image history localhost/hello-go:1 # show image layer history
```

15.10.2 7.2 Pattern: Build Dependencies First, Copy Source Later

This pattern maximizes cache reuse:

1. copy dependency manifests

2. install dependencies
3. copy application source
4. build

Even if you do not use multi-stage, the order still matters.

15.10.3 7.3 Example Pattern: Bun App (Build + Runtime)

This is an example Containerfile shape for Bun-based services. Adapt it to your project.

Try the repo-backed example:

- `examples/build/hello-bun/Containerfile`
- `examples/build/hello-bun/server.ts`

```
FROM docker.io/oven/bun:1.2.0 AS build
WORKDIR /app

# Install deps separately for caching
COPY package.json bun.lockb ./
RUN bun install --frozen-lockfile

COPY . ./
RUN bun run build

FROM docker.io/oven/bun:1.2.0 AS runtime
WORKDIR /app
ENV NODE_ENV=production

# Copy only the runtime artifacts you need
COPY --from=build /app/dist ./dist
COPY --from=build /app/package.json ./package.json

USER bun
EXPOSE 3000
CMD ["bun", "dist/server.js"]
```

Notes:

- If your build outputs different paths, adjust `COPY --from=build`.
- If you need native modules, your runtime base must be compatible.

15.10.4 7.4 Example Pattern: Static Web Build (Build Stage + nginx)

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /src
COPY package.json package-lock.json ./
RUN npm ci
COPY . ./
RUN npm run build

FROM docker.io/library/nginx:stable
COPY --from=build /src/dist/ /usr/share/nginx/html/
```

This keeps Node and build tools out of the runtime image.

[^] Go to TOC

15.11 8 Caching: Make Rebuilds Fast

Most slow builds are slow because caching is accidentally disabled.

15.11.1 8.1 Common Cache-Busters

- `COPY` . . . early in the file
- including `node_modules/` or `target/` in the context
- running `apt-get update` in a separate layer from `apt-get install`
- using floating package versions

15.11.2 8.2 Linux Packages: One Layer, Clean Up

For Debian/Ubuntu bases:

```
RUN apt-get update \  
  && apt-get install -y --no-install-recommends ca-certificates curl \  
  && rm -rf /var/lib/apt/lists/*
```

For Alpine:

```
RUN apk add --no-cache ca-certificates curl
```

15.11.3 8.3 Use Stage Targets for Faster Debugging

If a multi-stage build fails late, rebuild only to the stage you care about:

```
podman build --target build -t localhost/myapp:build . # build an image
```

Then you can run that stage as an image to inspect the filesystem:

```
podman run --rm -it localhost/myapp:build sh # run a container
```

[^] Go to TOC

15.12 9 ARG, ENV, and Configuration

15.12.1 9.1 ARG Is Build-Time

ARG values exist during build, and can influence caching.

```
ARG APP_VERSION  
LABEL org.opencontainers.image.version=$APP_VERSION
```

Build:

```
podman build --build-arg APP_VERSION=1.2.3 -t localhost/myapp:1 . # build an image
```

15.12.2 9.2 ENV Is Runtime Default

```
ENV PORT=3000  
EXPOSE 3000  
CMD ["/app/server"]
```

Override at runtime:

```
podman run --rm -e PORT=8080 localhost/myapp:1 # run a container
```

15.12.3 9.3 Do Not Put Secrets in ARG or ENV

If you do this:

```
ARG API_KEY
RUN curl -H "Authorization: Bearer $API_KEY" ...
```

You risk leaking secrets into:

- image history
- build logs
- intermediate layers

Use runtime secrets (Module 4) or build-time secret mechanisms (next section).

[^ Go to TOC](#)

15.13 10 Secrets and Private Dependencies (Build-Time)

Rules you can rely on:

- never COPY secret files into the image
- never commit secrets into the build context
- prefer fetching private dependencies outside the build and copying only artifacts

15.13.1 10.1 If Your Podman Supports Build Secrets

Some Podman/Buildah versions support `podman build --secret ....`

Example shape (do not assume your version supports it):

```
podman build --secret id=npmc,src=$HOME/.npmrc -t localhost/private-build:1 . # build an image
```

In a Containerfile, the secret is mounted at build time (not copied into layers).

If your version does not support it, use the safe fallback below.

15.13.2 10.2 Safe Fallback: Fetch in CI, Copy Artifacts

Instead of cloning or downloading private content during image build:

1. fetch private deps in CI with credentials
2. produce a build artifact (binary, bundle, wheel)
3. copy only the artifact into the build context
4. build a runtime image that contains only that artifact

This keeps secrets entirely out of the image build process.

[^ Go to TOC](#)

15.14 11 Labels, Metadata, and Image Introspection

Labels help you operate images later.

Recommended OCI labels:

- `org.opencontainers.image.title`
- `org.opencontainers.image.description`
- `org.opencontainers.image.source`
- `org.opencontainers.image.revision`

- `org.opencontainers.image.version`
- `org.opencontainers.image.licenses`

Example:

```

LABEL org.opencontainers.image.title="hello"
LABEL org.opencontainers.image.source="https://example.com/repo"
LABEL org.opencontainers.image.version="1.0.0"

```

Inspect labels:

```
podman image inspect localhost/myapp:1 --format '{{json .Labels}}' # inspect image metadata
```

[^ Go to TOC](#)

15.15 12 Tagging, Digests, and Promotion

15.15.1 12.1 Tags Are Mutable

`myapp:latest` can point to different content over time.

This is convenient, but it is not auditable.

15.15.2 12.2 Digests Are Immutable

Pull and run by digest:

```

podman pull docker.io/library/nginx@sha256:<digest> # pull an image
podman run --rm docker.io/library/nginx@sha256:<digest> # run a container

```

For production:

- build from pinned bases when you need repeatability
- promote images by digest (not by tag) when you need audit trails

15.15.3 12.3 A Simple Promotion Flow

1. build locally or in CI as `myapp:git-<sha>`
2. run tests
3. re-tag as `myapp:staging`
4. re-tag as `myapp:prod`

Commands:

```

podman tag localhost/myapp:git-abc123 localhost/myapp:staging # add another tag/name
podman tag localhost/myapp:git-abc123 localhost/myapp:prod # add another tag/name

```

[^ Go to TOC](#)

15.16 13 Pushing Images to a Registry

15.16.1 13.1 Login

```
podman login <registry> # log into a container registry
```

15.16.2 13.2 Tag for the Registry Namespace

```
podman tag localhost/myapp:1 registry.example.com/team/myapp:1 # add another tag/name
```

15.16.3 13.3 Push

```
podman push registry.example.com/team/myapp:1 # push an image to a registry
```

15.16.4 13.4 Pull and Verify

```
podman pull registry.example.com/team/myapp:1 # pull an image
```

```
podman image inspect registry.example.com/team/myapp:1 | less # inspect image metadata
```

Production habit:

- record the digest you deployed
- configure your runtime (Quadlet, Kubernetes YAML) to pull by digest

[^ Go to TOC](#)

15.17 14 Testing the Image You Built

Your build is not done when `podman build` finishes.

Minimum checks:

1. container starts
2. correct ports are exposed/published
3. process responds to a health endpoint
4. container runs as non-root (if intended)
5. container writes only to intended paths

15.17.1 14.1 Smoke Test

```
podman run --rm -p 8080:8080 localhost/myapp:1 # run a container
```

15.17.2 14.2 Confirm Effective User

```
podman run --rm localhost/myapp:1 id # run a container
```

15.17.3 14.3 Healthcheck (If You Define One)

If your Containerfile includes HEALTHCHECK:

- You may see warnings that HEALTHCHECK is ignored in OCI image format.
- If you need the healthcheck stored in the image, build using docker format:

```
podman build --format docker -t localhost/myapp:1 . # build an image
```

Alternative: define a healthcheck at runtime with `podman run --health-*` flags.

```
podman run -d --name hc localhost/myapp:1 # run a container
```

```
podman healthcheck run hc # run the container healthcheck
```

```
podman inspect hc --format '{{json .State.Health}}' # inspect container/image metadata
```

```
podman rm -f hc # stop and remove the container
```

[^ Go to TOC](#)

15.18 15 Troubleshooting Builds (Common Failures)

15.18.1 15.1 COPY failed: file not found in build context

Causes:

- wrong path (relative paths are relative to the build context)
- file excluded by `.containerignore`
- building from the wrong directory

Fix:

- confirm your build context: `podman build ... <context-dir>`
- list files in the context dir

flowchart TD

```
S(["Build or runtime failure"]) --> Q1["COPY file<br/>not found?"]
Q1 -->|"Yes"| F1["Check .containerignore<br/>Check build context dir<br/>Use explicit paths"]
Q1 -->|"No"| Q2["exec format<br/>error?"]
Q2 -->|"Yes"| F2["Architecture mismatch<br/>Use --platform linux/amd64"]
Q2 -->|"No"| Q3["Container exits<br/>immediately?"]
Q3 -->|"Yes"| F3["Wrong CMD / missing binary<br/>Debug: --entrypoint sh"]
Q3 -->|"No"| Q4["Permission<br/>error?"]
Q4 -->|"Yes"| F4["Check USER order<br/>Use COPY --chown=app:app"]
Q4 -->|"No"| Q5["Image very<br/>large?"]
Q5 -->|"Yes"| F5["Use multi-stage build<br/>Add .containerignore<br/>Check image history"]
Q5 -->|"No"| F6["Check podman logs<br/>Run interactively: -it --entrypoint sh"]
```

15.18.2 15.2 Permission Errors in RUN Steps

Typical in rootless builds when scripts assume root-only locations.

Fix patterns:

- write into `$HOME` or your work directory, not `/root`
- ensure `WORKDIR` exists
- if you switch to `USER app`, do it after you finish root-only install steps

15.18.3 15.3 Container Starts Then Exits Immediately

Causes:

- CMD is missing or wrong
- app binary not executable
- wrong working directory

Debug:

```
podman run --rm -it --entrypoint sh localhost/myapp:1 # run a container
```

15.18.4 15.4 exec format error

Cause:

- architecture mismatch (built for amd64, running on arm64, or vice versa)

Fix:

- build for the target platform (if your environment supports it):

```
podman build --platform linux/amd64 -t localhost/myapp:amd64 . # build an image
```

Cross-building often requires extra host setup (emulation). Treat it as an advanced topic.

15.18.5 15.5 Huge Images

Causes:

- build tools included in the runtime stage
- caches kept (package manager caches, build artifacts)
- copying your entire repo including junk

Fix:

- multi-stage builds
- `.containerignore`
- copy only artifacts into the final stage

Inspect what grew:

```
podman image history localhost/myapp:1 # show image layer history
```

[^ Go to TOC](#)

15.19 16 Cleanup: Keep Your Machine Healthy

Image builds create intermediate images and caches.

Useful commands:

```
podman images # list images
podman ps -a # list containers

podman image prune      # remove unused images (frees disk)
podman container prune  # remove stopped containers
podman system prune     # remove unused objects (be careful)

# builder-specific cache (if supported)
podman builder prune    # remove build cache (if supported)
```

Be careful:

- `podman system prune` can delete data you care about if you store it in unnamed volumes.
-

[^ Go to TOC](#)

15.20 17 Extended Lab: A Small “Real” Service Image

This lab builds a service image with:

- non-root runtime
- explicit copies
- reasonable labels
- a predictable port

It uses only shell + Python standard library so you do not need extra tooling.

15.20.1 17.1 Create a Small App

```
mkdir -p ./svc-lab # create directory
cd ./svc-lab # change directory
```

Create `server.py`:

```

cat > server.py <<'EOF'
import os
from http.server import BaseHTTPRequestHandler, HTTPServer

PORT = int(os.environ.get("PORT", "8080"))

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/plain; charset=utf-8")
        self.end_headers()
        self.wfile.write(b"ok\n")

HTTPServer(("0.0.0.0", PORT), Handler).serve_forever()
EOF

```

Create a .containerignore:

```

cat > .containerignore <<'EOF'
.git
.env
*.log
__pycache__
EOF

```

Create Containerfile:

```

cat > Containerfile <<'EOF'
FROM docker.io/library/python:3.13-alpine

LABEL org.opencontainers.image.title="svc-lab"
LABEL org.opencontainers.image.description="tiny HTTP server lab"

RUN addgroup -S app && adduser -S -G app app

WORKDIR /app
COPY --chown=app:app server.py /app/server.py

ENV PORT=8080
USER app
EXPOSE 8080

CMD ["python", "/app/server.py"]
EOF

```

Build and run:

```

podman build -t localhost/svc-lab:1 . # build an image
podman run --rm -p 8088:8080 localhost/svc-lab:1 # run a container

```

Verify:

```

curl -sS http://127.0.0.1:8088/ # verify HTTP endpoint

```

Cleanup:

```

podman rmi localhost/svc-lab:1 # remove the image from local storage
cd .. # change directory
rm -rf ./svc-lab # delete the lab directory

```

[^ Go to TOC](#)

15.21 Checkpoint

- You can explain what a build context is and why `.containerignore` matters.
- You can write a Containerfile that runs as non-root.
- You can explain why multi-stage builds keep runtime images small.
- You can inspect image history to understand what changed.
- You can explain why secrets do not belong in `ARG`, `ENV`, or `COPY`.

[^ Go to TOC](#)

15.22 Quick Quiz

- 1) Your build works, but rebuilds are always slow even when nothing changes. What Containerfile patterns usually cause cache misses?
- 2) Why is `COPY . .` risky even when you think your repo contains no secrets?
- 3) You want to build a Go binary and ship it in a small runtime image. What key multi-stage instruction copies the artifact into the final image?
- 4) You set `EXPOSE 8080` in the image but the service is not reachable from the host. What did you forget?
- 5) An auditor asks you to prove exactly which base image you shipped. Why does pinning by digest help?

[^ Go to TOC](#)

15.23 Further Reading

- `man podman-build`
- `man podman-image`
- `man Containerfile` (often via Buildah docs)
- OCI image spec labels: <https://github.com/opencontainers/image-spec/blob/main/annotations.md>

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

16 Module 9: Multi-Service Workflows



16.1 Table of Contents

- Learning Goals
- The Multi-Service Problem
- Three Patterns Compared
- Pattern 1: User-Defined Network + Separate Containers
- Pattern 2: Pod (Shared localhost)
- Pattern 3: podman play kube
- Lab: A Two-Service Stack (MariaDB + Adminer)
- Make It Repeatable (Script)
- Idempotency — What It Means and Why It Matters
- From Script to Quadlet
- Compose-ish Tooling (Context)
- Checkpoint
- Quick Quiz
- Further Reading

This module teaches patterns for running a small stack without jumping straight to a full orchestrator.

[^ Go to TOC](#)

16.2 Learning Goals

- Choose the right multi-service pattern for your workload.
- Build a repeatable “stack up / stack down” workflow.
- Explain how service discovery works via container names on a user-defined network.
- Understand why database containers should not publish ports.
- Know the tradeoffs of compose-style tooling vs Quadlet.

[^ Go to TOC](#)

16.3 The Multi-Service Problem

A web application typically needs at least two processes: the application server and the database. These must:

1. **Find each other** — service discovery.
2. **Be isolated** from unrelated services.
3. **Have independent lifecycles** — you restart the app without restarting the DB.
4. **Persist data** — the DB volume survives container restarts.
5. **Not expose internal services** — the DB should never have a port published to the host in production.

graph LR

```
subgraph "Host network"
    H["Browser / curl<br/>host port 8086"]
end
subgraph "stacknet (user-defined bridge)"
    W["stack-web<br/>(Adminer)<br/>port 8080 internal"]
    D["stack-db<br/>(MariaDB)<br/>port 3306 internal<br/>NO host port"]
    V[["dbdata volume<br/>/var/lib/mysql"]]
end
H -->|"8086:8080"| W
```

```
W -->|"DNS: stack-db:3306"| D
D --- V
```

Key insight: **containers on the same user-defined network can reach each other by name.** `stack-web` connects to MariaDB via `stack-db:3306` — the container name resolves automatically via Podman's embedded DNS.

^ Go to TOC

16.4 Three Patterns Compared

Pattern	Service discovery	Use case	Isolation
User-defined network + separate containers	DNS by container name	Default for small stacks	Good
Pod (shared localhost)	127.0.0.1	Sidecar patterns, tight coupling	Shared network namespace
podman play kube (Kube YAML)	DNS by container/pod name	Teams that prefer YAML config	Good

flowchart TD

```
A{"Which multi-service<br/>pattern?"}
A -->|"Independent lifecycles<br/>Separate DNS names"| B["User-defined network<br/>+ separate containers"]
A -->|"Sidecar needs localhost<br/>or shared /proc"| C["Pod (shared namespace)"]
A -->|"Team prefers Kubernetes YAML<br/>or testing k8s manifests"| D["podman play kube"]
A -->|"Team already uses compose"| E["podman-compose<br/>(wrapper over primitives)"]
```

^ Go to TOC

16.5 Pattern 1: User-Defined Network + Separate Containers

This is the **default choice** for a small multi-service stack. The network provides:

- **DNS-based service discovery:** containers find each other by name.
- **Isolation:** containers on `stacknet` cannot reach containers on other networks.
- **Flexible lifecycle:** start/stop/restart containers independently.

Create network and connect containers:

```
podman network create stacknet # create isolated bridge network
podman run -d --name db --network stacknet mydb # DB on stacknet, no published port
podman run -d --name app --network stacknet -p 8080:8080 myapp # app on stacknet, port published
```

Inside `app`, the DB is reachable at `db:5432` (or whatever port the DB listens on). No IP addresses needed.

```
podman network inspect stacknet # show containers attached and their IPs
```

^ Go to TOC

16.6 Pattern 2: Pod (Shared localhost)

A Pod groups containers so they share the same **network namespace** — meaning `127.0.0.1` inside any container in the pod reaches any other container in the same pod.

When to use a pod: - A sidecar proxy (e.g., Envoy, Nginx) fronts the main app and must bind `127.0.0.1`. - A log shipper must read from a Unix socket shared with the main process. - You want to run Kubernetes-compatible workloads locally.

When NOT to use a pod: - You want independent restart policies per container (pod restart restarts all). - Containers need strong network isolation from each other.

```
podman pod create --name mypod -p 8080:8080      # create pod, publish port via infra container
podman run -d --pod mypod --name app myapp      # app joins pod's network namespace
podman run -d --pod mypod --name sidecar mysidecar # sidecar also on 127.0.0.1
```

See Module 7 (Pods) for a deep-dive on the infra container and sidecar patterns.

^ Go to TOC

16.7 Pattern 3: podman play kube

podman play kube applies a Kubernetes-flavoured YAML file to create containers, pods, volumes, and secrets. It is useful when:

- Your team wants to maintain infrastructure as YAML (GitOps-friendly).
- You are building something that will eventually run in Kubernetes.
- You want to test Kubernetes manifests locally without a cluster.

```
podman play kube stack.yaml # create resources from Kubernetes YAML
podman play kube --down stack.yaml # tear down resources
```

See Module 10 (play kube) for the full lab.

^ Go to TOC

16.8 Lab: A Two-Service Stack (MariaDB + Adminer)

Goal: run MariaDB (private, persistent) and Adminer (web UI, published port) on a shared network.

Stack layout:

Container	Image	Role	Ports
stack-db	docker.io/library/mariadb:11	Database (private)	None published
stack-web	docker.io/library/adminer:4	Web UI (public)	8086:8080

Step 1: Create infrastructure objects:

```
podman network create stacknet      # create isolated network
podman volume create dbdata        # create persistent DB volume
```

Step 2: Create the database password secret:

```
printf '%s' 'choose-a-lab-password' | podman secret create stack_mariadb_root_password - # create secret
```

Step 3: Start the database (no published port):

```
podman run -d \
  --name stack-db \
  --network stacknet \
  -v dbdata:/var/lib/mysql \
  --secret stack_mariadb_root_password \
  -e MARIADB_ROOT_PASSWORD_FILE="/run/secrets/stack_mariadb_root_password" \
  docker.io/library/mariadb:11 # start MariaDB with persistent storage, secret password
```

Note: MARIADB_ROOT_PASSWORD_FILE tells MariaDB to read its password from a file — this is the correct pattern for secret-as-file.

Step 4: Start the web UI (published port):

```
podman run -d \
  --name stack-web \
  --network stacknet \
  -p 8086:8080 \
  -e ADMINER_DEFAULT_SERVER="stack-db" \
  docker.io/library/adminer:4 # start Adminer, pointing at stack-db by name
```

Step 5: Verify both containers are running:

```
podman ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}' # show running containers with ports
```

Step 6: Verify network topology:

```
podman network inspect stacknet # show network details: both containers should appear
```

Step 7: Verify DB has no host ports:

```
podman port stack-db # should print nothing - no published ports
podman port stack-web # should print: 8080/tcp -> 0.0.0.0:8086
```

Step 8: Verify service discovery (web can reach DB by name):

```
podman exec stack-web sh -lc 'nc -z stack-db 3306 && echo "DB reachable"' # test name resolution
```

Step 9: Access the web UI:

Open <http://127.0.0.1:8086> in a browser, or:

```
podman exec stack-web sh -lc 'wget -qO- http://127.0.0.1:8080 | head -5' # test locally inside container
```

Cleanup:

```
podman rm -f stack-web stack-db # stop and remove containers
podman network rm stacknet # remove network
podman volume rm dbdata # remove volume (DATA LOSS)
podman secret rm stack_mariadb_root_password # remove secret
```

^ Go to TOC

16.9 Make It Repeatable (Script)

The `examples/stack/stack.sh` script in the course repo implements an idempotent `up` / `down` / `status` workflow for exactly this stack.

```
bash examples/stack/stack.sh up # create infra, start containers (idempotent)
bash examples/stack/stack.sh status # show running containers
bash examples/stack/stack.sh down # stop and remove containers (leaves volume/network/secret)
```

Key patterns demonstrated in the script:

Idempotent infrastructure creation:

```
if ! podman network exists "$NET"; then
  podman network create "$NET"
fi
```

Idempotent container start:

```
if ! podman container exists "$DB_NAME"; then
  podman run -d --name "$DB_NAME" ...
fi
podman start "$DB_NAME" # no-op if already running
```

Secure password handling:

```
# If running interactively and secret doesn't exist, prompt for it:
read -r -s -p 'MariaDB root password: ' p
printf '%s' "$p" | podman secret create "$SECRET_NAME" -
```

This is a **learning tool**, not a production deploy mechanism. For production, use Quadlet units (Module 11).

^ Go to TOC

16.10 Idempotency — What It Means and Why It Matters

An idempotent operation produces the same result whether you run it once or ten times. For stack management, this means:

- Running `stack.sh` up twice does not create duplicate containers or fail on “network already exists”.
- Running `stack.sh` up after a crash picks up where it left off.

flowchart TD

```
A["stack.sh up"] --> B{"network exists?"}
B -->|"No"| C["podman network create stacknet"]
B -->|"Yes"| D["skip"]
C --> E{"volume exists?"}
D --> E
E -->|"No"| F["podman volume create dbdata"]
E -->|"Yes"| G["skip"]
F --> H{"secret exists?"}
G --> H
H -->|"No + interactive"| I["prompt for password<br/>podman secret create"]
H -->|"Yes"| J["skip"]
I --> K{"container exists?"}
J --> K
K -->|"No"| L["podman run -d ..."]
K -->|"Yes"| M["podman start (no-op)"]
L --> N["Stack running"]
M --> N
```

The key primitives: - `podman network exists $NET` — returns 0 if network exists - `podman volume exists $VOL` — returns 0 if volume exists - `podman secret exists $NAME` — returns 0 if secret exists - `podman container exists $NAME` — returns 0 if container exists

^ Go to TOC

16.11 From Script to Quadlet

The stack script is a useful learning tool, but for a long-running server you want systemd managing the containers. The Quadlet equivalent of the stack script is:

- `~/.config/containers/systemd/stacknet.network` — defines the `stacknet` network
- `~/.config/containers/systemd/dbdata.volume` — defines the `dbdata` volume
- `~/.config/containers/systemd/stack-db.container` — MariaDB service
- `~/.config/containers/systemd/stack-web.container` — Adminer service with `After=stack-db.service`

Benefits of Quadlet over the script:

Feature	Script	Quadlet
Restart on failure	[X] Manual	[OK] <code>Restart=on-failure</code>
Boot survival	[X] Need cron/rc	[OK] <code>WantedBy=default.target</code> + <code>linger</code>
Dependency ordering	Partial (sequential)	[OK] <code>After= / Requires=</code>

Feature	Script	Quadlet
Log integration	[X] Container logs only	[OK] journald
Upgrade/rollback	Manual	[OK] Digest pin + restart

See Module 11 for the Quadlet labs.

^ Go to TOC

16.12 Compose-ish Tooling (Context)

If your team already uses Docker Compose files, you may encounter `podman-compose` or `docker-compose` (via the Docker-compatible socket). These are convenience wrappers that translate `compose.yaml` into `podman` commands.

Key rule for this course: learn the primitives first (networks, volumes, pods, Quadlet). Compose tools are a thin layer on top of these primitives. If you understand the primitives, you can debug any compose problem.

If your organisation has already standardised on compose files: - Use compose for developer local environments. - Use Quadlet for production/server deployments. - Understand the mapping: `services:` -> containers, `networks:` -> `podman network`, `volumes:` -> `podman volume`, `secrets:` -> `podman secret`.

graph LR

```
A["compose.yaml<br/>services, networks,<br/>volumes, secrets"] -->|"podman-compose translates"| B[""]
B --> C["Same Podman primitives<br/>you already know"]
```

^ Go to TOC

16.13 Checkpoint

- You can stand up and tear down a two-service stack (DB + app) predictably.
- You can explain why the DB should not publish a port.
- You can describe how DNS-based service discovery works on a user-defined network.
- You can make a stack script idempotent using `podman network exists` etc.
- You can explain when to use a pod vs a user-defined network.
- You can explain the path from a stack script to Quadlet for production.

^ Go to TOC

16.14 Quick Quiz

- 1) Why should your DB container usually not publish a port to the host?
- 2) Container `stack-web` is on `stacknet`. It tries to connect to `stack-db:3306`. How does `stack-db` resolve to an IP address?
- 3) What makes a stack script “safe” to run repeatedly? Name two techniques used in `stack.sh`.
- 4) You run `stack.sh` up twice. The second run should not fail. What prevents duplicate containers?
- 5) Your app container starts before the DB is ready and crashes. In the `stack.sh` approach, what must the app do? In Quadlet, what can you add?
- 6) What is one concrete reason to prefer Quadlet over a stack script for a server deployment?

^ Go to TOC

16.15 Further Reading

- `podman-network(1)`: <https://docs.podman.io/en/latest/markdown/podman-network.1.html>
- `podman-pod(1)`: <https://docs.podman.io/en/latest/markdown/podman-pod.1.html>
- `podman-play-kube(1)`: <https://docs.podman.io/en/latest/markdown/podman-play-kube.1.html>
- Compose Specification (for mapping concepts): <https://compose-spec.io/>
- Module 11: Production Baseline (Quadlet) — systemd-managed multi-service stacks

[^] Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

17 Module 10: podman play kube



17.1 Table of Contents

- Learning Goals
- What Is `podman play kube`?
- What YAML Resources Are Supported?
- The YAML-to-Podman Mapping
- Lab: Run a Pod from YAML
- Examining the Generated Resources
- Generating YAML from Existing Containers
- Production Note: Quadlet `.kube` Units
- Secrets Note — Base64 Is Not Encryption
- Limitations To Know
- Teardown — Always Use `podman kube down`
- Checkpoint
- Quick Quiz
- Further Reading

`podman play kube` lets you run a subset of Kubernetes YAML locally — without a Kubernetes cluster. It is the bridge between Podman primitives and the Kubernetes resource model.

[^ Go to TOC](#)

17.2 Learning Goals

By the end of this module you will be able to:

- Explain what `podman play kube` does and what it cannot do.
- Run a Pod from Kubernetes YAML locally.
- Understand the mapping between YAML fields and Podman objects.
- Generate YAML from an existing Podman pod.
- Use a Quadlet `.kube` unit to manage a YAML-defined pod with `systemd`.
- Explain why `base64` in YAML `Secret` objects is not encryption.
- Tear down YAML-created resources cleanly.

[^ Go to TOC](#)

17.3 What Is `podman play kube`?

`podman play kube` reads a Kubernetes YAML file and creates the corresponding Podman objects:

- A Pod YAML spec -> a Podman pod with its containers.
- A Deployment YAML spec -> multiple Podman pods (limited support).
- A PersistentVolumeClaim YAML spec -> a Podman named volume.
- A ConfigMap YAML spec -> environment variables or mounted files.
- A Secret YAML spec -> mounted files (with important caveats — see below).

```
sequenceDiagram
    participant U as "User"
```

```

participant P as "podman play kube"
participant Y as "webpod.yaml"
participant PO as "Podman Pod"
participant C as "Container(s)"

U->>P: "podman play kube webpod.yaml"
P->>Y: "Parse YAML"
Y-->>P: "Pod spec + containers"
P->>PO: "podman pod create<br/>with published ports"
P->>C: "podman run --pod ...<br/>for each container"
C-->>U: "Pod running"

U->>P: "podman kube down webpod.yaml"
P->>Y: "Parse resource names"
P->>C: "stop + remove containers"
P->>PO: "podman pod rm"

```

The key value proposition: **YAML is a repeatable, reviewable definition**. You can commit it to git, share it with your team, and re-run it identically on any machine with Podman — no `docker-compose.yml` required.

[^ Go to TOC](#)

17.4 What YAML Resources Are Supported?

Podman's `play kube` supports a practical subset of the Kubernetes API, not the full API surface.

Resource kind	Support level	Notes
Pod	Full	The primary use case
Deployment	Partial	Creates pods but no rolling update controller
PersistentVolumeClaim	Yes	Creates a named volume
ConfigMap	Yes	Env vars or mounted files
Secret	Yes (with caveats)	See secrets section below
Service	No	No load balancer or ClusterIP
Ingress	No	No ingress controller
StatefulSet	No	Use Quadlet for stateful services
DaemonSet	No	

Rule of thumb: Use `play kube` for Pod specs. Anything more complex — use Quadlet (Module 11) or your CI/CD tooling.

[^ Go to TOC](#)

17.5 The YAML-to-Podman Mapping

Understanding the mapping prevents surprises when you inspect resources after `play kube`.

Kubernetes YAML field	Maps to Podman
<code>metadata.name</code> of Pod	Pod name and container name prefix
<code>spec.containers[].image</code>	Image used for <code>podman run</code>

Kubernetes YAML field	Maps to Podman
<code>spec.containers[].ports[].containerPort</code>	Not published automatically (use <code>hostPort</code>)
<code>spec.containers[].ports[].hostPort</code>	Published port (<code>-p hostPort:containerPort</code>)
<code>spec.volumes[].name + persistentVolumeClaim</code>	Named volume
<code>spec.volumes[].name + configMap</code>	ConfigMap data mounted as files
<code>spec.containers[].env</code>	Environment variables
<code>spec.containers[].command + args</code>	Override entrypoint/cmd
<code>spec.containers[].resources.limits</code>	<code>--memory</code> , <code>--cpus</code> limits
<code>spec.containers[].securityContext.runAsUser</code>	<code>--user</code>
<code>spec.containers[].securityContext.readOnlyRootFilesystem</code>	<code>--readonly</code>

[^ Go to TOC](#)

17.6 Lab: Run a Pod from YAML

Use the example file included in this course:

File: `examples/kube/webpod.yaml`

Step 1 — Examine the YAML first

```
cat examples/kube/webpod.yaml # read the pod spec before running it
```

Take a moment to understand the structure: the pod name, container name, image, and any published ports.

Step 2 — Apply the YAML

```
podman play kube examples/kube/webpod.yaml # create resources from YAML
```

You will see output like:

```
Pod:
webpod
Container:
webpod-nginx
```

Step 3 — Inspect the created resources

```
podman pod ps # list pods - should show webpod
podman ps --pod # list containers showing pod membership
podman pod inspect webpod # detailed pod info
```

Notice that the resources are named based on the YAML `metadata.name` field.

Step 4 — Verify the service

```
# Get the published port from the pod
podman port webpod # show published ports
```

Then test it:

```
podman run --rm docker.io/library/alpine:latest sh -lc "wget -qO- http://host.containers.internal:8080/
echo "Try: curl http://127.0.0.1:8080/ from the host"
```

Step 5 — Tear down cleanly

```
podman kube down examples/kube/webpod.yaml # tear down all resources created by this YAML
```

Verify cleanup:

```
podman pod ps # should be empty (or not show webpod)
podman ps -a --pod # verify containers are gone
```

Important: Always use `podman kube down <yaml>` to tear down resources created by `play kube`. It reads the YAML to determine what to remove and handles cleanup in the correct order. Using `podman pod rm` directly works too, but you risk missing volumes or other resources.

^ Go to TOC

17.7 Examining the Generated Resources

After `play kube`, you can inspect every generated object:

```
# Inspect the pod
podman pod inspect webpod | less # detailed pod state

# Inspect a specific container (name pattern: podname-containername)
podman inspect webpod-nginx # container details

# Check logs
podman logs webpod-nginx # container logs

# Check events for what happened during creation
podman events --filter pod=webpod # events for this pod
```

^ Go to TOC

17.8 Generating YAML from Existing Containers

`podman generate kube` does the reverse: it takes a running pod and produces Kubernetes YAML.

```
# Create a pod manually
podman pod create --name demo-pod -p 9090:80 # create a pod to export
podman run -d --pod demo-pod --name demo-nginx docker.io/library/nginx:stable # add nginx

# Export it to YAML
podman generate kube demo-pod # generate Kubernetes YAML from the pod
podman generate kube demo-pod > demo-pod.yaml # save it to a file

# Inspect the generated YAML
cat demo-pod.yaml # read the generated spec

# Clean up
podman pod rm -f demo-pod # remove the pod
```

This workflow is useful for:

- Capturing a working local setup as YAML for repeatability.
- Providing a starting point for Kubernetes manifests.
- Documenting your stack in a structured format.

Note: The generated YAML may need some cleanup before using in Kubernetes — generated specs often include Podman-specific annotations or fields that Kubernetes ignores or rejects.

^ Go to TOC

17.9 Production Note: Quadlet .kube Units

For production use on RHEL/Fedora, combine `play kube` with Quadlet using a `.kube` unit. This gives you `systemd` lifecycle management (auto-restart, boot start, `journald` logs) over a YAML-defined pod.

Example .kube unit:

```
# ~/.config/containers/systemd/webpod.kube
[Kube]
Yaml=webpod.yaml
```

```
[Service]
Restart=always
```

```
[Install]
WantedBy=default.target
```

Install and start:

```
mkdir -p ~/.config/containers/systemd # create Quadlet directory
cp examples/quadlet/webpod.kube ~/.config/containers/systemd/ # install the kube unit
cp examples/quadlet/webpod.yaml ~/.config/containers/systemd/ # install the YAML spec
systemctl --user daemon-reload # regenerate units from Quadlet files
systemctl --user start webpod.service # start the YAML-defined pod
systemctl --user status webpod.service # show status
```

Note: `examples/quadlet/webpod.yaml` publishes on host port **8084**, not 8080 like `examples/kube/webpod.yaml` from the earlier lab — intentionally, so you can run both versions side by side without a port conflict.

Verify:

```
podman port webpod 2>/dev/null || podman pod ps # confirm pod is running
```

Stop and clean up:

```
systemctl --user stop webpod.service # stop the service
```

When to use this: If you want the YAML definition workflow but need `systemd` reliability and reboot persistence. See Module 11 for full Quadlet details.

^ Go to TOC

17.10 Secrets Note — Base64 Is Not Encryption

Kubernetes `Secret` objects encode their values in base64:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
data:
  password: Y29ycmVjdC1ob3JzZS1iYXR0ZXJ5LXN0YXBsZQ== # base64 of "correct-horse-battery-staple"
```

Base64 is encoding, not encryption. Anyone who can read the YAML file can decode it in seconds:

```
echo 'Y29ycmVjdC1ob3JzZS1iYXR0ZXJ5LXN0YXBsZQ==' | base64 -d # decodes to plaintext instantly
```

flowchart TD

```
A["plaintext secret<br/>'correct-horse-battery-staple'"]
A -->|"base64 encode"| B["Y29ycmVjdC1ob3JzZS1iYXR0ZXJ5LXN0YXBsZQ=="]
B -->|"anyone can decode"| A
B -->|"commit to git"| C["Secret is now public [X]"]
```

Never commit YAML with base64-encoded secrets to a git repository unless: - The repository is fully private AND - You have audited all access AND - You accept that anyone with repo access can decode the secrets instantly.

Better alternatives for dev/local use:

- **Podman secrets** — store via `podman secret create`, reference by name in YAML (not the value).
- **SOPS** — encrypts the values in the YAML file using real cryptography.
- **Out-of-band provisioning** — create Podman secrets ahead of time; the YAML just references names.

^ Go to TOC

17.11 Limitations To Know

`podman play kube` is not a Kubernetes cluster. The following limitations are important to understand before designing around it:

Not supported or limited: - **Service** resources (no ClusterIP, no load balancer, no service discovery across pods). - **Ingress** resources (no ingress controller). - **StatefulSet**, **DaemonSet**, **Job**, **CronJob** controllers. - Rolling updates (no Deployment controller tracking replica state). - Namespace isolation (Kubernetes namespaces, not Linux namespaces). - Resource quotas and admission controllers.

Behavioral differences from Kubernetes: - Liveness/readiness probes are honored for the container but have no pod rescheduling effect. - `hostPort` semantics work but `containerPort` alone does not publish. - Volume claims create Podman named volumes, not Kubernetes PVs.

Practical rule: Use `podman play kube` for local development and testing of Pod specs. Use a real Kubernetes cluster (or at minimum `minikube/kind`) for testing Deployment/Service/Ingress resources.

^ Go to TOC

17.12 Teardown — Always Use `podman kube down`

When you create resources with `podman play kube`, use the matching teardown command:

```
podman kube down examples/kube/webpod.yaml # tear down all resources created by this YAML
```

Why `kube down` instead of `podman pod rm`:

- It reads the YAML to determine the exact resources that were created.
- It removes containers, the pod, and optionally volumes in the correct order.
- It handles ConfigMaps, multiple pods (if the YAML defines several), and associated volumes.

If you want to also remove volumes created by `PersistentVolumeClaims` in the YAML:

```
podman kube down --force examples/kube/webpod.yaml # tear down and remove associated volumes
```

Warning: `--force` deletes volumes and their data. Only use it in labs or when you have verified the data is not needed.

^ Go to TOC

17.13 Checkpoint

Before moving on, confirm you can answer these:

- ☐ I can run a pod from a Kubernetes YAML file and confirm it is working.
- ☐ I know the right command to tear down resources created by `podman play kube`.
- ☐ I can explain why base64 in a YAML `Secret` is not encryption.
- ☐ I can generate YAML from an existing Podman pod with `podman generate kube`.
- ☐ I understand the key limitations: no Service, no Ingress, no rolling updates.
- ☐ I know how to use a Quadlet `.kube` unit for production YAML-defined services.

^ Go to TOC

17.14 Quick Quiz

1. Why is base64 not a secret storage mechanism?
2. What is the safest teardown command for resources created by `podman play kube`?
3. You have a Kubernetes `Deployment` YAML that you want to run locally with `podman play kube`. It uses a `Service` for internal DNS. What will not work?
4. What is the advantage of using a Quadlet `.kube` unit over running `podman play kube` directly?

^ Go to TOC

17.15 Further Reading

- `podman-play-kube(1)`: <https://docs.podman.io/en/latest/markdown/podman-play-kube.1.html>
- `podman-kube-down(1)`: <https://docs.podman.io/en/latest/markdown/podman-kube-down.1.html>
- `podman-generate-kube(1)`: <https://docs.podman.io/en/latest/markdown/podman-generate-kube.1.html>
- Quadlet `.kube` units: <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>
- Kubernetes objects overview: <https://kubernetes.io/docs/concepts/overview/working-with-objects/kubernetes-objects/>
- Kubernetes Secrets (base64 caveat): <https://kubernetes.io/docs/concepts/configuration/secret/>
- SOPS (encrypted secrets in git): <https://github.com/getsops/sops>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

18 Module 11: Production Baseline (systemd + Quadlet)



18.1 Table of Contents

- Learning Goals
- Why Quadlet
- Mental Model: Quadlet as a Generator
- Boot Safety (Rootless)
- Where Quadlet Files Live
- Unit File Types
- Anatomy of a `.container` Unit
- How “Enable” Works for Quadlet
- Helpful Podman Commands
- Lab: Your First Quadlet Container
- Lab: Pre-Create a Network and Volume (Quadlet)
- Debugging Quadlet Syntax
- Dependencies Between Quadlets
- Upgrades and Rollback
- Restart Policies
- Secrets
- Checkpoint
- Quick Quiz
- Further Reading

Quadlet lets you define containers, pods, networks, and volumes as declarative unit files that systemd manages.

[^ Go to TOC](#)

18.2 Learning Goals

- Explain what Quadlet is and how it differs from `podman run` + a shell script.
- Manage containers with systemd user services.
- Use Quadlet `.container`, `.pod`, `.network`, and `.volume` units.
- Make services reboot-safe with predictable restarts.
- Debug generator failures quickly.
- Wire up dependencies so containers start in the right order.

[^ Go to TOC](#)

18.3 Why Quadlet

Before Quadlet, the common approach was `podman generate systemd` — which produced a fragile, auto-generated unit file that embedded the full `podman run` command. It was hard to maintain and broke on container name changes.

Quadlet is a **systemd generator** built into Podman. You write a small, human-readable `.container` file. Quadlet translates it to a `.service` unit at daemon-reload time. Advantages:

Approach	Maintainability	Restart safety	systemd integration
Shell script	Low	None	None
<code>podman generate systemd</code>	Low (auto-generated)	[OK]	Partial
Quadlet	[OK] High	[OK]	[OK] Full
Kubernetes	Very high but heavy	[OK]	[X] Not systemd

Quadlet gives you: - **Restart policies** — on-failure, always, etc. - **Boot integration** — services survive reboots automatically. - **Dependency ordering** — After= / Requires= between units. - **journal logs** — all container output routed to journalctl. - **No docker-compose dependency** — pure systemd + Podman.

^ Go to TOC

18.4 Mental Model: Quadlet as a Generator

sequenceDiagram

```
participant You
participant Quadlet as "Quadlet generator<br/>(podman-system-generator)"
participant systemd
participant Podman

You->>You: Write hello-nginx.container<br/>to ~/.config/containers/systemd/
You->>systemd: systemctl --user daemon-reload
systemd->>Quadlet: Run generator (reads .container files)
Quadlet->>systemd: Emit hello-nginx.service (generated unit)
You->>systemd: systemctl --user start hello-nginx.service
systemd->>Podman: ExecStart=podman run ... (generated command)
Podman->>systemd: Container running (PID tracked)
Note over systemd,Podman: Restart, logs, stop all via systemd
```

The key insight: **you write declarative config; Quadlet writes the systemd unit; systemd manages the container lifecycle.** You never maintain the generated `.service` file directly.

^ Go to TOC

18.5 Boot Safety (Rootless)

By default, user systemd sessions only start when a user logs in. For a server where you want services to start at boot (before login), enable **lingering**:

```
sudo loginctl enable-linger "$USER" # allow user services to start at boot
```

Check linger status:

```
loginctl show-user "$USER" | grep Linger # Linger=yes means boot-safe
```

Without linger, your Quadlet services will not start until you log in — which defeats the purpose of a server.

^ Go to TOC

18.6 Where Quadlet Files Live

Scope	Path	Notes
User (rootless)	<code>~/.config/containers/systemd/</code>	Most common for this course
System-wide	<code>/etc/containers/systemd/</code>	Requires root; maps to system units
Vendor / package	<code>/usr/share/containers/systemd/</code>	Read-only; for distribution packages

For rootless development and production on a single-user server, use `~/.config/containers/systemd/`.

```
mkdir -p ~/.config/containers/systemd # ensure the directory exists
```

^ Go to TOC

18.7 Unit File Types

Quadlet recognizes these file extensions:

Extension	Purpose	Generated unit type
<code>.container</code>	Run a container	<code>.service</code>
<code>.volume</code>	Create a named volume	<code>.service</code>
<code>.network</code>	Create a named network	<code>.service</code>
<code>.pod</code>	Create a pod (shared namespace)	<code>.service</code>
<code>.kube</code>	Apply a Kubernetes YAML	<code>.service</code>
<code>.image</code>	Pull an image	<code>.service</code>

Each generates a `<name>-<type>.service` or `<name>.service` unit that systemd can manage.

^ Go to TOC

18.8 Anatomy of a `.container` Unit

A `.container` file has INI-style sections. Here is a fully annotated example:

```
[Unit]
# Standard systemd [Unit] section - metadata and dependencies
Description=Hello nginx demo
# Wait for network and for the labnet network to be created:
After=network-online.target labnet-network.service
Requires=labnet-network.service

[Container]
# The image to run (pin by digest in production):
Image=docker.io/library/nginx:stable
# Port mapping (host:container):
PublishPort=8081:80
# Named network defined in labnet.network:
Network=labnet.network
# Named volume defined in labdata.volume:
Volume=labdata.volume:/data
# Secret defined via `podman secret create`:
Secret=db_password
# Drop all capabilities (security hardening):
CapDrop=ALL
# Prevent setuid escalation:
SecurityLabelDisable=false
NoNewPrivileges=true
# Read-only root filesystem:
ReadOnly=true
# tmpfs for nginx writable paths:
Tmpfs=/var/cache/nginx
Tmpfs=/var/run
# Run as UID 1001:
User=1001:1001
# Memory and PID limits:
Memory=128m
PidsLimit=50
# Optional: override container name (default is systemd-<unit>):
```

```
ContainerName=hello-nginx
```

[Service]

```
# Restart policy:
```

```
Restart=on-failure
```

```
RestartSec=5s
```

```
# Timeout before systemd considers start failed:
```

```
TimeoutStartSec=30
```

[Install]

```
# Start when the user's default target is reached (boot-safe with linger):
```

```
WantedBy=default.target
```

Key [Container] fields map to podman run flags:

Quadlet key	Equivalent podman run flag
Image=	positional image argument
PublishPort=	-p host:container
Network=	--network
Volume=	-v
Secret=	--secret
CapDrop=	--cap-drop
CapAdd=	--cap-add
ReadOnly=true	--read-only
Tmpfs=	--tmpfs
User=	--user
Memory=	--memory
NoNewPrivileges=true	--security-opt no-new-privileges
Environment=	-e KEY=VALUE
Exec=	command override (like CMD)

^ Go to TOC

18.9 How “Enable” Works for Quadlet

Quadlet units are generated at **daemon-reload** time, not at **systemctl enable** time. The lifecycle is:

flowchart TD

```
A["Write .container file<br/>to ~/.config/containers/systemd/"] --> B["systemctl --user daemon-reload"]
B --> C["Quadlet generator runs<br/>emits .service unit in memory"]
C --> D{"[Install] section present?"}
D -->|"WantedBy=default.target"| E["systemd creates .wants/ symlink<br/>Unit starts at boot (with 1"]
D -->|"No [Install]"| F["Unit exists but does NOT<br/>auto-start at boot"]
E --> G["systemctl --user start <name>.service"]
F --> G
G --> H["Container running"]
```

Important: - You generally do **not** need to run **systemctl enable** manually for Quadlet units. The [Install] section + **daemon-reload** is sufficient. - If you remove a Quadlet file, run **daemon-reload** again — the generated unit disappears. - If you edit a Quadlet file, run **daemon-reload** then **restart** the service.

^ Go to TOC

18.10 Helpful Podman Commands

List all Quadlet definitions discovered by Podman:

```
podman quadlet list # list discovered Quadlet definitions
```

Print the resolved Quadlet file (shows defaults filled in):

```
podman quadlet print hello-nginx.container # show resolved Quadlet file with defaults
```

Dry-run the generator (shows what units would be generated):

```
/usr/lib/systemd/system-generators/podman-system-generator --user --dryrun # dry-run generator
```

Show the generated service unit:

```
systemctl --user cat hello-nginx.service # show generated .service unit content
```

^ Go to TOC

18.11 Lab: Your First Quadlet Container

Use the example unit from the course repo:

- examples/quadlet/hello-nginx.container

Install:

```
mkdir -p ~/.config/containers/systemd # ensure Quadlet search path exists
cp examples/quadlet/hello-nginx.container ~/.config/containers/systemd/ # install Quadlet unit
systemctl --user daemon-reload # regenerate units from configuration
systemctl --user start hello-nginx.service # start the service
systemctl --user status hello-nginx.service # show status
```

Verify HTTP response:

```
podman exec -it systemd-hello-nginx sh -lc 'wget -qO- http://127.0.0.1:80 | head -5' # verify nginx responds
```

Or from the host (if PublishPort=8081:80):

```
podman port systemd-hello-nginx # show published ports
```

View logs:

```
journalctl --user -u hello-nginx.service -n 50 --no-pager # view container logs via journald
```

Stop and clean up:

```
systemctl --user stop hello-nginx.service # stop the service
rm -f ~/.config/containers/systemd/hello-nginx.container # remove the Quadlet definition
systemctl --user daemon-reload # remove generated unit files
```

Notes:

- The running container name is `systemd-<unit>` by default unless you set `ContainerName=` in the unit.
- Quadlet supports the majority of `podman run` flags via `[Container]` keys — see the Podman docs for the full list.

^ Go to TOC

18.12 Lab: Pre-Create a Network and Volume (Quadlet)

Using `.network` and `.volume` units decouples infrastructure objects from containers. This allows: - Clean `Requires=` / `After=` dependency chains. - Volumes and networks survive container restarts. - Safe cleanup (stop units, then remove objects).

Use the example units:

- examples/quadlet/labnet.network
- examples/quadlet/labdata.volume

Install:

```
mkdir -p ~/.config/containers/systemd # ensure Quadlet search path
cp examples/quadlet/labnet.network ~/.config/containers/systemd/ # install network unit
cp examples/quadlet/labdata.volume ~/.config/containers/systemd/ # install volume unit
systemctl --user daemon-reload # regenerate units
systemctl --user start labnet-network.service # create network
systemctl --user start labdata-volume.service # create volume
```

Verify objects exist:

```
podman network ls | grep labnet # list networks, expect labnet
podman volume ls | grep labdata # list volumes, expect labdata
```

View logs for a network/volume unit (useful for debugging):

```
journalctl --user -u labnet-network.service -n 20 --no-pager # view network unit logs
```

Cleanup:

```
systemctl --user stop labnet-network.service || true # stop network unit
systemctl --user stop labdata-volume.service || true # stop volume unit
rm -f ~/.config/containers/systemd/labnet.network # remove network Quadlet file
rm -f ~/.config/containers/systemd/labdata.volume # remove volume Quadlet file
systemctl --user daemon-reload # regenerate units (removes generated se
podman network rm labnet 2>/dev/null || true # remove network object
podman volume rm labdata 2>/dev/null || true # remove volume object (data loss!)
```

^ Go to TOC

18.13 Debugging Quadlet Syntax

If `systemctl --user daemon-reload` succeeds but your service is not found, the generator likely failed silently.

Step 1: Run the generator in dry-run mode to see errors:

```
/usr/lib/systemd/system-generators/podman-system-generator --user --dryrun # dry-run generator, shows
```

Step 2: Analyze the generated unit:

```
systemd-analyze --user --generators=true verify hello-nginx.service # verify unit syntax
```

Step 3: Check journal for generator errors:

```
journalctl --user -b -p err -n 50 --no-pager # show boot errors for this session
```

Common mistakes:

Mistake	Symptom	Fix
Unknown [Container] key	Service not generated at all	Check spelling, consult <code>podman-systemd.unit.5</code>
Wrong network reference	Container fails to start	Use <code>Network=labnet.network</code> (with extension)
Wrong volume reference	Container fails to start	Use <code>Volume=labdata.volume:/path</code>

Mistake	Symptom	Fix
Missing <code>daemon-reload</code> after edit	Old version still running	Always <code>daemon-reload</code> after editing
File in wrong directory	Generator ignores file	Confirm it is in <code>~/.config/containers/systemd/</code>
Extra whitespace in value	Silent parse failure	No trailing spaces in INI values

Minimal debugging workflow:

1. Start with the smallest possible `.container` file (just `Image=`).
2. `daemon-reload` and `start` — does it work?
3. Add one option at a time until you find the failing key.

^ Go to TOC

18.14 Dependencies Between Quadlets

Quadlet translates symbolic references between `.container`, `.network`, `.volume` files into proper `systemd` `After=` / `Requires=` relationships.

Pattern:

```
# In your .container file:
[Unit]
After=labnet-network.service labdata-volume.service
Requires=labnet-network.service labdata-volume.service
```

```
[Container]
Network=labnet.network
Volume=labdata.volume:/data
```

```
graph LR
    A["labnet.network<br/>-> labnet-network.service"] --> C
    B["labdata.volume<br/>-> labdata-volume.service"] --> C
    C["myapp.container<br/>-> myapp.service"]
    C --> D["Running container<br/>on labnet, with labdata"]
```

This prevents race conditions on boot where the container starts before its network or volume exists.

Multi-container chains (e.g., database before app):

```
# app.container - depends on db.container
[Unit]
After=db.service
Requires=db.service
```

Note: Quadlet does not have a “wait for healthy” mechanism like Docker Compose `depends_on: condition: service_healthy`. You must handle startup retries in your app or use a readiness loop in the entrypoint.

^ Go to TOC

18.15 Upgrades and Rollback

The Quadlet upgrade pattern is designed to be **safe and reversible**:

```
sequenceDiagram
    participant Ops
    participant Quadlet as "Quadlet unit file"
```

participant systemd

```
Note over Ops: Record current digest before upgrade
Ops->>Ops: OLD_DIGEST=$(podman inspect --format={{.Digest}} myimage:tag)
Ops->>Ops: podman pull myimage@sha256:NEW_DIGEST
Ops->>Quadlet: Update Image= to new digest
Ops->>systemd: systemctl --user daemon-reload
Ops->>systemd: systemctl --user restart myapp.service
systemd->>Ops: Service started
Ops->>Ops: Verify service healthy
Note over Ops: If something is wrong:
Ops->>Quadlet: Revert Image= to OLD_DIGEST
Ops->>systemd: systemctl --user daemon-reload
Ops->>systemd: systemctl --user restart myapp.service
```

Rule: always record the previous digest before upgrading.

```
# Before upgrade:
OLD=$(podman inspect --format='{{.Digest}}' docker.io/library/nginx:stable)
echo "Rollback digest: $OLD" # save this

# After pull and test:
podman pull docker.io/library/nginx:stable # pull new version
NEW=$(podman inspect --format='{{.Digest}}' docker.io/library/nginx:stable)
# Update unit: Image=docker.io/library/nginx@sha256:<NEW>

^ Go to TOC
```

18.16 Restart Policies

Quadlet [Service] section accepts all standard systemd restart directives:

Restart= value	When it restarts
no	Never (manual restart only)
on-failure	Only on non-zero exit codes
on-abnormal	On signal/timeout/watchdog failure
always	Always, including clean exits
unless-stopped	Always, unless explicitly stopped

For long-running services, `on-failure` is the safest default — it won't loop-restart if your container exits cleanly during shutdown.

Pair with `RestartSec=` to add a backoff delay:

```
[Service]
Restart=on-failure
RestartSec=5s # wait 5 seconds before restarting after a failure
```

^ Go to TOC

18.17 Secrets

Do not store secret material in unit files — not in `Environment=` lines, not in `Exec=` commands, not anywhere in the unit.

The correct pattern:

1. Create a Podman secret: `printf '%s' 'value' | podman secret create myapp_db_password -`
2. Reference it by name in the unit: `Secret=myapp_db_password`
3. The container reads from `/run/secrets/myapp_db_password`.

[Container]

```
Secret=myapp_db_password
# App reads: open('/run/secrets/myapp_db_password').read().strip()
```

See `modules/11a-quadlet-secrets.md` for the full lab with rotation.

For distributed or encrypted-at-rest secrets, see `modules/90-external-secrets-survey.md`.

^ Go to TOC

18.18 Checkpoint

- You can write a `.container` Quadlet unit from scratch.
- You can start/stop a container via `systemd` user services.
- You can find logs in `journald` with `journalctl --user`.
- You can debug why a unit did not generate (dry-run generator).
- You can wire up `After=` / `Requires=` dependencies between Quadlet units.
- You understand the upgrade/rollback pattern with digest pinning.

^ Go to TOC

18.19 Quick Quiz

- 1) What is the Quadlet generator, and when does it run?
- 2) You edit a `.container` file. What two commands must you run for the change to take effect?
- 3) Your app container keeps starting before the database container is ready. What Quadlet [Unit] directives should you add?
- 4) What is the default container name when using Quadlet, and how do you override it?
- 5) Why is `Restart=always` potentially dangerous for a container that exits cleanly on shutdown?
- 6) You run `systemctl --user start myapp.service` and get “Unit not found”. What is the most likely cause and how do you diagnose it?

^ Go to TOC

18.20 Further Reading

- Quadlet and Podman `systemd` integration: <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>
- `podman-quadlet(1)`: <https://docs.podman.io/en/latest/markdown/podman-quadlet.1.html>
- `systemd` unit basics: <https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html>
- `systemd` user services: <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html>
- `journald`: <https://www.freedesktop.org/software/systemd/man/latest/journald.html>
- `loginctl` enable-linger: <https://www.freedesktop.org/software/systemd/man/latest/loginctl.html>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

19 Module 11a: Secrets with Quadlet + systemd (Rootless)



19.1 Table of Contents

- Learning Goals
- Why This Matters at the systemd Level
- Recommended Pattern
- Where Secrets Are Stored
- The Three Levels of “Not in the Unit File”
- Lab: Quadlet Unit Consuming a Secret
- Reading the Secret Inside Your App
- Rotation
- Multi-Secret Services
- systemd Credentials as an Alternative
- What This Does NOT Solve
- Checkpoint
- Quick Quiz
- Further Reading

This add-on to Module 11 shows how to run reboot-safe services while keeping secret material out of unit files and environment variables.

[^ Go to TOC](#)

19.2 Learning Goals

- Run a rootless systemd user service that consumes a secret as a file.
- Keep secret material out of:
 - unit files (`.container` files)
 - `Environment=` lines
 - shell history
 - `journalctl` output
- Rotate secrets safely with a rollback window.
- Know when to use systemd credentials vs Podman secrets.

[^ Go to TOC](#)

19.3 Why This Matters at the systemd Level

At the `podman run` level, secrets are a runtime flag. At the Quadlet/systemd level, they become part of your **deployment configuration** — a thing that must be managed, versioned, and auditable.

The most common mistake when moving from `podman run` to Quadlet is putting the secret value directly in the unit file:

```
# WRONG - value visible in unit file, journald, podman inspect
[Container]
Environment=DB_PASSWORD=hunter2
```

This is visible in: - The `.container` file committed to git (accidental exposure). - `systemctl --user cat myapp.service` (anyone with read access can see it). - `journalctl --user` if the app logs its environment. - `podman inspect` on the running container.

The correct pattern uses `Secret=` and a pre-created Podman secret:

```
# CORRECT - value is in the secrets store, not the unit file
[Container]
Secret=db_password
^ Go to TOC
```

19.4 Recommended Pattern

flowchart TD

```
A["Admin creates secret<br/>printf '%s' value | podman secret create db_password -"] --> B["Secret : db_password"]
B --> C["Quadlet unit references<br/>Secret=db_password"]
C --> D["systemctl --user daemon-reload<br/>systemctl --user start myapp.service"]
D --> E["Podman mounts secret<br/>as /run/secrets/db_password (tmpfs)"]
E --> F["App reads file<br/>DB_PASSWORD=$(cat /run/secrets/db_password)"]
F --> G["Secret in process memory only<br/>Not in env, not in inspect, not in logs"]
```

Guidelines:

- Treat secret names as part of your deployment config (document them).
- Prefer versioned secret names for rotation: `db_password_v1`, `db_password_v2`.
- Assume many apps only read secrets at startup — rotation requires a restart.
- Never use `Environment=DB_PASSWORD=...` for secret material.

^ Go to TOC

19.5 Where Secrets Are Stored

Podman stores secrets locally at:

```
~/local/share/containers/storage/secrets/
```

Contents: - `secrets.json` — metadata (names, IDs, driver, timestamps) - `filedriver/` — blobs (base64-encoded by default)

The default driver (`file`) stores blobs on disk with filesystem permissions restricted to the owner. This means:

- **Encryption at rest:** not provided by default. The blob is base64-encoded but not encrypted.
- **Per-machine:** secrets are local to the host. They are not replicated.
- **Scoped to the user:** rootless secrets belong to the creating user.

For encrypted-at-rest or multi-host distribution, see Module 90.

^ Go to TOC

19.6 The Three Levels of “Not in the Unit File”

There are three levels of secret hygiene. Be explicit about which level you are operating at:

Level	What is hidden	What is still exposed
Level 0: env var in unit	Nothing — value is in the unit file	Value in file, in <code>systemctl cat</code> , in <code>podman inspect</code>
Level 1: Podman secret	Value is in secrets store, not unit file	Name of secret is in unit file (acceptable)
Level 2: systemd credentials	Value injected by systemd at start	Value not visible even in Podman tooling

Most workloads need Level 1. Level 2 (systemd credentials) is useful for bootstrapping secrets into containers without even a local secrets store.

^ Go to TOC

19.7 Lab: Quadlet Unit Consuming a Secret

Prerequisites: - Rootless Podman installed. - systemd user session available.

Step 1: Enable lingering (for boot-safe service):

```
sudo loginctl enable-linger "$USER" # allow user services to start at boot
loginctl show-user "$USER" | grep Linger # verify Linger=yes
```

Step 2: Create the secret (example only — do not use this value):

```
printf '%s' 'example-password' | podman secret create db_password - # create secret from stdin, no tra
podman secret ls # confirm secret exists
```

Step 3: Write the Quadlet unit:

```
mkdir -p ~/.config/containers/systemd # ensure directory exists
```

Create ~/.config/containers/systemd/example-app.container:

```
[Unit]
Description=Example app with Podman secret

[Container]
Image=docker.io/library/busybox:latest
Secret=db_password
Exec=sh -lc 'while true; do echo heartbeat; sleep 30; done'
NoNewPrivileges=true
ReadOnly=true
Tmpfs=/tmp

[Service]
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=default.target
```

Step 4: Reload and start:

```
systemctl --user daemon-reload # regenerate units from Quadlet files
systemctl --user start example-app.service # start the service
systemctl --user status example-app.service # show status
```

Step 5: Verify the secret is mounted as a file:

```
podman exec systemd-example-app sh -lc 'ls -la /run/secrets' # list secret files
podman exec systemd-example-app sh -lc 'wc -c /run/secrets/db_password' # verify size without printing
```

Step 6: Verify the secret is NOT in the environment:

```
podman exec systemd-example-app sh -lc 'env | grep -i password || echo "not in env"' # should print: n
```

Step 7: Verify logs do not contain the secret value:

```
journalctl --user -u example-app.service -n 50 --no-pager # inspect logs - no secret value should appe
```

Cleanup:

```

systemctl --user stop example-app.service           # stop the service
systemctl --user disable example-app.service 2>/dev/null || true # disable if enabled
rm -f ~/.config/containers/systemd/example-app.container # remove Quadlet file
systemctl --user daemon-reload                     # remove generated unit
podman secret rm db_password                        # remove the secret

```

^ Go to TOC

19.8 Reading the Secret Inside Your App

The secret is mounted as a file. Here is how to read it in common languages:

// Node.js

```

const fs = require('fs');
const dbPassword = fs.readFileSync('/run/secrets/db_password', 'utf8').trim();

```

Python

```

with open('/run/secrets/db_password') as f:
    db_password = f.read().strip()

```

// Go

```

import "os"
data, err := os.ReadFile("/run/secrets/db_password")
dbPassword := strings.TrimSpace(string(data))

```

Shell

```

DB_PASSWORD=$(cat /run/secrets/db_password)

```

// Spring Boot - application.properties

```

spring.datasource.password=#{T(java.nio.file.Files).readString(T(java.nio.file.Path).of("/run/secrets/d

```

Important: trim the value. Depending on how the secret was created, there may or may not be a trailing newline. Always use `printf '%s' value | ...` when creating secrets to avoid trailing newlines.

^ Go to TOC

19.9 Rotation

Use versioned secret names. This allows parallel running of old and new versions during the rollback window.

sequenceDiagram

```

    participant Ops
    participant Podman
    participant systemd
    participant App

```

```

Note over App: Running with db_password_v1
Ops->>Podman: secret create db_password_v2 (new value)
Ops->>Ops: Edit unit: Secret=db_password_v1 -> Secret=db_password_v2
Ops->>systemd: daemon-reload
Ops->>systemd: restart example-app.service
systemd->>App: New container starts<br/>mounts /run/secrets/db_password_v2
Ops->>App: Verify healthy (check logs, test endpoint)
Note over Ops: Rollback window open - keep v1 secret
Ops->>Podman: secret rm db_password_v1 (only after verification)

```

19.9.1 Rotation Procedure

Step 1: Create the new secret version:

```
printf '%s' 'new-value' | podman secret create db_password_v2 - # create version 2
```

Step 2: Update the Quadlet file — change `Secret=db_password_v1` to `Secret=db_password_v2`.

Step 3: Reload and restart:

```
systemctl --user daemon-reload # pick up Quadlet changes
systemctl --user restart example-app.service # restart to use new secret
```

Step 4: Verify:

```
systemctl --user status example-app.service # check running status
journalctl --user -u example-app.service -n 100 --no-pager # check logs for errors
podman exec systemd-example-app sh -lc 'wc -c /run/secrets/db_password_v2' # confirm new secret mounted
```

Step 5: Remove old secret only after rollback window closes:

```
podman secret rm db_password_v1 # delete old secret - rollback no longer possible after this
```

Rule: never remove the old secret before the new deployment is verified and the rollback window has passed.

^ Go to TOC

19.10 Multi-Secret Services

A single container can consume multiple secrets. Declare each one on a separate `Secret=` line:

```
[Container]
Secret=db_password
Secret=api_key
Secret=tls_cert
```

Inside the container, these appear as: `- /run/secrets/db_password - /run/secrets/api_key - /run/secrets/tls_cert`

Each can have independent options (target path, uid, mode):

```
[Container]
Secret=db_password,target=/etc/myapp/db.pass,mode=0400,uid=1001
Secret=api_key,target=/etc/myapp/api.key,mode=0400,uid=1001
```

Rotate each independently — you do not need to restart for unrelated secret changes.

^ Go to TOC

19.11 systemd Credentials as an Alternative

systemd 250+ supports **credentials** — a way to pass secret material to a service via systemd itself, without using Podman secrets at all. This is useful when you want the secret to be managed entirely outside of Podman.

```
[Service]
LoadCredential=db_password:/etc/myapp/secrets/db_password
# Secret available inside the service at $CREDENTIALS_DIRECTORY/db_password
```

For containers specifically, you can combine systemd credentials with a bind mount (with caution — always ask before using bind mounts) or use it to pre-populate the Podman secrets store via an `ExecStartPre=` script.

The systemd credentials approach is more appropriate for:

- Secrets provisioned by configuration management (Ansible, Puppet).
- Secrets that must survive Podman being reinstalled.
- Environments where the secrets store itself needs to be audited.

See the systemd documentation linked in Further Reading for details.

[^ Go to TOC](#)

19.12 What This Does NOT Solve

Podman secrets + Quadlet solves local-machine secret hygiene. It does not solve:

Problem	Solution
Distributing secrets to many hosts	HashiCorp Vault, AWS SSM, etc. (see Module 90)
Encryption at rest for secrets on disk	Custom Podman secret driver, or external secrets manager
Automatic rotation without restart	Requires external rotation agent or app-level hot reload
Secret access control between users	OS-level filesystem permissions only
Audit trail of who read which secret	External secrets manager with audit logging

[^ Go to TOC](#)

19.13 Checkpoint

- You can write a Quadlet unit that references a secret by name without embedding the value.
- You can create a Podman secret safely (no shell history leakage).
- You can verify that the secret is mounted as a file and NOT present in the environment or logs.
- You can execute the full rotation procedure: create v2 -> update unit -> reload -> restart -> verify -> remove v1.
- You can explain what Podman secrets do NOT protect against.

[^ Go to TOC](#)

19.14 Quick Quiz

- 1) Why is it safer to mount secrets as files rather than pass them in environment variables?
- 2) You run `echo 'mysecret' | podman secret create db_password -`. The app fails to authenticate. What is the likely cause?
- 3) You have `Environment=DB_PASSWORD=hunter2` in your Quadlet unit and you check it into git. What are two vectors through which the value can leak?
- 4) Why should you keep the old secret version around until after the new deployment is verified?
- 5) A colleague says “I’ll just put the secret in `ExecStartPre=` as a shell variable”. What is wrong with this?
- 6) What does the `mode=0400,uid=1001` option on `Secret=` control?

[^ Go to TOC](#)

19.15 Further Reading

- `podman-secret(1)`: <https://docs.podman.io/en/latest/markdown/podman-secret.1.html>
- Quadlet and Podman systemd integration: <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>
- systemd credentials (service-provisioned files): <https://www.freedesktop.org/software/systemd/man/latest/systemd.exec.1.html#credentials>
- OWASP Secrets Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- Module 04: Secrets (Local-First) — podman secret commands
- Module 90: External Secrets Survey — HashiCorp Vault, AWS SSM

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

20 Module 12: Security Deep Dive



20.1 Table of Contents

- Learning Goals
- The Defence-in-Depth Model
- Baseline Hardening Checklist
- Linux Capabilities — What They Are
- Lab: Drop Capabilities
- Resource Limits — Why They Matter
- Lab: Resource Limits
- Lab: No-New-Privileges
- Lab: Read-Only Root FS
- User Namespace and Non-Root Inside Container
- SELinux (Fedora/RHEL)
- Seccomp Profiles
- Image Trust (Practical)
- Putting It Together: Hardened Run Template
- Checkpoint
- Quick Quiz
- Further Reading

This module focuses on reducing blast radius and making your container posture auditable.

[^ Go to TOC](#)

20.2 Learning Goals

- Apply least privilege inside and outside containers.
- Use read-only filesystems and drop capabilities.
- Understand the difference between rootless (on the host) and non-root (inside the container).
- Understand SELinux labels and why they matter on Fedora/RHEL.
- Apply resource limits to contain runaway or compromised workloads.
- Understand image trust at a practical level.
- Build a secure-by-default run configuration you can reuse.

[^ Go to TOC](#)

20.3 The Defence-in-Depth Model

Container security is not one setting — it is a stack of independent layers. If one layer fails, the others still limit damage.

graph TD

```
subgraph "Layer 6: Image trust"
    L6["Digest pinning<br/>Minimal base images<br/>Supply chain hygiene"]
end
subgraph "Layer 5: SELinux / AppArmor"
    L5["Mandatory access control<br/>Labels restrict what files<br/>containers can access"]
end
subgraph "Layer 4: Seccomp"
    L4["Syscall filtering<br/>Block kernel attack surface<br/>Default profile: ~300 syscalls blocked"]
end
subgraph "Layer 3: Capabilities"
```

```

    L3["Drop ALL, add-back only needed<br/>--cap-drop=ALL --cap-add=NET_BIND_SERVICE"]
end
subgraph "Layer 2: Filesystem"
    L2["--read-only root FS<br/>--tmpfs for writable scratch<br/>No bind-mounting host dirs"]
end
subgraph "Layer 1: User / namespace"
    L1["Rootless Podman<br/>Non-root UID inside container<br/>--security-opt no-new-privileges"]
end
L6 --> L5 --> L4 --> L3 --> L2 --> L1

```

The closer a layer is to the kernel, the harder it is to bypass. Layer 1 (user namespace + no-new-privileges) is your last line of defence if everything else fails.

^ Go to TOC

20.4 Baseline Hardening Checklist

Apply these for every long-running service container:

Control	Flag / setting	Why
Rootless Podman	(run as non-root user on host)	Limits kernel attack surface
Non-root inside container	User=1001 in Containerfile or --user 1001	Process cannot affect host even if namespace leaks
No new privileges	--security-opt no-new-privileges	Prevents setuid escalation inside container
Drop all capabilities	--cap-drop=ALL	Removes almost all kernel privileges from PID 1
Add back only what's needed	--cap-add=NET_BIND_SERVICE etc.	Least privilege
Read-only root FS	--read-only	Prevents code injection into app files
tmpfs for writable scratch	--tmpfs /tmp	Controlled, in-memory, not persistent
Memory limit	--memory 256m	Contain memory bombs / OOM blast radius
PID limit	--pids-limit 200	Contain fork bombs
No host network	(default — do not use --network=host)	Keeps container off host network stack
No privileged	(never use --privileged in prod)	Hands over the whole kernel
Avoid Docker socket equivalent	(never mount Podman socket into container)	Container-escape risk

^ Go to TOC

20.5 Linux Capabilities — What They Are

Traditional Unix had one privilege boundary: root (UID 0) vs non-root. Linux **capabilities** split root's privileges into ~40 independent tokens. A process can hold some capabilities without being UID 0.

Key capabilities relevant to containers:

Capability	What it allows	Typical need
NET_BIND_SERVICE	Bind ports < 1024	Web servers on port 80/443

Capability	What it allows	Typical need
CHOWN	Change file ownership arbitrarily	Init scripts, entrypoints
DAC_OVERRIDE	Bypass file permission checks	Package managers
SETUID / SETGID	Change process UID/GID	su, sudo, sshd
SYS_ADMIN	Huge catch-all: mount, sethostname, etc.	Almost never needed
SYS_PTRACE	Attach debuggers to processes	Debuggers only
NET_ADMIN	Configure network interfaces	Network tools
KILL	Send signals to other processes	Process managers

Podman's default capability set is narrower than Docker's. Even so, always drop ALL and add back only what you've verified.

flowchart LR

```

    A["Default capability set<br/>(podman run without flags)"] -->|"--cap-drop=ALL"| B["No capabilities"]
    B -->|"--cap-add=NET_BIND_SERVICE"| C["Only NET_BIND_SERVICE"]
    C --> D["Minimal attack surface"]

```

^ Go to TOC

20.6 Lab: Drop Capabilities

- 1) Run with the default capability set and list active caps:

```
podman run --rm docker.io/library/alpine:latest sh -lc 'cat /proc/1/status | grep Cap' # show capabilities
```

- 2) Drop all capabilities and verify:

```
podman run --rm --cap-drop=ALL docker.io/library/alpine:latest sh -lc 'cat /proc/1/status | grep Cap'
```

- 3) Attempt an operation that needs a dropped capability:

```
podman run --rm --cap-drop=ALL docker.io/library/alpine:latest sh -lc 'chown 0 /tmp && echo ok || echo blocked'
```

Expected: blocked (operation not permitted).

- 4) Add back only what you need:

```
podman run --rm --cap-drop=ALL --cap-add=CHOWN docker.io/library/alpine:latest \
sh -lc 'chown 0 /tmp && echo ok' # only CHOWN added back
```

- 5) The wrong answer — do not do this in production:

```

# WRONG: --privileged gives ALL capabilities plus more
# podman run --privileged ...
echo "Do not use --privileged - it defeats all capability controls"

```

Notes:

- Prefer `--cap-drop=ALL` as baseline, then add specific capabilities as errors reveal requirements.
- Document which capabilities your service needs and why.

^ Go to TOC

20.7 Resource Limits — Why They Matter

Resource limits serve two purposes:

1. **Blast radius containment:** a compromised or buggy container cannot take down the whole host by consuming all memory/CPU.
2. **Denial-of-service mitigation:** fork bombs and memory bombs are bounded.

Key limits:

Flag	What it limits	Example
<code>--memory</code>	Max RAM	<code>--memory 256m</code>
<code>--memory-swap</code>	RAM + swap total	<code>--memory-swap 256m (no swap)</code>
<code>--cpus</code>	CPU cores (fractional)	<code>--cpus 0.5</code>
<code>--pids-limit</code>	Max PIDs / threads	<code>--pids-limit 200</code>
<code>--ulimit nofile</code>	Open file descriptors	<code>--ulimit nofile=1024:1024</code>

^ Go to TOC

20.8 Lab: Resource Limits

1) Run with memory and PID limits:

```
podman run --rm --memory 256m --pids-limit 200 docker.io/library/alpine:latest \
  sh -lc 'echo "memory and pids limited"' # run a container with limits
```

2) Attempt a fork bomb (safe — will be contained):

```
podman run --rm --pids-limit 20 docker.io/library/alpine:latest \
  sh -lc ':(){ :|:& };; || echo "fork bomb contained"' # fork bomb blocked by pids-limit
```

The container will be killed or the shell will error — it cannot escape the PID limit.

3) Verify memory limit with a big allocation attempt:

```
podman run --rm --memory 32m docker.io/library/alpine:latest \
  sh -lc 'dd if=/dev/zero of=/dev/null bs=1M count=64 || echo OOM-killed' # dd into /dev/null is fine
```

Note: `dd` to `/dev/null` does not actually allocate memory; this just illustrates the pattern. Use `stress` or `python` for real OOM testing.

^ Go to TOC

20.9 Lab: No-New-Privileges

`--security-opt no-new-privileges` prevents a process inside the container from gaining new capabilities through `setuid` binaries or capability-setting file attributes.

Without this flag, a `setuid` `root` binary inside the container can elevate privileges. With it, the kernel ignores `setuid` bits.

1) Baseline — verify current UID:

```
podman run --rm docker.io/library/alpine:latest id # show current uid/gid
```

2) Apply no-new-privileges:

```
podman run --rm --security-opt no-new-privileges docker.io/library/alpine:latest id # same, but setuid
```

3) Combine with drop-all and non-root:

```
podman run --rm \
  --cap-drop=ALL \
  --security-opt no-new-privileges \
  --user 1001:1001 \
  docker.io/library/alpine:latest \
  id # running as 1001, no capabilities, no setuid escalation possible
```

^ Go to TOC

20.10 Lab: Read-Only Root FS

A read-only root filesystem means a compromised process cannot modify application code, replace binaries, or install backdoors. Writes are only possible to explicitly mounted tmpfs or volumes.

1) Attempt to write to the root FS:

```
podman run --rm --read-only docker.io/library/alpine:latest \
  sh -lc 'touch /etc/pwnted || echo "read-only blocked write"' # write attempt fails
```

2) Provide writable tmpfs for scratch space:

```
podman run --rm --read-only \
  --tmpfs /tmp \
  --tmpfs /var/run \
  docker.io/library/alpine:latest \
  sh -lc 'touch /tmp/scratch && echo ok' # tmpfs is writable
```

3) Run nginx with read-only root and appropriate tmpfs mounts:

```
podman run --rm -p 8080:80 \
  --read-only \
  --tmpfs /var/cache/nginx \
  --tmpfs /var/run \
  --tmpfs /tmp \
  docker.io/library/nginx:stable # run nginx read-only
```

If it fails, read the error message — it is a map of which paths nginx needs to write. Add a targeted `--tmpfs` for each.

4) Combine everything:

```
podman run --rm -p 8080:80 \
  --read-only \
  --cap-drop=ALL \
  --security-opt no-new-privileges \
  --memory 128m \
  --pids-limit 50 \
  --tmpfs /var/cache/nginx \
  --tmpfs /var/run \
  --tmpfs /tmp \
  docker.io/library/nginx:stable # fully hardened nginx
```

flowchart TD

```
A["podman run --read-only nginx"] --> B{"Write to root FS?"}
B -->|"Yes"| C["EROFS: read-only file system"]
B -->|"No, write to tmpfs"| D["Success: /tmp /var/cache/nginx /var/run"]
C --> E["Error message shows exact path needed"]
E --> F["Add --tmpfs /that/path"]
F --> A
```

Debugging read-only failures:

- Read the error message — it tells you the exact path.
- Add `--tmpfs <path>` for each path the app needs to write.
- Prefer small, targeted tmpfs mounts over giving up and removing `--read-only`.

^ Go to TOC

20.11 User Namespace and Non-Root Inside Container

There are two distinct concepts students often confuse:

Concept	What it means
Rootless Podman	Podman itself runs as a non-root user on the host. The container's UID 0 maps to an unprivileged host UID via user namespace.
Non-root inside container	The process inside the container runs as UID != 0, e.g., UID 1001. Even if the user namespace map gives UID 0 inside the container, the app process itself doesn't use it.

Both are desirable. They are independent:

graph TD

```
A["Host: non-root user (uid 1000)<br/>Rootless Podman"] --> B["Container: uid 0 (root inside)<br/>mapped to uid 100001 on host via userns"]
A --> C["Container: uid 1001 (non-root inside)<br/>mapped to uid 100001 on host via userns"]
B -->|"better"| D["User namespace isolates from host"]
C -->|"best"| E["Non-root inside + userns isolation"]
```

Best practice: use USER 1001 in your Containerfile or --user 1001:1001 at runtime.

Check the UID mapping on a running container:

```
podman run --rm docker.io/library/alpine:latest \
  sh -lc 'cat /proc/self/uid_map' # show uid_map (how container UIDs map to host UIDs)
```

^ Go to TOC

20.12 SELinux (Fedora/RHEL)

SELinux is a **Mandatory Access Control (MAC)** system baked into the RHEL/Fedora kernel. Unlike standard Unix permissions (which the process owner controls), SELinux policies are set by the system administrator and cannot be overridden by root.

For containers, SELinux adds a type label to every process and file. By default:

- Containers run with label `container_t`.
- Container processes can only access files labelled `container_file_t` or `svirt_sandbox_file_t`.
- Host files have labels like `user_home_t`, `etc_t` — containers cannot access them even if permissions allow it.

graph LR

```
subgraph "SELinux label check"
  P["Process: container_t"] -->|"open()"| F1["File: container_file_t<br/>Label matches policy"]
  P -->|"open()"| F2["Host file: user_home_t<br/>Label DENIED by policy"]
end
F1 -->|"allow"| R1["Access granted"]
F2 -->|"deny"| R2["Permission denied<br/>even if Unix perms allow"]
```

Practical rules:

- Prefer named volumes over bind mounts — volumes are automatically labelled correctly.
- If you must use a bind mount, add `:Z` for private relabelling:

```
podman run --rm -v /host/data:/app/data:Z myimage # :Z relabels for this container only
```
- Use `:z` (lowercase) for shared bind mounts accessed by multiple containers.

- **Never disable SELinux to fix container issues** — that removes protection for the entire host.

Check if SELinux is enforcing:

```
getenforce # show SELinux mode: Enforcing / Permissive / Disabled
```

Check SELinux denials:

```
ausearch -m avc -ts recent # show recent SELinux denial audit messages
```

^ Go to TOC

20.13 Seccomp Profiles

Seccomp (Secure Computing Mode) filters which Linux **syscalls** a container process can make. Podman applies a default seccomp profile that blocks ~300 dangerous syscalls (**reboot**, **kexec_load**, **create_module**, etc.).

You rarely need to change the default. But knowing it exists matters:

- If a container fails with **EPERM** doing something unusual (e.g., using **ptrace**, unusual socket types), seccomp may be blocking it.
- You can disable seccomp with **--security-opt seccomp=unconfined** — avoid this in production.
- Custom profiles allow you to allow or deny specific syscalls.

Check that the default profile is active:

```
podman run --rm docker.io/library/alpine:latest \
  sh -lc 'cat /proc/1/status | grep Seccomp' # 2 = SECCOMP_MODE_FILTER (active)
```

Expected: Seccomp: 2

^ Go to TOC

20.14 Image Trust (Practical)

A container is only as trustworthy as its image. The image supply chain is a real attack vector (typosquatting, compromised upstream, malicious layer injection).

Digest pinning: instead of a mutable tag like **:latest**, pin to an immutable digest:

Mutable - can change without warning:

```
podman pull docker.io/library/nginx:stable
```

Immutable - always the exact same bytes:

```
podman pull docker.io/library/nginx@sha256:abc123... # pin by digest
```

Workflow:

1. Pull by tag to get the current digest.
2. Record the digest in your Quadlet unit or Containerfile.
3. Upgrades are explicit: pull new digest, test, update the pinned value.

```
podman pull docker.io/library/nginx:stable # pull by tag
podman inspect --format='{{.Digest}}' docker.io/library/nginx:stable # get digest
```

Minimal base images:

Base	Approximate size	Notes
ubuntu:22.04	~70 MB	Many packages, large attack surface
debian:slim	~30 MB	Smaller, still apt available
alpine:latest	~7 MB	musl libc, common for small images

Base	Approximate size	Notes
ubi10-minimal	~30 MB	RHEL-based, supported by Red Hat
scratch	0 MB	No OS — statically compiled binaries only

Supply chain habits:

- Prefer official images or your organization's curated base images.
- Avoid running `curl | sh` installer scripts inside builds.
- Use reproducible builds — identical inputs should produce identical digests.
- Scan images for known CVEs (`trivy`, `grype`, `podman image scan` depending on availability).

```

flowchart TD
    A["Pull by tag :stable"] --> B["Inspect digest"]
    B --> C["Pin digest in unit file"]
    C --> D["Deploy / test"]
    D --> E{"Upgrade needed?"}
    E -->|"Yes"| F["Pull new tag<br/>get new digest"]
    F --> G["Test in staging"]
    G --> H["Update pinned digest<br/>in unit file"]
    H --> I["Restart service"]
    E -->|"No"| D

```

^ Go to TOC

20.15 Putting It Together: Hardened Run Template

A template combining all controls. Start here and remove only what your workload genuinely cannot function with:

```

podman run -d \
  --name myservice \
  --read-only \
  --cap-drop=ALL \
  --security-opt no-new-privileges \
  --user 1001:1001 \
  --memory 256m \
  --memory-swap 256m \
  --pids-limit 200 \
  --tmpfs /tmp \
  --secret db_password \
  --network app_net \
  docker.io/library/myapp@sha256:<digest> # hardened container run template

```

As a Quadlet unit (production):

```

[Unit]
Description=Hardened application service

[Container]
Image=docker.io/library/myapp@sha256:<digest>
ReadOnly=true
CapDrop=ALL
SecurityLabelDisable=false
NoNewPrivileges=true
User=1001:1001
Memory=256m

```

```
PidsLimit=200
Tmpfs=/tmp
Secret=db_password
Network=app_net.network
```

```
[Service]
Restart=on-failure
RestartSec=5s
```

```
[Install]
WantedBy=default.target
```

^ Go to TOC

20.16 Checkpoint

- You can explain the difference between rootless Podman and non-root inside the container — and why both matter.
- You can explain why `--privileged` is almost always the wrong answer.
- You can apply the full hardening stack: drop capabilities, read-only FS, no-new-privileges, resource limits.
- You can explain what SELinux does and when to use `:Z`.
- You can make a service run read-only, or explain exactly why it cannot.

^ Go to TOC

20.17 Quick Quiz

- 1) What is the difference between `--cap-drop=ALL` and `--privileged`? Which one should you never use in production?
- 2) A container writes to `/run/app.pid` at startup. You add `--read-only`. It fails. What is the correct fix?
- 3) You bind-mount `/home/user/data` into a container and get `Permission denied` even though Unix permissions look correct. What is the likely cause on RHEL?
- 4) Why is digest pinning useful even if you fully trust the upstream image maintainer?
- 5) What does `Seccomp: 2` in `/proc/1/status` tell you about the container?
- 6) Your app needs to bind port 80. You've dropped all capabilities. What single capability must you add back?

^ Go to TOC

20.18 Further Reading

- Linux capabilities (man7): <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- `seccomp(2)` (man7): <https://man7.org/linux/man-pages/man2/seccomp.2.html>
- SELinux with containers (RHEL docs): https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/using_selinux-with-containers_using-selinux
- Podman security docs: <https://github.com/containers/podman/blob/main/docs/tutorials/security.md>
- Rootless containers (rootlesscontainers.org): <https://rootlesscontainers.org/>
- Trivy image scanner: <https://aquasecurity.github.io/trivy/>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

21 Module 13: Troubleshooting and Ops



21.1 Table of Contents

- Learning Goals
- Minimum Path (If You Are Short on Time)
- 1 The Debug Loop (Mental Model)
- 2 Container State and Lifecycle Commands
- 3 Reading Logs
- 4 Deep Inspection with `podman inspect`
- 5 Interactive Debugging
- 6 Events and Timeline
- 7 Resource Monitoring
- 8 Networking Troubleshooting
- 9 Storage Troubleshooting
- 10 `systemd` and Quadlet Troubleshooting
- 11 SELinux Troubleshooting
- 12 Failure Drills (Do These in Practice)
- 13 Recovery Playbooks
- Checkpoint
- Quick Quiz
- Further Reading

This module is about turning “it doesn’t work” into a short, repeatable checklist. Diagnosis before action — do not restart blindly.

[^ Go to TOC](#)

21.2 Learning Goals

By the end of this module you will be able to:

- Apply a structured debug loop instead of guessing.
- Locate container logs, events, and inspection data efficiently.
- Debug networking and storage issues methodically.
- Troubleshoot Quadlet + `systemd` service failures.
- Diagnose and resolve SELinux label problems.
- Perform failure drills so you are fast under pressure.
- Build repeatable recovery playbooks for common incidents.

[^ Go to TOC](#)

21.3 Minimum Path (If You Are Short on Time)

- Master the four-step debug loop (Section 1) — use it every time.
- Run the port-conflict and DNS failure drills (Section 12).
- Know the three `journalctl` commands for Quadlet failures (Section 10).

[^ Go to TOC](#)

21.4 1 The Debug Loop (Mental Model)

Before reaching for a restart, follow this loop exactly once:

flowchart TD

```
S(["Something is broken"]) --> A["Step 1: Check state<br/>podman ps -a"]
A --> B["Step 2: Read logs<br/>podman logs / journalctl"]
B --> C["Step 3: Inspect config<br/>podman inspect"]
C --> D{"Root cause<br/>identified?"}
D -->|"Yes"| E["Apply fix"]
D -->|"No"| F["Step 4: Reproduce interactively<br/>podman run --rm -it ... sh"]
F --> G["Trace in isolation"]
G --> E
E --> H(["Verify fix works"])
```

The goal is always to **understand before acting**. Random restarts hide real problems.

21.4.1 1.1 The Four Steps

Step 1 — State check:

```
podman ps -a # list all containers (running and stopped)
```

Look for: status (Exited, Up, Created), uptime, exit codes.

Step 2 — Logs:

```
podman logs <name> # full log output
podman logs --tail 50 <name> # last 50 lines
podman logs -f <name> # follow live
```

Step 3 — Inspect config:

```
podman inspect <name> | less # full JSON: ports, mounts, env, entrypoint
```

Step 4 — Interactive reproduction:

```
podman run --rm -it <same-image> sh # shell into a fresh container
```

If the container exits too fast to exec into, override the entrypoint:

```
podman run --rm -it --entrypoint sh <image> # bypass the app entrypoint
```

[^] Go to TOC

21.5 2 Container State and Lifecycle Commands

21.5.1 2.1 Exit Codes Matter

Exit code	Common meaning
0	Clean exit (process ran to completion or was stopped cleanly)
1	Application error
125	Podman/OCI runtime error (before app started)
126	Command found but not executable
127	Command not found
137	Killed by SIGKILL (OOM, manual kill -9, resource limit)
143	Killed by SIGTERM (normal shutdown, timeout)

```
podman inspect <name> --format '{{.State.ExitCode}}' # get exit code
podman inspect <name> --format '{{.State.Error}}'     # get runtime error string
```

21.5.2 2.2 Container State Transitions

```
flowchart LR
    A(["podman run / create"]) --> B["Created"]
    B --> C["Running"]
    C -->|"process exits"| D["Exited"]
    C -->|"podman stop"| D
    C -->|"OOM kill"| D
    D -->|"podman start"| C
    D -->|"podman rm"| E(["Removed"])
    C -->|"podman pause"| F["Paused"]
    F -->|"podman unpause"| C
```

21.5.3 2.3 Useful State Commands

```
podman ps -a # all containers with status
podman ps --filter status=exited # only exited containers
podman ps --filter name=myapp # filter by name pattern
podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" # custom table
```

^ Go to TOC

21.6 3 Reading Logs

21.6.1 3.1 Basic Log Commands

```
podman logs <name> # full output since container start
podman logs --tail 100 <name> # last N lines
podman logs --since 30m <name> # logs from last 30 minutes
podman logs -f <name> # follow live (Ctrl+C to stop)
podman logs --timestamps <name> # include timestamps
```

21.6.2 3.2 Multiple Containers (Quick Scan)

```
for name in app db proxy; do
    echo "=== $name ===" && podman logs --tail 10 "$name" 2>&1
done # scan logs for multiple containers
```

21.6.3 3.3 When the Container Is Gone

If the container was removed with `--rm`, its logs are gone. This is why ephemeral containers are not suitable for debugging production services.

For Quadlet/systemd services, logs survive in `journald` even after the container restarts:

```
journalctl --user -u cap-mariadb.service -n 200 --no-pager # retrieve logs from journald
```

21.6.4 3.4 Log Verbosity Tricks

Some images respect `DEBUG=1` or `LOG_LEVEL=debug`:

```
podman run --rm -e DEBUG=1 <image> # enable debug logging if app supports it
```

[^ Go to TOC](#)

21.7 4 Deep Inspection with podman inspect

21.7.1 4.1 Full Dump

```
podman inspect <name> | less # full JSON
```

21.7.2 4.2 Targeted Extractions

What ports are published?

```
podman inspect <name> --format '{{json .NetworkSettings.Ports}}' # inspect ports
```

What networks is it on?

```
podman inspect <name> --format '{{json .NetworkSettings.Networks}}' # inspect networks
```

What mounts does it have?

```
podman inspect <name> --format '{{json .Mounts}}' # inspect mounts
```

What is the effective command?

```
podman inspect <name> --format '{{.Config.Cmd}}' # inspect command
```

What user is it running as?

```
podman inspect <name> --format '{{.Config.User}}' # inspect user
```

What is the current exit code?

```
podman inspect <name> --format '{{.State.ExitCode}} {{.State.Error}}' # inspect state
```

What environment variables are set?

```
podman inspect <name> --format '{{range .Config.Env}}{{println .}}{{end}}' # inspect env
```

21.7.3 4.3 Image Inspection

```
podman image inspect <image>:<tag> | less # full image metadata
```

```
podman image history <image>:<tag> # layer history + sizes
```

```
podman image inspect <image>:<tag> --format '{{.Os}}/{{.Architecture}}' # check arch
```

[^ Go to TOC](#)

21.8 5 Interactive Debugging

21.8.1 5.1 Exec Into a Running Container

```
podman exec -it <name> sh # open a shell
```

```
podman exec -it <name> bash # if bash is available
```

```
podman exec -it <name> env # print environment
```

```
podman exec -it <name> ss -tlnp # check listening ports
```

```
podman exec -it <name> cat /etc/resolv.conf # check DNS config
```

21.8.2 5.2 Debug a Failing Container (Override Entrypoint)

```
podman run --rm -it --entrypoint sh <image>:<tag> # bypass CMD/ENTRYPOINT
```

Now you have a shell inside the image and can: - check if files exist at expected paths - check permissions - run the app command manually to see the real error

21.8.3 5.3 Debug with a Sidecar on the Same Network

```
podman run --rm -it --network <same-net> docker.io/library/alpine:latest sh # network debug sidecar
```

From here you can getent hosts <name>, nc -zv <name> <port>, etc.

21.8.4 5.4 netshoot — When You Need More Tools

```
podman run --rm -it --network <net> docker.io/nicolaka/netshoot:latest # network diagnostics image
```

netshoot contains: curl, dig, nmap, tcpdump, ss, iftop, mtr, and more.

[^ Go to TOC](#)

21.9 6 Events and Timeline

podman events gives you a chronological record of Podman operations — starts, stops, network connects, volume mounts, errors.

```
podman events # stream live events
podman events --filter type=container # only container events
podman events --filter container=<name> # events for a specific container
podman events --filter event=die # only death events
podman events --since 1h # last hour of events
podman events --until 30m # events before 30 min ago (relative)
```

Useful for answering: “when did this container restart, and why?”

```
podman events --filter container=<name> --filter event=die --since 24h # find crashes in last day
```

[^ Go to TOC](#)

21.10 7 Resource Monitoring

21.10.1 7.1 Live Stats

```
podman stats # live CPU/mem/net/io for all running containers
podman stats <name> # single container
podman stats --no-stream <name> # one snapshot, then exit
```

21.10.2 7.2 Process Table

```
podman top <name> # show processes (like `ps aux` inside)
podman top <name> pid,user,comm,args # custom columns
```

21.10.3 7.3 Disk Usage Summary

```
podman system df # disk usage: images, containers, volumes
podman system df -v # verbose (per-item)
```

21.10.4 7.4 OOM Kills

If a container exits with code 137, it was OOM killed. Check:

```
journalctl --user -u <service> --since "1 hour ago" | grep -i oom # find OOM kills in journald
podman events --filter event=oom # OOM events
```

Remedy: add --memory limit or fix a memory leak.

^ Go to TOC

21.11 8 Networking Troubleshooting

This section summarizes the networking debug flows. See Module 6 (Section 13) for the full flowchart.

21.11.1 8.1 Checklist: Container Cannot Reach Another by Name

flowchart TD

```
S(["DNS failure"]) --> N1{"Both on same<br/>user-defined network?"}
N1 -->|"No"| F1["podman network connect <net> <name>"]
N1 -->|"Yes"| N2{"dns_enabled on<br/>that network?"}
N2 -->|"No"| F2["Recreate as user-defined network<br/>(default podman net has no DNS)"]
N2 -->|"Yes"| N3{"Both containers<br/>running?"}
N3 -->|"No"| F3["podman start <name>"]
N3 -->|"Yes"| F4["Run debug sidecar<br/>getent hosts <target>"]
```

Quick commands:

Are they on the same network?

```
podman inspect <name> --format '{{json .NetworkSettings.Networks}}' # inspect networks
```

Is DNS enabled?

```
podman network inspect <net> --format '{{.DNSEnabled}}' # check DNS flag
```

Live resolution test

```
podman run --rm --network <net> docker.io/library/alpine:latest sh -lc 'getent hosts <target>' # test
```

21.11.2 8.2 Checklist: Port Reachable from Host

Is the port mapped?

```
podman port <name> # list port mappings
```

Is the container actually listening?

```
podman exec <name> ss -tlnp # inspect listening ports
```

Is the host binding correct (0.0.0.0 vs 127.0.0.1)?

```
podman inspect <name> --format '{{json .NetworkSettings.Ports}}' # check HostIp field
```

Is the host firewall blocking?

```
sudo firewall-cmd --list-all # firewalld rules
```

```
sudo nft list ruleset # nftables rules
```

21.11.3 8.3 Checklist: Container Cannot Reach the Internet

Is the network marked internal?

```
podman network inspect <net> --format '{{.Internal}}' # check internal flag
```

Can it ping a known IP (skips DNS)?

```
podman exec <name> ping -c 3 8.8.8.8 # test raw IP connectivity
```

DNS works?

```
podman exec <name> sh -lc 'getent hosts example.com' # test DNS resolution
```

```
# Check resolv.conf
podman exec <name> cat /etc/resolv.conf # view DNS config
```

^ Go to TOC

21.12 9 Storage Troubleshooting

21.12.1 9.1 Checklist: Permission Denied on a Volume

Common causes: 1. Container runs as a non-root UID that does not own the volume data. 2. SELinux label mismatch (RHEL/Fedora — see Section 11). 3. Bind mount ownership set by the host user, not the container user.

```
# What mounts does the container have?
podman inspect <name> --format '{{json .Mounts}}' # list mounts
```

```
# Check UID inside the container
podman exec <name> id # effective UID/GID
```

```
# Check ownership inside the volume
podman run --rm -v <volname>:/data:ro docker.io/library/alpine:latest ls -lan /data # view ownership
```

Fix: set volume data ownership before starting the service, or use `podman unshare`:

```
podman unshare chown 1000:1000 ~/.local/share/containers/storage/volumes/<volname>/_data # fix ownership
```

21.12.2 9.2 Checklist: Volume Data Missing After Restart

Verify you named the volume correctly and the unit references it:

```
podman volume ls # list volumes
podman volume inspect <volname> # check mount point and driver
podman inspect <name> --format '{{json .Mounts}}' # confirm mount
```

Named volumes are persistent; anonymous volumes (created without a name) are deleted with `podman rm` unless you use `-v` during removal explicitly to keep them.

21.12.3 9.3 Volume Disk Usage

```
podman system df -v # per-volume disk usage
podman volume inspect <volname> --format '{{.Mountpoint}}' # find physical path
```

^ Go to TOC

21.13 10 systemd and Quadlet Troubleshooting

21.13.1 10.1 The Three Commands You Always Need

```
# 1. Is the service running?
systemctl --user status <service> # shows active/failed + last log lines
```

```
# 2. Full log output
journalctl --user -u <service> -n 200 --no-pager # last 200 log lines
```

```
# 3. After changing unit files, reload and restart
systemctl --user daemon-reload && systemctl --user restart <service> # apply unit changes
```

21.13.2 10.2 Quadlet Unit Errors

Quadlet translates `.container`, `.network`, `.volume`, `.kube` files into systemd units. If it fails silently, run:

```
/usr/lib/systemd/user-generators/podman-user-generator "$HOME/.config/containers/systemd" /tmp/quadlet-
```

Or check the systemd generator log:

```
journalctl --user -b --grep quadlet # search boot log for Quadlet errors
```

21.13.3 10.3 Dependency Failures

If a container service fails because a network or volume unit failed first:

```
systemctl --user status capnet.service # check network unit
systemctl --user status mariadb-data-volume.service # check volume unit
journalctl --user -u capnet.service # read network unit logs
```

Quadlet auto-generates `After=` and `Requires=` dependencies when you use `Network=` and `Volume=` in `.container` units. If those dependencies are misconfigured, fix the unit name references.

21.13.4 10.4 Service Does Not Start at Boot

Is linger enabled?

```
loginctl show-user "$USER" | grep Linger # check linger status
```

Is the service enabled?

```
systemctl --user is-enabled <service> # check enable status
```

Enable if needed

```
systemctl --user enable <service> # enable at boot
sudo loginctl enable-linger "$USER" # allow boot start without login
```

21.13.5 10.5 Common Quadlet Troubleshooting Flow

flowchart TD

```

    S(["Service failed to start"]) --> A["systemctl --user status <service>"]
    A --> B["ExecStart error<br/>or unit not found?"]
    B --> C["Unit not found"]
    C --> C1["Check .container file location<br/>~/.config/containers/systemd/"]
    B --> D["ExecStart error"]
    D --> D1["journalctl --user -u <service>"]
    C --> E["Run daemon-reload"]
    D --> F["Image pull<br/>failure?"]
    F --> G["Yes"]
    G --> G1["podman pull <image><br/>Check registry credentials"]
    F --> H["No"]
    H --> H1["Network or volume<br/>dependency failed?"]
    H1 --> I["Yes"]
    I --> I1["Check .network / .volume unit status"]
    H1 --> J["No"]
    J --> J1["Read logs: container startup crash<br/>Use debug loop (Section 1)"]

```

[^ Go to TOC](#)

21.14 11 SELinux Troubleshooting

On RHEL 10 and Fedora, SELinux adds a second layer of access control on top of Unix permissions. Container workloads interact with SELinux primarily through **file labels**.

21.14.1 11.1 Most Common Symptom

A container exits with `permission denied` even though Unix permissions look correct.

21.14.2 11.2 Check the Denial

```
sudo ausearch -m avc -ts recent | tail -30 # show recent SELinux denials
sudo journalctl -k --grep avc | tail -30 # kernel AVC denials
```

21.14.3 11.3 The :Z Fix for Bind Mounts

For bind mounts (host paths mounted into containers), SELinux requires the correct label:

```
# Apply private label (owned by this one container only)
podman run -v /host/path:/container/path:Z <image> # relabel for single container
```

```
# Apply shared label (readable by multiple containers)
podman run -v /host/path:/container/path:z <image> # relabel for shared use
```

:Z relabels the **entire host directory** — use with caution on important paths. Named volumes (not bind mounts) are automatically labeled correctly by Podman.

21.14.4 11.4 Check Current Labels

```
ls -laZ /host/path # show SELinux context
podman exec <name> ls -laZ /container/path # show label inside container
```

21.14.5 11.5 When in Doubt: Prefer Named Volumes

Named volumes (`podman volume create`) are managed by Podman and automatically receive correct SELinux labels. Bind mounts require manual label management.

[^] Go to TOC

21.15 12 Failure Drills (Do These in Practice)

Do these deliberately. Running scenarios on purpose makes you significantly faster during real incidents.

21.15.1 Drill 1 — Port Conflict

```
# Start a service on 8080
podman run -d --name svc1 -p 8080:80 docker.io/library/nginx:stable # start first service

# Try to start a second on the same port
podman run -d --name svc2 -p 8080:80 docker.io/library/nginx:stable # this should fail

# Observe: error message, exit code
podman ps -a # check state
podman logs svc2 # read the error

# Fix: change port
podman run -d --name svc2 -p 8081:80 docker.io/library/nginx:stable # use different port

# Cleanup
podman rm -f svc1 svc2 # cleanup
```

Expected observation: Podman reports address already in use or similar.

21.15.2 Drill 2 — Bad Command (Immediate Exit)

```
podman run -d --name badcmd docker.io/library/alpine:latest /bin/nonexistent # bad command

# Observe
podman ps -a # status: Exited(127)
podman logs badcmd # "not found"
podman inspect badcmd --format '{{.State.ExitCode}}' # should be 127

# Cleanup
podman rm badcmd # cleanup
```

21.15.3 Drill 3 — DNS Failure (Wrong Network)

```
podman network create net-a # create network
podman network create net-b # create another network

podman run -d --name server-a --network net-a docker.io/library/alpine:latest sleep 600 # start server
podman run -d --name client-b --network net-b docker.io/library/alpine:latest sleep 600 # wrong network

# DNS fails
podman exec client-b sh -lc 'getent hosts server-a || echo DNS FAILED' # this should fail

# Fix: connect to correct network
podman network connect net-a client-b # attach to same network
podman exec client-b sh -lc 'getent hosts server-a && echo DNS OK' # now works

# Cleanup
podman rm -f server-a client-b # cleanup containers
podman network rm net-a net-b # cleanup networks
```

21.15.4 Drill 4 — Permission Denied on Volume

```
podman volume create drill-vol # create volume

# Write as root (uid 0)
podman run --rm -v drill-vol:/data docker.io/library/alpine:latest sh -lc 'echo data > /data/file.txt && echo ok > /data/file.txt'

# Try to write as a different user
podman run --rm -v drill-vol:/data --user 1000:1000 docker.io/library/alpine:latest sh -lc 'echo fail > /data/file.txt'

# Fix: correct ownership
podman run --rm -v drill-vol:/data docker.io/library/alpine:latest chown -R 1000:1000 /data # fix ownership
podman run --rm -v drill-vol:/data --user 1000:1000 docker.io/library/alpine:latest sh -lc 'echo ok > /data/file.txt'

# Cleanup
podman volume rm drill-vol # cleanup
```

[^ Go to TOC](#)

21.16 13 Recovery Playbooks

Keep short, tested playbooks for common incidents. Copy these and adapt to your services.

21.16.1 Playbook: Service Not Starting

1. `systemctl --user status <service>` — check status
2. `journalctl --user -u <service> -n 100` — read logs
3. `podman inspect <name>` — check config (command, mounts, ports)
4. `podman events --filter container=<name> --since 1h` — check event timeline
5. Fix root cause (image, config, resource)
6. `systemctl --user daemon-reload && systemctl --user restart <service>` — apply fix
7. Verify: `systemctl --user status <service>`

21.16.2 Playbook: Service Crashing in a Loop

1. `systemctl --user status <service>` — note restart count
2. `journalctl --user -u <service> --since "10 min ago"` — get crash context
3. `podman run --rm -it --entrypoint sh <image>` — reproduce in isolation
4. Fix the crash cause (binary, config, env, permissions)
5. Reload + restart, watch logs: `journalctl --user -fu <service>`

21.16.3 Playbook: Data Volume Issue

1. `podman inspect <name> --format '{{json .Mounts}}'` — verify mount
2. `podman run --rm -v <vol>:/data:ro alpine ls -lan /data` — check contents
3. `podman volume inspect <vol>` — check driver and mount point
4. Fix permissions / label / volume name
5. Restart container

[^ Go to TOC](#)

21.17 Checkpoint

You should be able to answer the following without looking at commands:

- What are the four steps of the debug loop, in order?
- What exit code means “command not found”?
- What is the difference between `podman logs` and `journalctl --user -u`?
- What does `:Z` do on a bind mount, and why might you prefer a named volume?
- How do you check whether SELinux is blocking a container?
- What is the first command you run when a Quadlet service fails?
- How do you find the crash timestamp of a container that already restarted?

[^ Go to TOC](#)

21.18 Quick Quiz

1. A container exits immediately with code 127. What does this mean and where do you look first?
2. You have a running container that you cannot exec into because the shell is not installed. What alternative debugging technique gives you the most information?
3. `podman logs` shows nothing but the service is crashing. Why might this happen and where else do you look?
4. Your Quadlet `.container` unit was edited. What two commands must you run in order for the change to take effect?
5. A container can ping 8.8.8.8 but cannot resolve `my-service`. What does this tell you, and what is the most likely fix?

6. A volume write fails with `permission denied`. You confirm SELinux is not the issue and Unix permissions look correct. What else could cause this?

^ Go to TOC

21.19 Further Reading

- `podman-events(1)`: <https://docs.podman.io/en/latest/markdown/podman-events.1.html>
- `podman-stats(1)`: <https://docs.podman.io/en/latest/markdown/podman-stats.1.html>
- `podman-inspect(1)`: <https://docs.podman.io/en/latest/markdown/podman-inspect.1.html>
- `systemd journalctl`: <https://www.freedesktop.org/software/systemd/man/latest/journalctl.html>
- `systemd-analyze(1)` (verify, generators): <https://www.freedesktop.org/software/systemd/man/latest/systemd-analyze.html>
- SELinux container troubleshooting: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/using_selinux_for_containers/using_selinux_for_containers-chapter-10.html
- netshoot debug image: <https://github.com/nicolaka/netshoot>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

22 Module 14: Maintenance and Auto-Updates



22.1 Table of Contents

- Learning Goals
- What Auto-Update Is
- How It Works — Internals
- Tags, Digests, and Policy
- The `io.containers.autoupdate` Label
- Lab: Enable Registry Auto-Update (Single Service)
- Healthchecks and Auto-Rollback
- Automating Auto-Update with a `systemd` Timer
- Rollback Plan (Required If You Auto-Update)
- Safe Rollout Rules
- Routine Maintenance Tasks
- Checkpoint
- Quick Quiz
- Further Reading

Auto-updates can be useful, but they are a policy decision — not a default.

[^] Go to TOC

22.2 Learning Goals

- Explain what Podman auto-update does and what it does not do.
- Configure a Quadlet-managed container for registry-based auto-update.
- Understand the `io.containers.autoupdate` label and its values.
- Build a safe rollout + rollback plan.
- Understand when digest pinning is safer than tag-based auto-update.
- Know the routine maintenance tasks for a Podman host.

[^] Go to TOC

22.3 What Auto-Update Is

`podman auto-update` is a command that:

1. Finds all running containers labeled with `io.containers.autoupdate`.
2. Checks if a newer image is available (by pulling from the registry).
3. If yes: restarts the `systemd` unit that manages the container.

flowchart TD

```
A["podman auto-update<br/>(manual or via timer)"] --> B["Find containers with<br/>io.containers.autoupdate"]
B --> C{"label = registry?"}
C -->|"Yes"| D["Pull image from registry"]
D --> E{"Digest changed?"}
E -->|"Yes"| F["systemctl restart <unit>"]
E -->|"No"| G["No-op - already up to date"]
C -->|"local"| H["Check if local image changed<br/>(e.g., after manual build)"]
H --> I{"Healthcheck passes?"}
I -->|"Yes"| J["Update complete"]
I -->|"No + --rollback"| K["Revert to previous image<br/>restart unit"]
```

What it **cannot** do by itself:

- Perform multi-step database migrations before restarting.
- Coordinate updates across multiple hosts simultaneously.
- Guarantee backward compatibility between versions.
- Drain connections before restarting.

^ Go to TOC

22.4 How It Works — Internals

When `podman auto-update` detects a new image and restarts a unit:

1. It pulls the new image and records the old image digest.
2. It calls `systemctl restart <unit>` — the unit stops the old container and starts a new one with the new image.
3. If the container has a healthcheck and the `--rollback` flag was passed (or the unit has `AutoUpdatePolicy=registry` with rollback configured), Podman waits for the healthcheck to pass.
4. If the healthcheck fails, it reverts to the old image and restarts.

The unit must be a Quadlet-generated service for this to work. Auto-update does not manage containers started with bare `podman run`.

sequenceDiagram

```

participant Timer as "systemd timer<br/>(optional)"
participant AU as "podman auto-update"
participant Registry
participant systemd
participant Container

Timer->>AU: trigger
AU->>Registry: pull image (check digest)
Registry->>AU: new digest available
AU->>AU: record old digest for rollback
AU->>systemd: systemctl restart myapp.service
systemd->>Container: stop old container
systemd->>Container: start new container (new image)
Container->>AU: healthcheck status
alt healthcheck passes
    AU->>AU: update complete
else healthcheck fails
    AU->>systemd: systemctl restart (with old image)
    AU->>AU: log rollback event
end
end

```

^ Go to TOC

22.5 Tags, Digests, and Policy

Auto-update works with **tags** (mutable pointers like `:stable`, `:latest`, `:3.2`). Tags can move — `:stable` today may point to a different image tomorrow. This is what enables auto-update to detect changes.

Digest pinning (e.g., `@sha256:abc123...`) is **immutable** — it always refers to the exact same bytes. Auto-update cannot move a digest — there is nothing to update.

flowchart LR

```

T["Tag: nginx:stable"] -->|"can move"| I1["Image A (sha256:aaa...)"]
T -->|"after update"| I2["Image B (sha256:bbb...)"]
D["Digest: nginx@sha256:aaa..."] -->|"always points to"| I1

```

```
style T fill:#1a3a1a,color:#ccffcc
style D fill:#3a1a1a,color:#ffcccc
```

Choose intentionally:

Strategy	Tag	Digest pinned
Always latest patch automatically	[OK] Use tag + auto-update	[X] Cannot auto-update
Reproducible, audited upgrades	[X] Tags drift	[OK] Explicit digest bump
Regulated environment	Only if you have monitoring + rollback	Default choice
Development server	Acceptable	Overkill

In regulated or production environments, **digest pinning is the default**. Auto-update with tags is an operational choice that requires monitoring, rollback readiness, and health checks.

^ Go to TOC

22.6 The `io.containers.autoupdate` Label

For `podman auto-update` to manage a container, the container (or its image) must have the label:

```
io.containers.autoupdate=registry
```

In a Quadlet `.container` file, set it with:

```
[Container]
AutoUpdate=registry
```

This translates to `--label io.containers.autoupdate=registry` on the `podman run` command.

Possible values:

Value	Behaviour
<code>registry</code>	Pull from registry, compare digest, restart if changed
<code>local</code>	Only check local image store — useful after a local <code>podman build</code>
(absent)	Container is ignored by <code>podman auto-update</code>

For the `local` policy, this is useful when you build images locally with a CI step and want `systemd` to restart on a new local build.

^ Go to TOC

22.7 Lab: Enable Registry Auto-Update (Single Service)

Use the provided example unit:

- `examples/quadlet/autoupdate-nginx.container`

Step 1: Install the unit:

```
mkdir -p ~/.config/containers/systemd # ensure path exists
cp examples/quadlet/autoupdate-nginx.container ~/.config/containers/systemd/ # install unit
systemctl --user daemon-reload # regenerate units
systemctl --user start autoupdate-nginx.service # start service
systemctl --user status autoupdate-nginx.service # verify running
```

Step 2: Inspect the label on the running container:

```
podman inspect systemd-autoupdate-nginx --format='{{index .Config.Labels "io.containers.autoupdate"}}'
```

Step 3: Trigger auto-update manually:

```
podman auto-update # check registry for updates, restart labeled units if changed
```

Expected output: shows which containers were checked and whether any were updated.

Step 4: Trigger with verbose output:

```
podman auto-update --dry-run # show what WOULD be updated without actually restarting
```

Step 5: Observe via journald:

```
journalctl --user -u autoupdate-nginx.service -n 100 --no-pager # view container logs and restarts
```

Step 6: Check for a systemd timer (if available on your distro):

```
systemctl --user list-unit-files | grep auto-update # look for podman-auto-update.timer
```

If present, you can enable it to run auto-update on a schedule.

Cleanup:

```
systemctl --user stop autoupdate-nginx.service # stop  
rm -f ~/.config/containers/systemd/autoupdate-nginx.container # remove unit  
systemctl --user daemon-reload # remove generat
```

^ Go to TOC

22.8 Healthchecks and Auto-Rollback

Auto-update's rollback feature only works if:

1. The container has a healthcheck (HEALTHCHECK instruction in Containerfile, or HealthCmd= in the Quadlet unit).
2. The podman auto-update command is run with --rollback (or rollback is configured in the unit).

Adding a healthcheck in a Quadlet unit:

```
[Container]  
Image=docker.io/library/nginx:stable  
AutoUpdate=registry  
HealthCmd=CMD-SHELL curl -f http://localhost/ || exit 1  
HealthInterval=10s  
HealthTimeout=3s  
HealthRetries=3  
HealthStartPeriod=5s
```

Running auto-update with rollback:

```
podman auto-update --rollback # update and rollback automatically if healthcheck fails
```

If the new image starts but the healthcheck fails within the startup period, Podman reverts to the previous image and restarts the unit.

Check rollback events:

```
podman auto-update --rollback 2>&1 # stderr shows rollback events  
journalctl --user -b -n 100 --no-pager | grep -i autoupdate # look for auto-update log entries
```

^ Go to TOC

22.9 Automating Auto-Update with a systemd Timer

Rather than a cron job, use a systemd user timer. Create two files:

`~/.config/systemd/user/podman-auto-update.service:`

```
[Unit]
Description=Podman auto-update containers
Documentation=man:podman-auto-update(1)

[Service]
Type=oneshot
ExecStart=/usr/bin/podman auto-update --rollback
```

`~/.config/systemd/user/podman-auto-update.timer:`

```
[Unit]
Description=Podman auto-update timer

[Timer]
# Run at 3:00 AM daily:
OnCalendar=*-*-* 03:00:00
RandomizedDelaySec=600
Persistent=true

[Install]
WantedBy=timers.target
```

Enable and start:

```
mkdir -p ~/.config/systemd/user           # ensure directory exists
# (write the files above)

systemctl --user daemon-reload             # pick up new units
systemctl --user enable --now podman-auto-update.timer # enable timer to start at boot
systemctl --user list-timers podman-auto-update.timer # verify timer is scheduled
```

Note: on some distributions (Fedora, RHEL), Podman ships a pre-built `podman-auto-update.timer` you can simply enable. Check first:

```
systemctl --user list-unit-files | grep podman-auto-update # check if pre-built timer exists
```

^ Go to TOC

22.10 Rollback Plan (Required If You Auto-Update)

You must have a rollback plan before enabling auto-update in production. This is not optional.

Minimum rollback plan:

1. **Record the previous working digest** before updating (or have a way to retrieve it from the registry).
2. **Pin and restart:** update the Quadlet unit with the previous digest and restart.
3. **Monitoring:** detect restart loops quickly (alert on `systemd` unit restarted >3 times in 5 minutes).

flowchart TD

```
A["Auto-update triggers"] --> B{"Healthcheck passes?"}
B -->|"Yes"| C["Update complete<br/>Record new digest as known-good"]
B -->|"No"| D["Rollback: set Image= to old digest"]
D --> E["systemctl --user daemon-reload"]
E --> F["systemctl --user restart myapp.service"]
```

```
F --> G["Alert on-call: rollback occurred"]
G --> H["Investigate: check logs<br/>podman diff<br/>compare changelogs"]
```

If you rely on tags (mutable), your rollback procedure needs to:

- Pin to the specific old digest (record it before the update).
- Or maintain a local image cache/registry with the previous version.

```
# Record the current digest BEFORE auto-update runs:
PREV=$(podman inspect --format='{{.Image}}' systemd-myapp)
echo "Rollback image: $PREV" # save this somewhere

# To rollback manually:
# Edit unit: Image=docker.io/library/nginx@sha256:<PREV_DIGEST>
# Then:
systemctl --user daemon-reload && systemctl --user restart myapp.service

^ Go to TOC
```

22.11 Safe Rollout Rules

1. **Prefer digest-pinned images for production** unless you explicitly accept the risk of tag-based updates.
2. **Always have healthchecks** before enabling auto-update. Without them, a broken image will restart successfully and you won't know until users report errors.
3. **Test in staging first:** auto-update staging, verify, then allow production.
4. **Alert on restart loops:** a container restarting 5 times in 2 minutes is a signal.
5. **Coordinate with DB migrations:** if your update includes a DB schema migration, auto-update is not the right tool — use a controlled deploy.
6. **Document which services have auto-update enabled:** make it visible in your runbook.

Treat auto-update as an **operational feature**, not a convenience hack.

^ Go to TOC

22.12 Routine Maintenance Tasks

Beyond auto-update, here are the routine Podman maintenance tasks you should schedule:

Prune unused images (safe to run regularly — only removes images with no running containers):

```
podman image prune -f # remove dangling (untagged) images
podman image prune -a -f # remove ALL images not used by any container (more aggressive)
```

Prune stopped containers:

```
podman container prune -f # remove all stopped containers
```

Prune unused volumes (!) DATA LOSS — only run if you are sure):

```
podman volume prune -f # remove volumes not used by any container
```

Prune everything unused at once:

```
podman system prune -f # remove stopped containers, unused images, unused networks
podman system prune -a -f # also removes unused volumes (destructive!)
```

Check disk usage:

```
podman system df # show disk usage by images, containers, volumes
```

Recommended maintenance schedule:

Task	Frequency	Risk
<code>podman image prune -f</code>	Weekly	Low
<code>podman container prune -f</code>	Daily	Low
<code>podman system df</code>	Weekly (just observing)	None
<code>podman auto-update</code>	Daily (if enabled)	Medium — requires rollback plan
<code>podman volume prune</code>	Manual only	High — DATA LOSS

^ Go to TOC

22.13 Checkpoint

- You can explain what `podman auto-update` does and what it requires (Quadlet-managed + label).
- You can explain the difference between tag-based auto-update and digest pinning.
- You can enable auto-update for a single service using `AutoUpdate=registry` in a Quadlet unit.
- You can articulate a minimum rollback plan.
- You can set up a systemd timer to run auto-update on a schedule.
- You know the routine maintenance commands (`image prune`, `container prune`, `system df`).

^ Go to TOC

22.14 Quick Quiz

- 1) Why can auto-update increase risk for stateful services (e.g., a database)?
- 2) What must exist before you turn on auto-update in production?
- 3) You have `Image=docker.io/library/nginx@sha256:abc123` in your Quadlet unit. Will `podman auto-update` do anything? Why?
- 4) Auto-update runs but the new container immediately fails its healthcheck. What happens if you passed `--rollback`?
- 5) What is the difference between `podman image prune -f` and `podman image prune -a -f`?
- 6) A container is restarting every 30 seconds. How do you tell if this is caused by auto-update or a bad restart policy?

^ Go to TOC

22.15 Further Reading

- `podman-auto-update(1)`: <https://docs.podman.io/en/latest/markdown/podman-auto-update.1.html>
- systemd timers: <https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html>
- systemd service restart policies: <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html>
- `podman-system-prune(1)`: <https://docs.podman.io/en/latest/markdown/podman-system-prune.1.html>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

23 Capstone: Reboot-Safe Local Stack with Secrets, Backups, and Upgrades



23.1 Table of Contents

- Goal
- Reference Stack
- Deliverables
- Architecture Overview
- Build It
- First Data (Required)
- Optional: Scheduled Backups
- Backup and Restore (Required)
- Upgrade and Rollback (Required)
- Password Rotation (Required)
- Operations Runbook
- Notes
- Checkpoint
- Quick Quiz
- Further Reading

This capstone focuses on **operational excellence**, not app development. You have all the individual skills — now wire them together into a production-grade pattern.

[^ Go to TOC](#)

23.2 Goal

Run a small stack as rootless systemd user services (Quadlet-first) that:

- survives reboot without manual intervention
- keeps state in named volumes (never in container layers)
- uses secrets as mounted files (never as environment variables)
- has a tested backup + restore flow
- has an upgrade + rollback flow using digest-pinned images

Success criteria:

- After a full system reboot, both services come back automatically.
- You can produce a backup file and prove you can restore it to a clean volume.
- You can upgrade MariaDB and Adminer versions with a documented rollback to previous digests.
- DB has **no published host ports**.

[^ Go to TOC](#)

23.3 Reference Stack

- **DB:** MariaDB 11 — stateful, password-protected, no published ports
- **UI:** Adminer — web DB admin, published to localhost only

This gives you a realistic stateful service without writing any application code. Every pattern here applies directly to production app stacks.

[^ Go to TOC](#)

23.4 Deliverables

At the end of this capstone you should have:

Quadlet unit files (in ~/.config/containers/systemd/): - `capnet.network` — private bridge, DNS enabled, internal - `mariadb-data.volume` — persistent DB volume - `cap-backups.volume` — backup output volume - `cap-mariadb.container` — DB service, digest-pinned - `cap-adminer.container` — UI service, digest-pinned - `cap-backup.container` (*optional*) — backup job container

Written runbook covering: - First deploy procedure - DB password rotation - Manual backup + restore - Upgrade procedure (change digest -> reload -> restart) - Rollback procedure (restore previous digest -> reload -> restart)

^ Go to TOC

23.5 Architecture Overview

flowchart TD

```
subgraph "Host OS (rootless user)"
  subgraph "systemd --user"
    QN["capnet.network<br/>(Quadlet .network unit)"]
    QV["mariadb-data.volume<br/>(Quadlet .volume unit)"]
    QBV["cap-backups.volume<br/>(Quadlet .volume unit)"]
    QDB["cap-mariadb.container<br/>(Quadlet .container unit)"]
    QUI["cap-adminer.container<br/>(Quadlet .container unit)"]
  end
  subgraph "capnet (--internal bridge)"
    DB["MariaDB<br/>alias: db<br/>no host port"]
    UI["Adminer<br/>port 127.0.0.1:8082:8080"]
  end
  SEC["Podman Secret<br/>mariadb_root_password"]
  VOL["Volume: mariadb-data"]
  BVOL["Volume: cap-backups"]
end
BROWSER["Browser<br/>http://127.0.0.1:8082/"] -->|"port 8082"| UI
UI -->|"SQL via DNS alias 'db'"| DB
DB --- VOL
SEC -->|"mounted as file<br/>/run/secrets/..."| DB
SEC -->|"mounted as file<br/>/run/secrets/..."| UI
QN --> DB
QN --> UI
QV --> VOL
QBV --> BVOL
QDB --> DB
QUI --> UI
```

Key design decisions: - The `capnet` network is `--internal`: DB cannot make outbound connections. - The DB secret is a **Podman secret** mounted as a file — never passed as an env var. - Both containers are managed by systemd with `WantedBy=default.target` for boot start. - Images are pinned to SHA256 digests for reproducible deploys and clean rollbacks.

^ Go to TOC

23.6 Build It

Use the provided example units:

- `examples/quadlet/capnet.network`

- examples/quadlet/mariadb-data.volume
- examples/quadlet/cap-backups.volume
- examples/quadlet/cap-mariadb.container
- examples/quadlet/cap-adminer.container
- examples/quadlet/cap-backup.container (optional)

23.6.1 Step 1 — Create the DB Root Password Secret

Choose a password without quotes or newlines to avoid shell/SQL escaping issues.

```
read -s -p 'MariaDB root password: ' P # prompt for password input
printf '\n' # print newline after silent input
printf '%s' "$P" | podman secret create mariadb_root_password - # create secret from stdin
unset P # clear password from shell memory
```

Verify the secret exists (value is never shown):

```
podman secret ls # list secrets
```

23.6.2 Step 2 — Install Quadlet Units

```
mkdir -p ~/.config/containers/systemd # create Quadlet unit directory
cp examples/quadlet/capnet.network ~/.config/containers/systemd/ # copy network unit
cp examples/quadlet/mariadb-data.volume ~/.config/containers/systemd/ # copy DB volume unit
cp examples/quadlet/cap-backups.volume ~/.config/containers/systemd/ # copy backup volume unit
cp examples/quadlet/cap-mariadb.container ~/.config/containers/systemd/ # copy DB container unit
cp examples/quadlet/cap-adminer.container ~/.config/containers/systemd/ # copy UI container unit
```

23.6.3 Step 3 — Enable Linger (Boot Start Without Login)

```
sudo loginctl enable-linger "$USER" # allow user services to start at boot without a login session
```

Verify:

```
loginctl show-user "$USER" | grep Linger # should show Linger=yes
```

23.6.4 Step 4 — Start Services

```
systemctl --user daemon-reload # regenerate systemd units from Quadlet files
systemctl --user start cap-mariadb.service # start DB first
systemctl --user start cap-adminer.service # start UI
```

23.6.5 Step 5 — Validate

Check service status:

```
systemctl --user status cap-mariadb.service # DB status
systemctl --user status cap-adminer.service # UI status
```

Adminer should be available at <http://127.0.0.1:8082/>

Verify DB has **no published host ports**:

```
podman port cap-mariadb || true # expected: no output (no published ports)
```

Test connectivity inside the stack:

```
podman run --rm --network capnet --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; mysql -h db -u root -e "SHOW DATABASES"
```

Expected: list of databases including `information_schema`.

^ Go to TOC

23.7 First Data (Required)

Create test data so you have something meaningful to back up and restore.

```
podman run --rm --network capnet --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; \
  mysql -h db -u root -e "
    CREATE DATABASE IF NOT EXISTS cap;
    CREATE TABLE IF NOT EXISTS cap.t1 (id INT PRIMARY KEY, label VARCHAR(80));
    INSERT IGNORE INTO cap.t1 VALUES (1, ''''first row''');
    INSERT IGNORE INTO cap.t1 VALUES (2, ''''second row''');
    SELECT * FROM cap.t1;
  ' # create and populate test table
```

Expected: a two-row result set.

^ Go to TOC

23.8 Optional: Scheduled Backups

23.8.1 Install Backup Units and Timer

```
cp examples/quadlet/cap-backup.container ~/.config/containers/systemd/ # copy backup container unit
mkdir -p ~/.config/systemd/user # create user systemd directory
cp examples/systemd-user/cap-backup.timer ~/.config/systemd/user/ # copy timer unit
systemctl --user daemon-reload # reload new units
systemctl --user enable --now cap-backup.timer # enable and start timer
```

23.8.2 Trigger a Backup Immediately

```
systemctl --user start cap-backup.service # run a one-shot backup now
```

23.8.3 Verify Backup Files Exist

```
podman run --rm -v cap_backups:/backups docker.io/library/alpine:latest ls -lah /backups # list backup
```

Notes: - Backups are stored in the `cap_backups` volume as timestamped `.sql` files. - The backup unit runs `mysqldump` inside a MariaDB container — no client tools needed on the host. - The timer runs daily by default; edit the `.timer` file to adjust.

^ Go to TOC

23.9 Backup and Restore (Required)

23.9.1 The Backup Lifecycle

flowchart LR

```
DB["MariaDB<br/>(cap-mariadb)"] -->|"mysqldump via capnet"| BJ["Backup Job Container<br/>(cap-backup)"]
BJ -->|"writes all-<timestamp>.sql"| BV["cap-backups volume"]
BV -->|"copy out for offsite"| HOST["Host filesystem<br/>(optional export)"]
HOST -->|"restore on disaster"| DB2["Fresh MariaDB<br/>container"]
```

23.9.2 Backup (Manual)

Trigger a backup immediately:

```
systemctl --user start cap-backup.service # run backup job
```

Find the newest backup file:

```
podman run --rm -v cap_backups:/backups docker.io/library/alpine:latest sh -lc 'ls -lt /backups | head -1'
```

Export a backup file to the host (optional):

```
BACKUP_FILE="all-$(date +%Y%m%d%H%M%S).sql"
podman run --rm -v cap_backups:/backups docker.io/library/alpine:latest \
  sh -lc "cat /backups/\$(ls -lt /backups | head -1)" > "/tmp/${BACKUP_FILE}" # export backup to host
echo "Saved: /tmp/${BACKUP_FILE}" # confirm export
```

23.9.3 Restore (Manual)

Pick the backup file name from the list above, then restore:

```
BACKUP_FILE=all-<timestamp>.sql # replace with actual filename
```

```
podman run --rm --network capnet -v cap_backups:/backups --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; \
  mysql -h db -u root < "/backups/"$BACKUP_FILE"' # restore from backup file
```

Verify the data is present after restore:

```
podman run --rm --network capnet --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; \
  mysql -h db -u root -e "SELECT * FROM cap.t1;"' # verify restored data
```

23.9.4 Testing Restore on a Clean Volume (Advanced)

For a true restore test, create a fresh volume, start a temporary DB on it, restore the backup, verify the data, then discard the test volume:

```
podman volume create cap-restore-test # create a clean volume for restore testing
```

```
# Start a temporary DB on the clean volume
```

```
podman run -d --name test-db --network capnet \
  -v cap-restore-test:/var/lib/mysql \
  --secret mariadb_root_password \
  -e MARIADB_ROOT_PASSWORD_FILE=/run/secrets/mariadb_root_password \
  docker.io/library/mariadb:11 # start temporary test DB
```

```
sleep 10 # wait for MariaDB init
```

```
# Restore the backup into test-db
```

```
BACKUP_FILE=all-<timestamp>.sql
```

```
podman run --rm --network capnet -v cap_backups:/backups --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; \
  mysql -h test-db -u root < "/backups/"$BACKUP_FILE"' # restore into test DB
```

```
# Verify
```

```
podman run --rm --network capnet --secret mariadb_root_password \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password)"; \
  mysql -h test-db -u root -e "SELECT * FROM cap.t1;"' # verify test DB data
```

Cleanup test resources

```
podman rm -f test-db # remove test container
podman volume rm cap-restore-test # remove test volume
```

This pattern is called **restore-to-alternate** and proves your backup is actually usable before you ever need it in production.

^ Go to TOC

23.10 Upgrade and Rollback (Required)

23.10.1 The Upgrade/Rollback Lifecycle

flowchart TD

```
A(["Current: digest v1"]) --> B["Record current digest<br/>podman inspect --format ImageName"]
B --> C["Pull new image<br/>podman pull mariadb:11"]
C --> D["Get new digest<br/>podman images --digests"]
D --> E["Edit .container unit<br/>Image=...@sha256:newdigest"]
E --> F["daemon-reload + restart"]
F --> G{"Service healthy?"}
G -->|"Yes"| H(["Upgrade complete"])
G -->|"No"| I["Edit .container unit<br/>Image=...@sha256:olddigest"]
I --> J["daemon-reload + restart"]
J --> K(["Rollback complete"])
```

23.10.2 Record Current Image Digests (Before Any Upgrade)

```
podman inspect cap-mariadb --format '{{.ImageName}}' # current DB image with digest
podman inspect cap-adminer --format '{{.ImageName}}' # current UI image with digest
podman images --digests | grep -E 'mariadb|adminer' # all local digests
```

Save these to a file before upgrading:

```
podman inspect cap-mariadb --format '{{.ImageName}}' > /tmp/current-digests.txt # save DB digest
podman inspect cap-adminer --format '{{.ImageName}}' >> /tmp/current-digests.txt # save UI digest
cat /tmp/current-digests.txt # verify saved digests
```

23.10.3 Pin by Digest in Quadlet Units

In your `.container` files, replace tag-based references with digest-pinned ones:

```
# Before (mutable tag - can change without warning)
Image=docker.io/library/mariadb:11

# After (immutable - this exact SHA will always be the same)
Image=docker.io/library/mariadb@sha256:<digest>
```

Then apply:

```
systemctl --user daemon-reload # regenerate units after edits
systemctl --user restart cap-mariadb.service # restart DB with new image
systemctl --user restart cap-adminer.service # restart UI with new image
```

23.10.4 Rollback

Simply restore the old digest values in the `.container` files and repeat the same two commands:

```
# Edit unit files to restore previous digests, then:
systemctl --user daemon-reload
systemctl --user restart cap-mariadb.service
systemctl --user restart cap-adminer.service
```

Verify the rollback:

```
podman inspect cap-mariadb --format '{{.ImageName}}' # confirm old digest is back
```

^ Go to TOC

23.11 Password Rotation (Required)

Password rotation is a two-phase operation: you change the password in the DB engine first, then update the secret reference, then restart the service. Never delete the old secret until the new one is proven.

23.11.1 Rotation Flow

flowchart TD

```
A(["Start rotation"]) --> B["Create new secret<br/>mariadb_root_password_v2"]
B --> C["Run ALTER USER inside DB<br/>(uses both old and new secrets)"]
C --> D["Update .container unit<br/>Secret=mariadb_root_password_v2"]
D --> E["daemon-reload + restart MariaDB"]
E --> F{"Login with new secret<br/>works?"}
F -->|"Yes"| G["podman secret rm mariadb_root_password_v1"]
F -->|"No"| H["Restore old secret reference<br/>daemon-reload + restart"]
G --> I(["Rotation complete"])
H --> J(["Rollback to old password"])
```

23.11.2 Step 1 — Create New Secret

```
read -s -p 'New MariaDB root password: ' P # prompt for new password
printf '\n' # print newline
printf '%s' "$P" | podman secret create mariadb_root_password_v2 - # create new versioned secret
unset P # clear from memory
```

23.11.3 Step 2 — Change the Password in MariaDB (While Old One Is Still Active)

```
podman run --rm --network capnet \
  --secret mariadb_root_password \
  --secret mariadb_root_password_v2 \
  docker.io/library/mariadb:11 sh -lc '
  old=$(cat /run/secrets/mariadb_root_password)
  new=$(cat /run/secrets/mariadb_root_password_v2)
  export MYSQL_PWD="$old"
  mysql -h db -u root -e "ALTER USER \"root\"@\"%\" IDENTIFIED BY \"${new}\"; FLUSH PRIVILEGES;"
  ' # change password in DB using both old and new secrets
```

23.11.4 Step 3 — Update Quadlet Unit to Reference New Secret

Edit `~/config/containers/systemd/cap-mariadb.container` — change:

```
# Old
Secret=mariadb_root_password,type=mount
```

```
# New
Secret=mariadb_root_password_v2,type=mount
```

23.11.5 Step 4 — Reload and Restart

```
systemctl --user daemon-reload # reload unit changes
systemctl --user restart cap-mariadb.service # restart with new secret
```

23.11.6 Step 5 — Verify Login with New Secret

```
podman run --rm --network capnet --secret mariadb_root_password_v2 \
  docker.io/library/mariadb:11 sh -lc \
  'export MYSQL_PWD="$(cat /run/secrets/mariadb_root_password_v2)"; \
  mysql -h db -u root -e "SELECT 1;"' # verify new password works
```

23.11.7 Step 6 — Remove Old Secret Only After Verification

```
podman secret rm mariadb_root_password # remove old secret after confirmed rotation
```

^ Go to TOC

23.12 Operations Runbook

Keep this as a living document for your stack. A runbook that is never tested is not a runbook.

Operation	Commands	Validation
First deploy	Copy units, daemon-reload, start	systemctl status, curl 127.0.0.1:8082
After reboot	Automatic (linger + WantedBy=default.target)	systemctl status, check data
Tail logs	journalctl --user -fu cap-mariadb.service	—
Manual backup	systemctl --user start cap-backup.service	ls in cap_backups volume
Restore backup	See “Restore” section	SELECT * to confirm row count
Upgrade image	Edit digest in unit, daemon-reload, restart	podman inspect --format ImageName
Rollback image	Restore old digest, daemon-reload, restart	podman inspect --format ImageName
Rotate password	6-step rotation procedure above	mysql -u root with new secret
Full teardown	systemctl --user stop + podman volume rm	—

^ Go to TOC

23.13 Notes

- Password rotation often implies updating both the Podman secret and the DB user credentials in the correct order.
- Keep the old password available until the new one is verified.
- `WantedBy=default.target` (set in the [Install] section of .container units) is what causes systemd to auto-start the service on boot.

- The backup container uses `mysqldump` — it is a logical backup (SQL text). It is suitable for small-to-medium databases. For large databases, consider volume-level snapshots.
- Always test restore on a **separate volume** before you need it in an emergency.

^ Go to TOC

23.14 Checkpoint

You have completed the capstone when:

- ☐ `systemctl --user status cap-mariadb.service` shows **active (running)** after a reboot.
- ☐ `podman port cap-mariadb` returns nothing (no published host ports).
- ☐ You can produce a `.sql` backup file and restore it to a clean volume successfully.
- ☐ You can upgrade the MariaDB digest, verify the service works, and roll back to the previous digest.
- ☐ You have completed a full password rotation and confirmed login with the new password.

^ Go to TOC

23.15 Quick Quiz

1. Why is it important to test restore, not just backup?
2. What is the operational advantage of deploying by digest rather than by tag?
3. Why must you change the password inside MariaDB *before* updating the Quadlet unit during rotation?
4. What would happen if you deleted the old secret before verifying the new password works?
5. A teammate says “I’ll back up the volume directory directly using `cp -r`.” What problem might arise with this approach for a running database?

^ Go to TOC

23.16 Further Reading

- `podman-secret(1)`: <https://docs.podman.io/en/latest/markdown/podman-secret.1.html>
- Quadlet and Podman systemd integration: <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>
- MariaDB logical backup (`mysqldump`): <https://mariadb.com/kb/en/mysqldump/>
- Adminer project docs: <https://www.adminer.org/>
- systemd timers: <https://www.freedesktop.org/software/systemd/man/latest/systemd.timer.html>
- `loginctl enable-linger`: <https://www.freedesktop.org/software/systemd/man/latest/loginctl.html>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

24 External Secrets Survey (Thorough Intro, Optional Implementation)



24.1 Table of Contents

- Why This Module Exists
- What You Are Optimizing For
- The Decision Tree
- Option 1: systemd Credentials (Host-Native)
- Option 2: SOPS (GitOps-Friendly Encrypted Files)
- Option 3: Vault-Class Secret Managers (Centralized)
- Comparison Table
- What Does Not Change
- Migration Path from Podman Secrets
- Anti-Patterns to Avoid
- Checkpoint
- Quick Quiz
- Further Reading

Podman secrets are a good local-first baseline. But most teams eventually need one or more of:

- encryption-at-rest on the host
- multi-host distribution
- automated rotation
- auditing and access policy

This module teaches the landscape so you can choose an external approach confidently. It is intentionally **not** a single “do this” recipe — external secrets are an architecture and operations decision.

[^ Go to TOC](#)

24.2 Why This Module Exists

When you move from a single dev machine to production, the question is no longer “*how do I use a secret?*” but “*how do I get the secret onto the host, keep it fresh, and revoke it when needed?*”

Podman secrets (`podman secret create`) answer the first question. They do **not** answer:

- How does a brand new host get the secret material?
- How does a secret get rotated across 10 hosts automatically?
- Who approved this secret being read by this service?
- Was there an unauthorized read of this secret last Tuesday?

This module surveys three families of answers. You apply the right one for your context.

[^ Go to TOC](#)

24.3 What You Are Optimizing For

Use this checklist before choosing a system:

Scale and distribution: - How many hosts need the secret? - Does it need to rotate without touching every host manually?

Policy and governance: - Who/what is allowed to read the secret (policy)? - Do you need audit logs of every read? - How do you revoke access?

Availability and bootstrapping: - What happens when the secrets system is down during a deploy or reboot? - How does a brand new host bootstrap its ability to decrypt/fetch? - How do you handle break-glass scenarios (on-call, secrets system itself is broken)?

Credential type: - Static long-lived secret (database password, API key) — any system works. - Dynamic short-lived credential (Vault-leased DB creds) — requires a Vault-class system.

[^ Go to TOC](#)

24.4 The Decision Tree

flowchart TD

```
S(["Which secret manager?"]) --> Q1{"More than<br/>one host?"}
Q1 -->|"No"| Q2{"Need encryption-at-rest<br/>beyond filesystem?"}
Q2 -->|"No"| F1["Podman secrets<br/>(local baseline - this course)"]
Q2 -->|"Yes"| F2["systemd credentials<br/>(encrypted at rest, host-native)"]
Q1 -->|"Yes"| Q3{"Need central policy<br/>and audit? Or dynamic creds?"}
Q3 -->|"No - GitOps / small team"| F3["SOPS<br/>(encrypted in git, age/GPG)"]
Q3 -->|"Yes"| F4["Vault-class<br/>(HashiCorp Vault, AWS Secrets Manager,<br/>etc.)"]
F1 --> OUT["Delivery: file mount<br/>(all systems)"]
F2 --> OUT
F3 --> OUT
F4 --> OUT
```

All paths converge on the same delivery model: **a file** the container reads at runtime. This keeps your application code unchanged regardless of which secrets backend you use.

[^ Go to TOC](#)

24.5 Option 1: systemd Credentials (Host-Native)

24.5.1 What It Is

systemd can provision credentials to services as files at runtime. Credentials can be stored **encrypted at rest** on the host and are decrypted by systemd when the service starts.

24.5.2 How It Fits This Course

The production baseline already uses systemd user services (Quadlet-first). systemd credentials are a natural next step:

- Delivery model: a file at a path like `/run/credentials/<unit>/dbpassword`
- Container consumes: a volume mount or `--secret` pointing to that path
- Application code: unchanged — it still reads a file

24.5.3 Typical Pattern

sequenceDiagram

```
participant H as Host / Ansible
participant S as systemd
participant C as Container
```

```

H->>H: Store encrypted credential<br/>(systemd-creds encrypt)
H->>S: Deploy unit file with<br/>LoadCredentialEncrypted=
S->>S: Decrypt on service start<br/>(uses host TPM or key)
S->>C: Mount credential as file<br/>at /run/credentials/...
C->>C: Read file at startup

```

24.5.4 Operational Notes

- Works entirely within the systemd + Podman stack — no extra services needed.
- Policy is: “who can run this systemd service on this host”.
- You still need a story for distributing the encrypted credential material to new hosts (typically via your config management tool: Ansible, Salt, etc.).
- Encryption uses the host’s TPM or a host-derived key — decryption only works on the provisioned host.

24.5.5 When to Choose It

- Single host or small fleet you manage directly.
- You want minimal moving parts (no extra services to run).
- You already use systemd for everything.

^ Go to TOC

24.6 Option 2: SOPS (GitOps-Friendly Encrypted Files)

24.6.1 What It Is

SOPS (Secrets OPERATIONs) lets you store **encrypted secrets in git**. The secret file is committed to your repository in encrypted form. Decryption happens on the host (or in CI) using an identity: an **age** key, a GPG key, or a cloud KMS key.

24.6.2 Architecture

flowchart LR

```

    subgraph "Git Repository"
        EF["secrets.env.enc<br/>(encrypted, safe to commit)"]
    end
    subgraph "CI / Host"
        KEY["age private key<br/>(never in git)"]
        SOPS["sops --decrypt<br/>secrets.env.enc"]
        FILE["secrets.env<br/>(decrypted, 0600, root-owned)"]
    end
    subgraph "Container"
        C["App reads<br/>/run/secrets/dbpassword"]
    end
    EF -->|"git clone / pull"| SOPS
    KEY -->|"decryption identity"| SOPS
    SOPS --> FILE
    FILE -->|"mount"| C

```

24.6.3 Benefits

- **Change history:** every secret rotation is a git commit with author and timestamp.
- **Reviews:** secret changes can go through pull requests.
- **Bootstrapping:** provisioning a new host means deploying its age key (or granting KMS access).

24.6.4 Tradeoffs

- Rotation is a manual process: edit, encrypt, commit, deploy.
- You must carefully secure decryption keys — if an age key leaks, all secrets encrypted to it are compromised.
- Not suitable for dynamic/leased credentials (see Option 3).

24.6.5 Best-Fit Pattern with Containers

In CI or host provisioning:

```
sops --decrypt secrets/db.yaml | \
  install -m 600 /dev/stdin /run/secrets/dbpassword # decrypt and write with restricted permissions
```

Container reads as usual:

```
podman run --secret dbpassword,type=mount ... # mount as file
```

Never persist decrypted files into images or build contexts.

^ Go to TOC

24.7 Option 3: Vault-Class Secret Managers (Centralized)

24.7.1 What It Is

A centralized secret manager — HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, Infisical, etc. — provides:

- Central policy and auth (who is allowed to read what)
- Audit logs (who read what and when)
- Automated rotation (the manager rotates DB passwords, API keys on a schedule)
- **Dynamic credentials** (leased credentials that expire automatically)

24.7.2 Architecture

flowchart TD

```
subgraph "Secret Manager (e.g. Vault)"
    POL["Policy Engine"]
    AUD["Audit Log"]
    ROT["Auto-Rotation"]
    DB_ROLE["DB secrets engine<br/>(dynamic creds)"]
end
subgraph "Host (Deployment)"
    AUTH["Auth method<br/>(AppRole, OIDC, etc.)"]
    AGENT["Vault Agent / sidecar<br/>OR systemd fetch unit"]
    FILE["Secret file<br/>0600, root-owned"]
end
subgraph "Container"
    APP["App reads<br/>/run/secrets/..."]
end
AUTH -->| "login" | POL
POL --> DB_ROLE
DB_ROLE -->| "lease: user + pass<br/>valid for 1h" | AGENT
AGENT -->| "writes file" | FILE
FILE -->| "volume mount" | APP
DB_ROLE --> AUD
AUTH --> AUD
```

24.7.3 Benefits

- Strong governance: every read is audited, every identity is explicit.
- Short-lived credentials reduce blast radius when a secret leaks.
- Scales to hundreds of services and multiple teams.

24.7.4 Costs

- **Operational overhead:** Vault is another service to run, maintain, back up, and HA-ize.
- **Availability dependency:** if Vault is unreachable at boot, services that need a fresh credential cannot start. You must design for this.
- **Bootstrap complexity:** how does the first container on a fresh host authenticate to Vault?

24.7.5 Recommended Bridge Patterns for Containers

Do not call Vault from inside your app. Instead:

1. **Sidecar/agent writes a file:** a `vault-agent` container shares a volume with your app; it writes the secret to a file; your app reads the file.
2. **systemd fetch unit:** a `systemd ExecStartPre=` step fetches the secret and writes it to a protected `tmpfs` path before the container starts.

Both approaches keep the delivery model consistent: **the container reads a file**.

24.7.6 When to Choose It

- Multiple teams sharing secrets infrastructure.
- Compliance requirements (audit every read, enforce rotation policies).
- Dynamic credentials (DB leases, ephemeral API tokens).

[^] Go to TOC

24.8 Comparison Table

Dimension	Podman Secrets	systemd Credentials	SOPS	Vault-class
Encryption at rest	No (plain on disk)	Yes (TPM/host key)	Yes (age/GPG/KMS)	Yes (transit)
Multi-host	No	Needs config management	Yes (git)	Yes (native)
Audit logs	No	No	Git history	Yes (full)
Auto rotation	No	No	Manual	Yes
Dynamic creds	No	No	No	Yes
Operational cost	Minimal	Low	Low-medium	High
Bootstrap story	Manual	Config management	Key distribution	Auth method
Best fit	Dev / single host	Single host + systemd	Small-medium fleet, GitOps	Org-scale / compliance

[^] Go to TOC

24.9 What Does Not Change

Regardless of which external system you choose, the container interface stays the same:

1. The secret arrives on the host as a **file** (decrypted, 0600, root-owned).
2. The container reads it via a **mount** or Podman **--secret** (which is itself a file mount).
3. The application code reads a file path — it does not know or care which backend provided it.

This is the most important design insight in this module:

Standardize on file delivery. Swap the backend without changing application code.

[^ Go to TOC](#)

24.10 Migration Path from Podman Secrets

If you start with Podman secrets (local-first, this course) and later need to migrate:

flowchart LR

```
A["Podman secrets<br/>(current)"] -->|"same delivery model"| B["systemd credentials<br/>(next step)"]
A -->|"scale to team"| C["SOPS<br/>(GitOps fleet)"]
A -->|"scale to org"| D["Vault-class<br/>(governance + dynamic)"]
B --> E["App reads /run/secrets/... file<br/>(unchanged)"]
C --> E
D --> E
```

Clean migration path: 1. External manager writes/refreshes a **file** to a well-known path. 2. Your **.container** unit mounts that file (or the directory containing it). 3. The service consumes it as it always has.

This avoids rewriting applications that already expect file-based secrets.

[^ Go to TOC](#)

24.11 Anti-Patterns to Avoid

These are the most common secret management mistakes, regardless of which system you use:

Anti-pattern	Why it is dangerous	Correct alternative
Secret in ENV in Containerfile	Baked into image layers, visible in history	Runtime file mount
Secret in -e MY_SECRET=value on CLI	Visible in process list and shell history	Podman --secret , type=mount
Secret in .env committed to git Base64-encoded “secret”	Leaked to anyone with repo access Base64 is encoding, not encryption — trivial to decode	SOPS-encrypted file or git-ignored Actual encryption at rest
App reads \$\$SECRET_ENV_VAR	Env vars can leak via /proc , subprocesses, logs	File read via explicit path
Deleting old secret before verifying new	Locks you out if new secret is wrong	Keep both during rotation window

[^ Go to TOC](#)

24.12 Checkpoint

You should be able to:

- Explain the tradeoffs between systemd credentials, SOPS, and Vault-class secret managers in one paragraph each.
- Describe a bootstrap story for a new host under each approach (how it gets the ability to decrypt/fetch).
- Describe a file-based delivery pattern that keeps application code unchanged across all backends.
- Identify at least three anti-patterns and explain why they are dangerous.
- Choose the right option given a simple scenario: single host, small fleet with GitOps, compliance-heavy org.

^ Go to TOC

24.13 Quick Quiz

1. In one sentence: why is base64 not encryption?
2. What question best distinguishes SOPS-style encrypted files from Vault-style leased secrets?
3. A teammate proposes storing the age private key in the git repo alongside the SOPS-encrypted secrets. What is wrong with this?
4. You are using Vault for dynamic DB credentials. The credential has a 1-hour lease. The container has been running for 2 hours without renewal. What happens, and how should you handle lease renewal?
5. A new engineer says: “I’ll just pass all secrets as environment variables — it’s simpler.” Name three specific ways this can lead to a secret leak.

^ Go to TOC

24.14 Further Reading

- systemd credentials (service-provisioned files): <https://www.freedesktop.org/software/systemd/man/latest/systemd.exec.1>
- **systemd-creds** man page: <https://www.freedesktop.org/software/systemd/man/latest/systemd-creds.html>
- Mozilla SOPS: <https://github.com/getsops/sops>
- age (file encryption tool often used with SOPS): <https://github.com/FiloSottile/age>
- HashiCorp Vault: <https://www.vaultproject.io/>
- Vault Agent for file-based delivery: <https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent>
- Infisical (open-source Vault alternative): <https://infisical.com/>
- Kubernetes Secrets (baseline for comparison): <https://kubernetes.io/docs/concepts/configuration/secret/>

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

25 Cheatsheets

25.1 podman-cli.md

26 Podman CLI Cheat Sheet

26.1 Table of Contents

- Containers
- Images
- Port Publishing
- Networks
- Volumes
- Pods
- Secrets

26.2 Containers

```
podman run --rm <image> <cmd>           # run a one-shot container
podman run -d --name <name> <image>      # run in background with a stable name
podman ps -a                             # list containers (including stopped)
podman logs <name>                       # show container logs
podman exec -it <name> sh                 # run a shell in a running container
podman stop <name>                       # stop a running container
podman rm -f <name>                      # force remove container
```

^ Go to TOC

26.3 Images

```
podman pull <image>                     # download an image
podman build -t localhost/<name>:<tag> .   # build from Containerfile in current dir
podman tag localhost/<name>:<tag> <registry>/<ns>/<name>:<tag> # add another name
podman push <registry>/<ns>/<name>:<tag>   # upload to a registry
podman image history <image>             # show layer history
podman images                            # list local images
podman inspect <image-or-container>      # show JSON metadata
podman rmi <image>                       # remove an image from local storage
```

Cleanup

```
podman image prune          # remove unused images (frees disk)
podman system prune        # remove unused objects (be careful)
podman builder prune       # remove build cache (if supported)
```

^ Go to TOC

26.4 Port Publishing

```
podman run -d -p 8080:80 <image>        # all interfaces
podman run -d -p 127.0.0.1:8080:80 <image> # loopback only
podman run -d -p 8080:80 -p 8443:443 <image> # multiple ports
podman run -d -p 5053:53/udp <image>      # UDP port
podman run -d -p 80 <image>              # random host port
podman port <name>                      # show active port mappings
podman inspect <name> --format '{{json .NetworkSettings.Ports}}' # show raw port mapping JSON
```

^ Go to TOC

26.5 Networks

```
# Create / list / inspect / remove
podman network create <net> # create a user-defined network
podman network create --internal <net> # no outbound access
podman network create --subnet 172.28.0.0/24 --gateway 172.28.0.1 <net> # create a network
podman network ls # list networks
podman network inspect <net> # show network details
podman network rm <net> # remove the network

# Connect / disconnect a running container
podman network connect <net> <name> # attach an existing container
podman network disconnect <net> <name> # detach an existing container

# Which networks is a container on?
podman inspect <name> --format '{{json .NetworkSettings.Networks}}' # show network attachments

# Container IP per network
podman inspect <name> --format '{{range $n,$v := .NetworkSettings.Networks}}{{$.Name}}: {{$v.IPAddress}}'

# Which containers are on a network?
podman network inspect <net> --format '{{range $id,$c := .Containers}}{{$.Name}} {{end}}' # list containers

# Check DNS state of a network
podman network inspect <net> --format '{{.DNSEnabled}}' # true if container name DNS works

# Live DNS lookup from a debug container
podman run --rm --network <net> docker.io/library/alpine:latest sh -lc 'getent hosts <target>' # run a command

# All container IPs (quick overview)
podman ps -q | xargs -I{} podman inspect {} --format '{{.Name}}: {{range .NetworkSettings.Networks}}{{.IP}}'
```

^ Go to TOC

26.6 Volumes

```
podman volume create <vol> # create a volume
podman volume ls # list volumes
podman volume inspect <vol> # inspect a volume
```

^ Go to TOC

26.7 Pods

```
podman pod create --name <pod> -p 8080:80 # create a pod
podman pod ps # list pods
podman pod rm -f <pod> # stop and remove pod + containers
```

^ Go to TOC

26.8 Secrets

```
podman secret create <name> - # create a secret
podman secret ls # list secrets
```

```
podman run --secret <name> <image> # run a container
```

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

26.9 quadlet.md

27 Quadlet Cheat Sheet



27.1 Table of Contents

- Files
- Workflow
- Boot Start

27.2 Files

- put Quadlet files in: `~/.config/containers/systemd/`
- common extensions: `.container`, `.pod`, `.network`, `.volume`

[^ Go to TOC](#)

27.3 Workflow

```
systemctl --user daemon-reload  # regenerate units from files
systemctl --user start <name>.service  # start a user service
systemctl --user status <name>.service  # show service status
journalctl --user -u <name>.service -n 100 --no-pager  # view user-service logs
```

[^ Go to TOC](#)

27.4 Boot Start

```
sudo loginctl enable-linger "$USER"  # allow user services to start at boot
systemctl --user enable <name>.service  # enable the service for your user
```

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

27.5 rootless.md

28 Rootless Cheat Sheet



28.1 Table of Contents

- Key Idea
- Useful Checks
- Common Paths
- Boot Start for systemd User Services

28.2 Key Idea

Rootless Podman runs as your user and uses user namespaces.

[^ Go to TOC](#)

28.3 Useful Checks

```
podman info    # show Podman host configuration
grep "^$USER:" /etc/subuid /etc/subgid # filter output
podman unshare id # run a command inside the user namespace
```

[^ Go to TOC](#)

28.4 Common Paths

- storage: ~/.local/share/containers/
- runtime: /run/user/<uid>/containers/

[^ Go to TOC](#)

28.5 Boot Start for systemd User Services

```
sudo loginctl enable-linger "$USER" # allow user services to start at boot
```

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

28.6 security.md

29 Security Cheat Sheet



29.1 Table of Contents

- Baseline
- Hardening Flags (Examples)
- SELinux (Fedora/RHEL)

29.2 Baseline

- rootless when possible
- non-root user inside container
- do not pass secrets in env vars
- pin images by digest in production

[^ Go to TOC](#)

29.3 Hardening Flags (Examples)

```
podman run --read-only --tmpfs /tmp <image> # read-only root FS + writable temp
```

[^ Go to TOC](#)

29.4 SELinux (Fedora/RHEL)

- bind mount with :Z (private) or :z (shared)

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

29.5 troubleshooting.md

30 Troubleshooting Cheat Sheet



30.1 Table of Contents

- Fast Triage
- systemd/Quadlet
- Network Checks
- Storage Checks

30.2 Fast Triage

```
podman ps -a                # is it running? exit code?
podman logs <name>          # app output / crash reason
podman inspect <name> | less # config: mounts, ports, command, env
```

^ Go to TOC

30.3 systemd/Quadlet

```
systemctl --user status <service>          # systemd view: active/failed
journalctl --user -u <service> -n 200 --no-pager # service logs from journald
```

^ Go to TOC

30.4 Network Checks

- verify ports: -p host:container
- verify container name DNS on the network

^ Go to TOC

30.5 Storage Checks

- volume mounted where the app expects
- permissions for the container user
- on Fedora/RHEL: use :Z for private bind mounts

^ Go to TOC

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

31 Appendix

31.1 ASSESSMENTS.md

32 Assessments



These assessments focus on practical skills.

32.1 Table of Contents

- Module Checkpoints
- Practical Exam A (Mid-Course)
- Practical Exam B (Final)

32.2 Module Checkpoints

Each module ends with a checkpoint. Treat it as “must be able to do without notes”.

[^ Go to TOC](#)

32.3 Practical Exam A (Mid-Course)

Scenario:

- You are given a container that exits immediately.

Requirements:

- Determine why it exits.
- Fix it without rebuilding the image.
- Provide a short runbook: commands used, what you observed, final fix.

Rubric:

- Uses `podman ps -a`, `podman logs`, `podman inspect` effectively.
- Fix is minimal and reproducible.
- No secrets printed.

[^ Go to TOC](#)

32.4 Practical Exam B (Final)

Scenario:

- You are given a two-service stack: web + db.
- The stack must survive reboot.

Requirements:

- Use Quadlet (systemd user service) to run both services.
- Use a named volume for state.
- Use a secret mounted as a file (not env vars).
- DB is private; only web is published.
- Provide backup + restore steps.

Rubric:

- Rootless and reboot-safe (linger configured if required).
- Correct storage and networking.
- Secrets handled safely.
- Clear, testable runbook.

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

32.5 GLOSSARY.md

33 Glossary



33.1 Table of Contents

- [Terms](#)

33.2 Terms

- container: a running (or stopped) instance of an image with its own writable layer and runtime config
- image: an OCI artifact composed of layers + config; used as a template for containers
- registry: a service that stores and distributes images
- tag: a movable name that points to an image (example: `:latest`)
- digest: a content-addressed identifier for an image (example: `@sha256:...`)
- rootless: running Podman as a normal user using user namespaces
- Quadlet: systemd integration that generates service units from `.container/.pod/.volume/.network` files
- SELinux: a Linux MAC system; on Fedora/RHEL it can affect mounts and container permissions
- namespace: a Linux kernel isolation feature; containers commonly use PID, mount, and network namespaces
- cgroups v2: the kernel resource-control mechanism used for CPU/memory limits and systemd integration
- user namespace: a namespace that remaps UIDs/GIDs; enables rootless containers to have “root” inside without host root
- subuid/subgid: per-user UID/GID ranges used for user namespace mappings (rootless)
- writable layer: the per-container filesystem layer on top of the image; it is not durable persistence
- volume: Podman-managed persistent storage intended for stateful data
- bind mount: a host path mounted into a container; often used for config and source code
- network (user-defined): a named bridge network with DNS enabled for container name resolution
- aardvark-dns: Podman’s embedded DNS service for user-defined networks
- netavark: Podman’s networking stack used to configure networks and DNS (modern Podman)
- slirp4netns: a user-mode networking helper commonly used for rootless containers
- pasta: a newer user-mode networking helper often used for rootless containers
- healthcheck: an image or runtime-defined command that reports container health (used by tooling/systemd policies)
- linger: systemd feature that allows user services to run at boot without an interactive login

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0

33.3 FAQ.md

34 FAQ / Gotchas



This file collects common failure modes and the fastest fixes.

34.1 Table of Contents

- Rootless: “permission denied” publishing port 80
- Container name DNS does not work
- SELinux: bind mounts fail with permission denied (Fedora/RHEL)
- Local registry lab fails with TLS/HTTPS errors
- HEALTHCHECK missing after build
- `exec format error`
- Quadlet service does not exist after adding a file

34.2 Rootless: “permission denied” publishing port 80

Rootless users typically cannot bind ports below 1024.

Fix options:

- Use a high port (recommended for labs): `-p 8080:80`
- If you own the system, lower the unprivileged port start:

```
sudo sysctl -w net.ipv4.ip_unprivileged_port_start=80 # allow low ports for rootless (system-wide)
```

See: `modules/06-networking.md`

^ Go to TOC

34.3 Container name DNS does not work

Container DNS names work on user-defined networks (DNS enabled), not on the default network.

Fix:

```
podman network create appnet # create a network
podman run --network appnet ... # run a container
```

See: `modules/06-networking.md`

^ Go to TOC

34.4 SELinux: bind mounts fail with permission denied (Fedora/RHEL)

If SELinux is enforcing, bind mounts may require labels.

Fix patterns:

- Private mount: `-v ./dir:/mnt:Z`
- Shared mount: `-v ./dir:/mnt:z`

See: `modules/05-storage.md`

^ Go to TOC

34.5 Local registry lab fails with TLS/HTTPS errors

Some Podman configs treat an HTTP registry as insecure and require explicit configuration.

For the learning lab only:

```
podman push --tls-verify=false localhost:5000/alpine:course # push an image to a registry
```

See: [modules/03-images-registries.md](#)

[^ Go to TOC](#)

34.6 HEALTHCHECK missing after build

Podman warns that HEALTHCHECK metadata is not stored in the OCI image format.

Fix options:

- Build in docker format when you need the healthcheck stored in the image:

```
podman build --format docker -t localhost/myapp:1 . # build an image
```

- Or define a runtime healthcheck using `podman run --health-*` flags.

See: [modules/08-building-images.md](#), [examples/build/hello-bun/README.md](#)

[^ Go to TOC](#)

34.7 exec format error

This almost always means an architecture mismatch (built for amd64, running on arm64, or vice versa).

Fix:

- Build on the target architecture, or
- Use a platform-aware build workflow (advanced topic)

See: [modules/08-building-images.md](#), [examples/build/hello-go/README.md](#)

[^ Go to TOC](#)

34.8 Quadlet service does not exist after adding a file

Quadlet units are generated at reload time.

Fix:

```
systemctl --user daemon-reload # regenerate units from files  
systemctl --user status <unit> # show service status
```

See: [modules/11-quadlet.md](#)

[^ Go to TOC](#)

© 2026 UncleJS — Licensed under CC BY-NC-SA 4.0